

IE437 Data-Driven Decision Making and Control

2023 Spring Semester

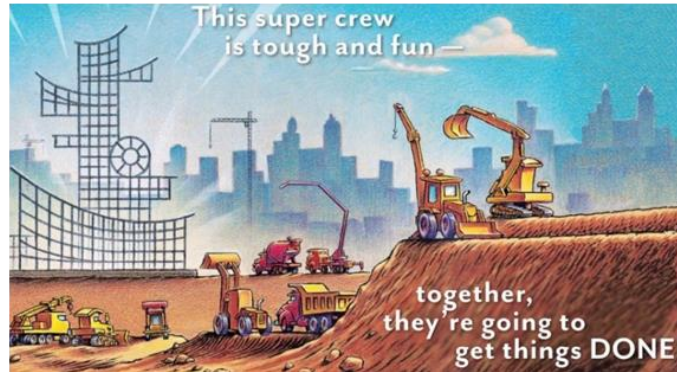
0. Introduction

Jinkyoo Park
Associate Professor
Industrial and Systems Engineering Department



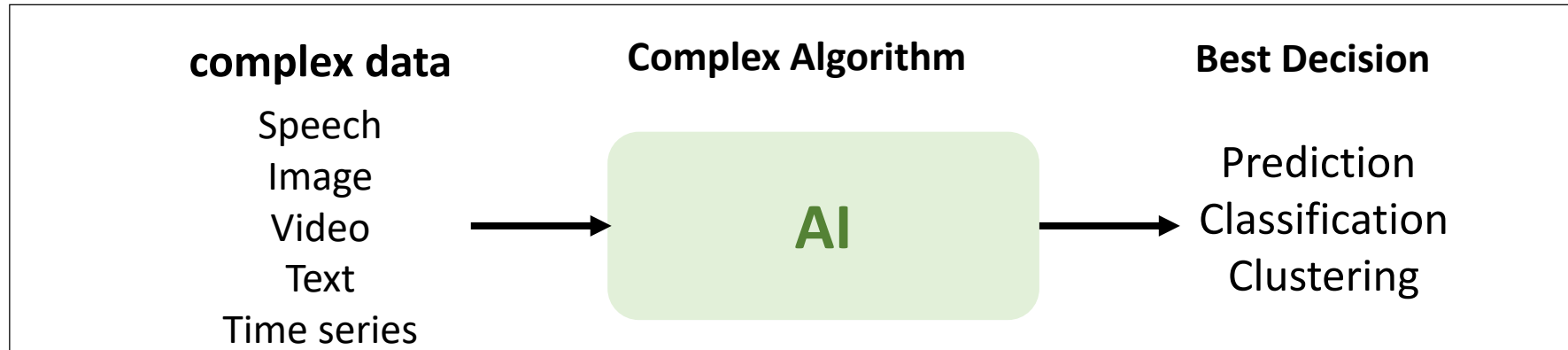
Decision-Centric Artificial Intelligence

How we can solve complex problems using AI?



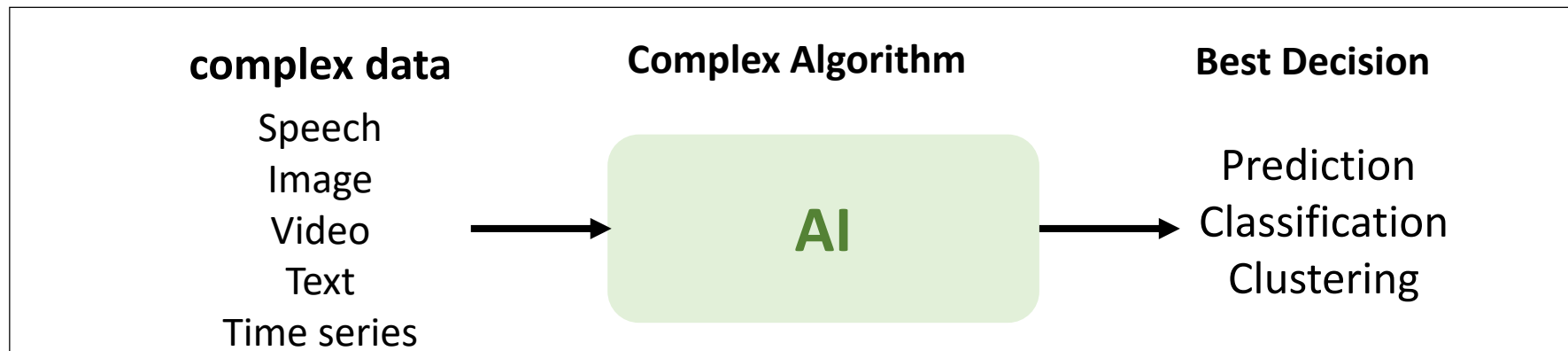
- Complex problems cannot be solved with data alone.
- Artificial intelligence algorithms alone are not sufficient to solve problems.

Difference between AI and Decision-Making

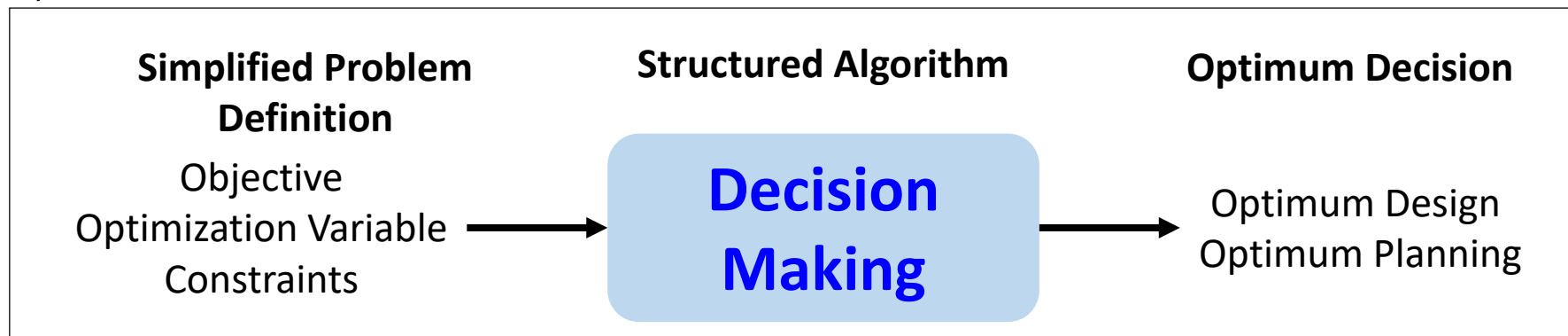


Artificial intelligence utilizes complex data to efficiently perform well-structured tasks such as prediction, classification, and differentiation.

Difference between AI and Decision-Making

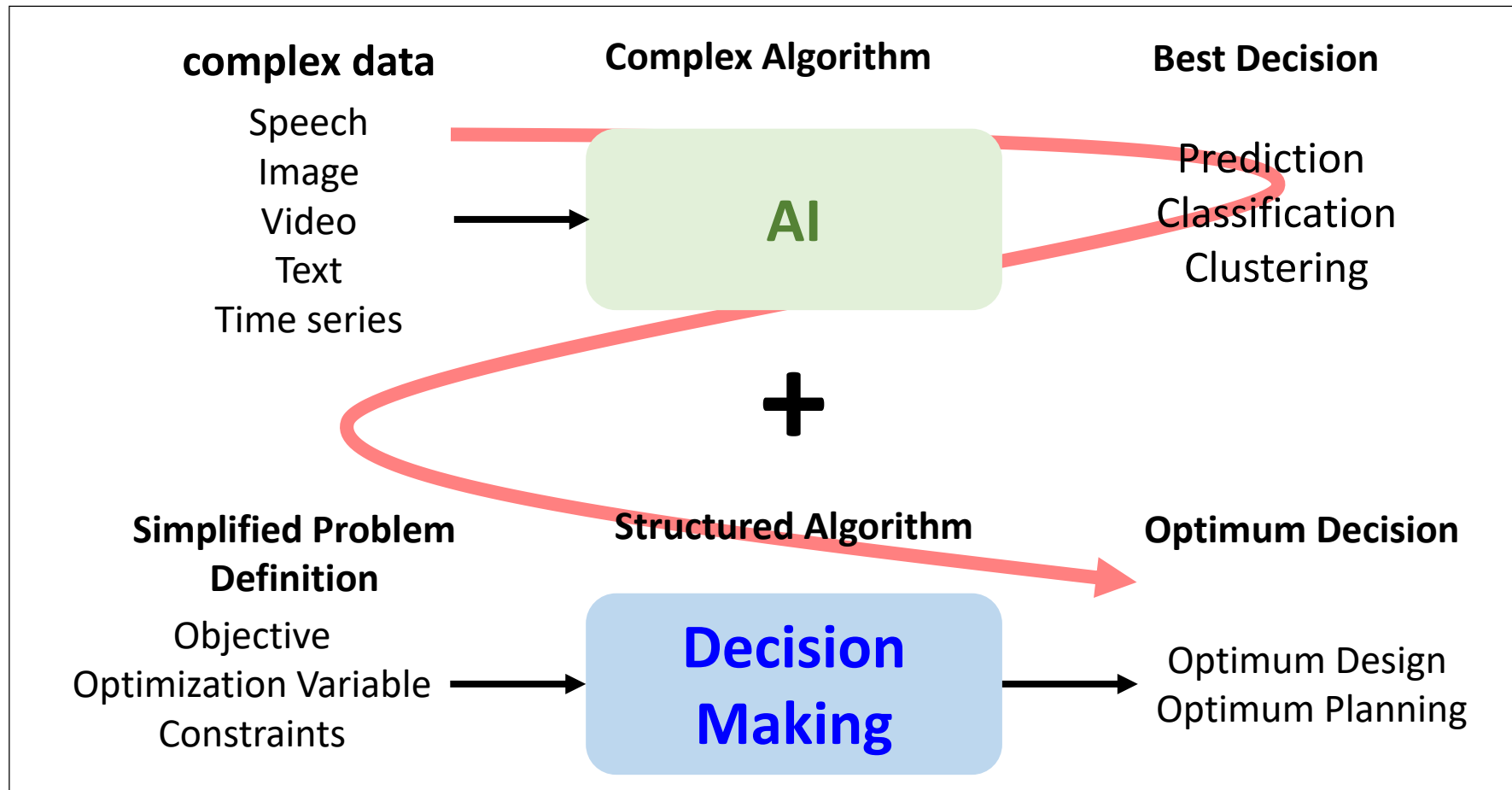


Artificial intelligence utilizes complex data to efficiently perform well-structured tasks such as prediction, classification, and differentiation.



Decision-making is the process of formalizing a problem mathematically and deriving an optimal solution to perform complex tasks.

Objective of Decision Centric AI



Utilizing artificial intelligence and decision-making (optimization) techniques, high-dimensional decision-making can be efficiently derived from complex data.

1. Motivation

Engineering is all about decision makings

- Machine Learning
- **Artificial Intelligence**
- Optimization
- Optimum Control
- Planning
- Markov Decision Process
- Influential Diagram
- Decision Tree
- Dynamic Control
- **Game Theory**
- Search
- Stochastic Programming
- Dynamic programming
- Reinforcement Learning
- Bandit problem
- $\vdots$



What are ***the differences*** in these decision-making strategies

What are ***the common aspects*** in these decision-making strategies?

1. Motivation

Main Topics in Decision Makings

What type of decision making framework will be used?

- Single stage or multi stages
- Single decision maker or many decision makers
- Model based or model-free

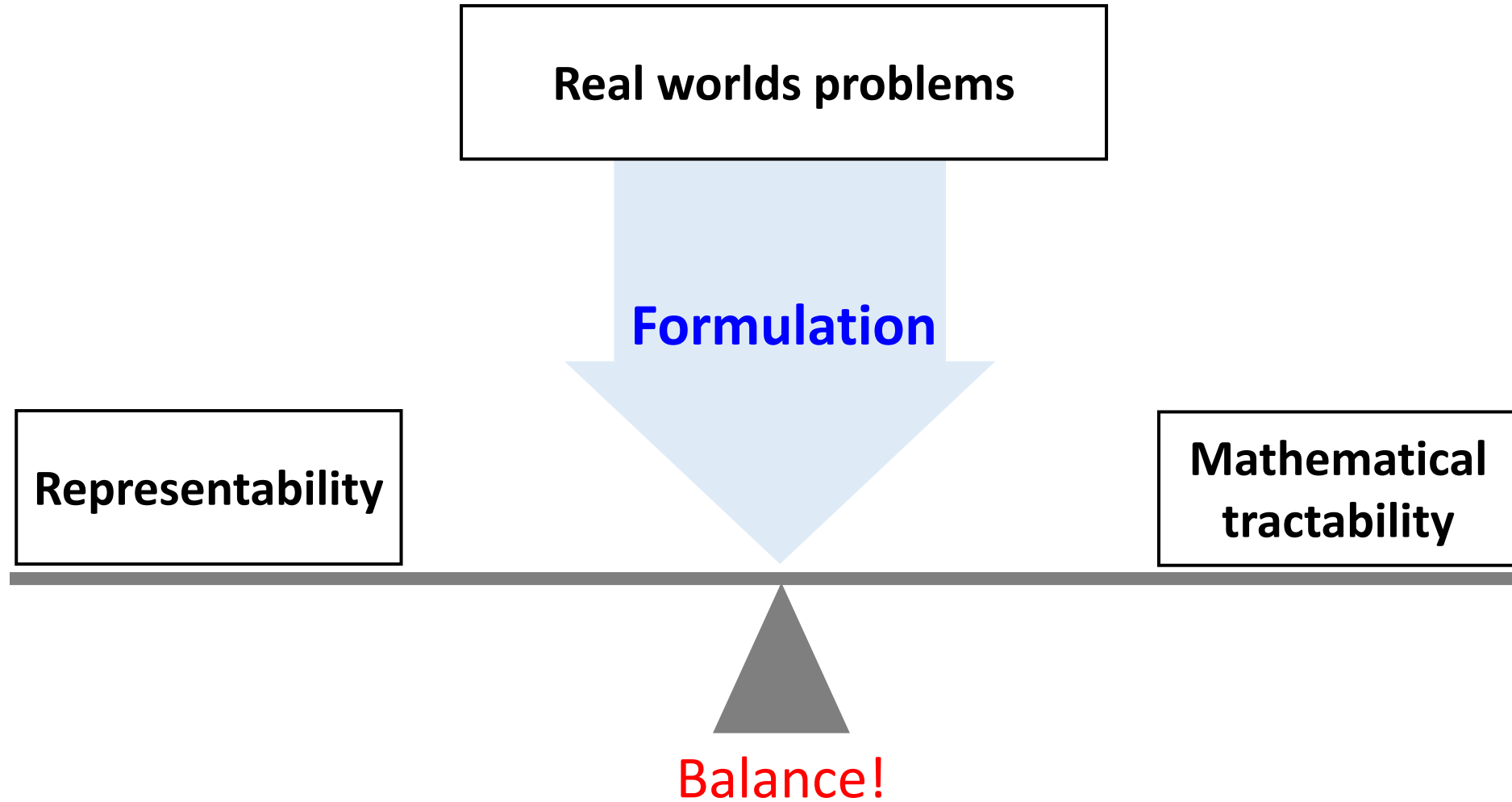
“**Decision makings** under **uncertainties**”

How to model uncertainties?

- **Epistemic Uncertainty** (systemic uncertainty) :
Uncertainty arising through lack of knowledge
 - Model uncertainty
 - State uncertainty
- **Aleatoric uncertainty** (statistical uncertainty):
Uncertainty arising through an underlying stochastic system

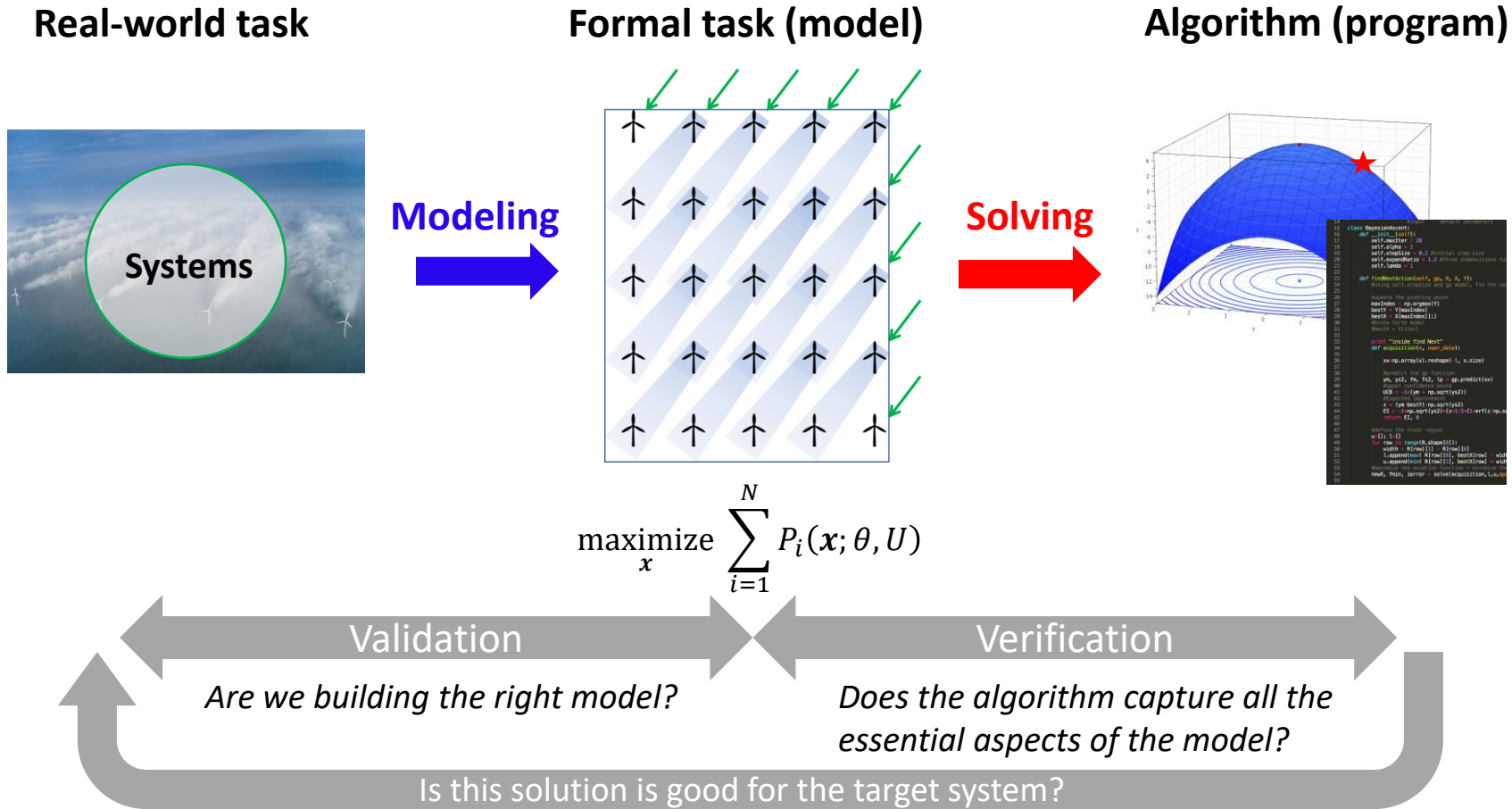
1. Motivation

How to Solve Complex Problems?



1. Motivation

How to Solve Complex Problems?

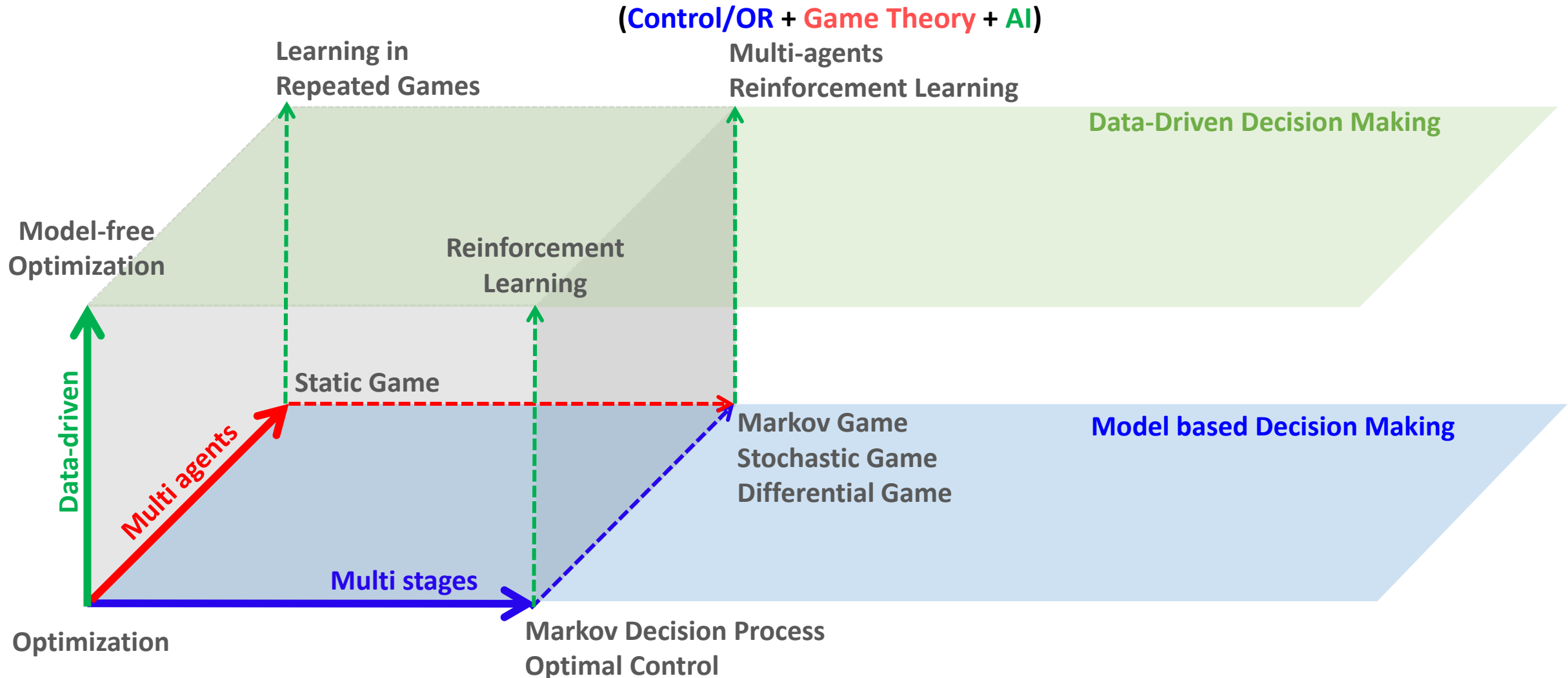


Data can help model more realistically and derive more accurate solution!

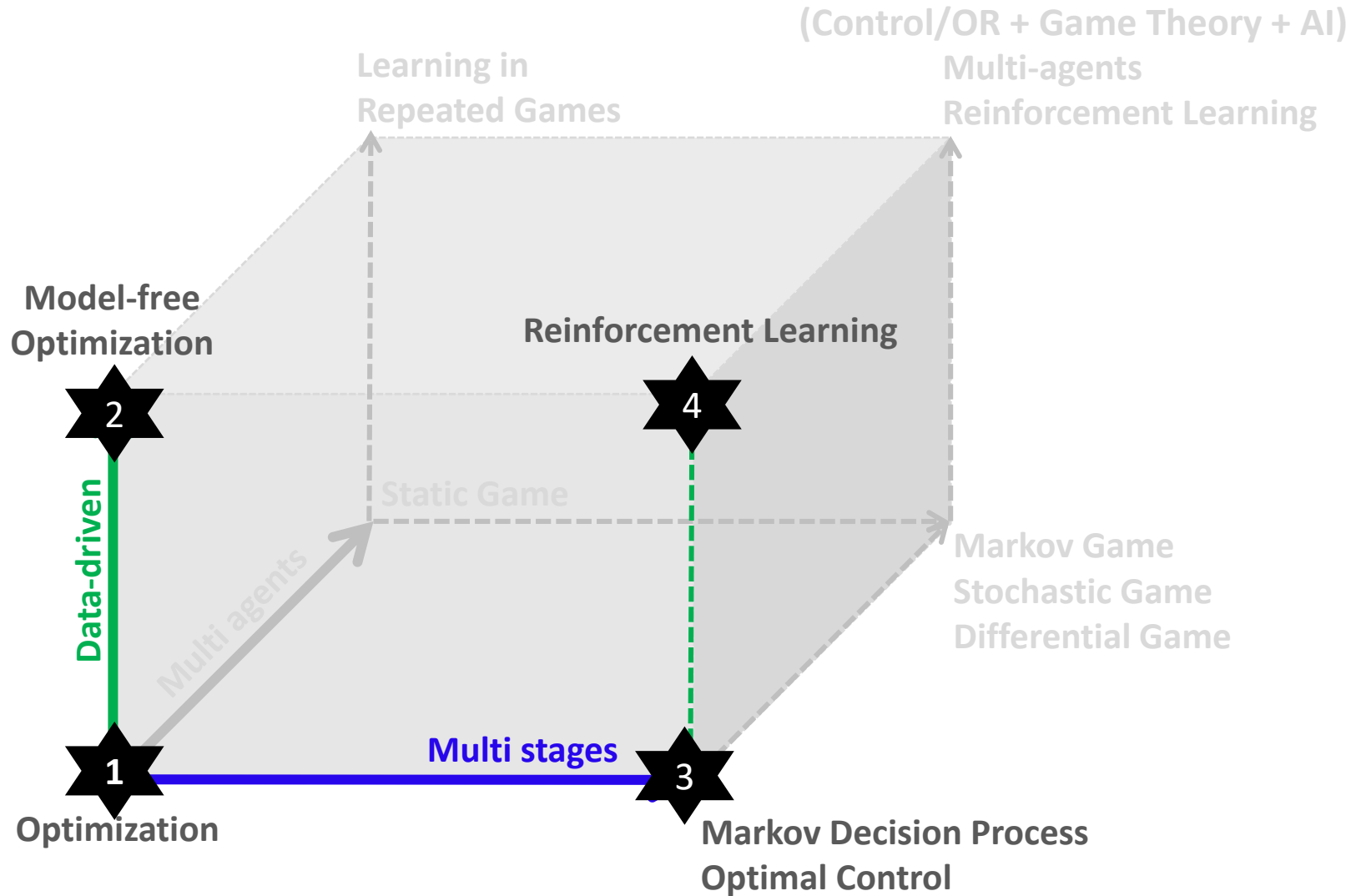
Categorization of Decision Making



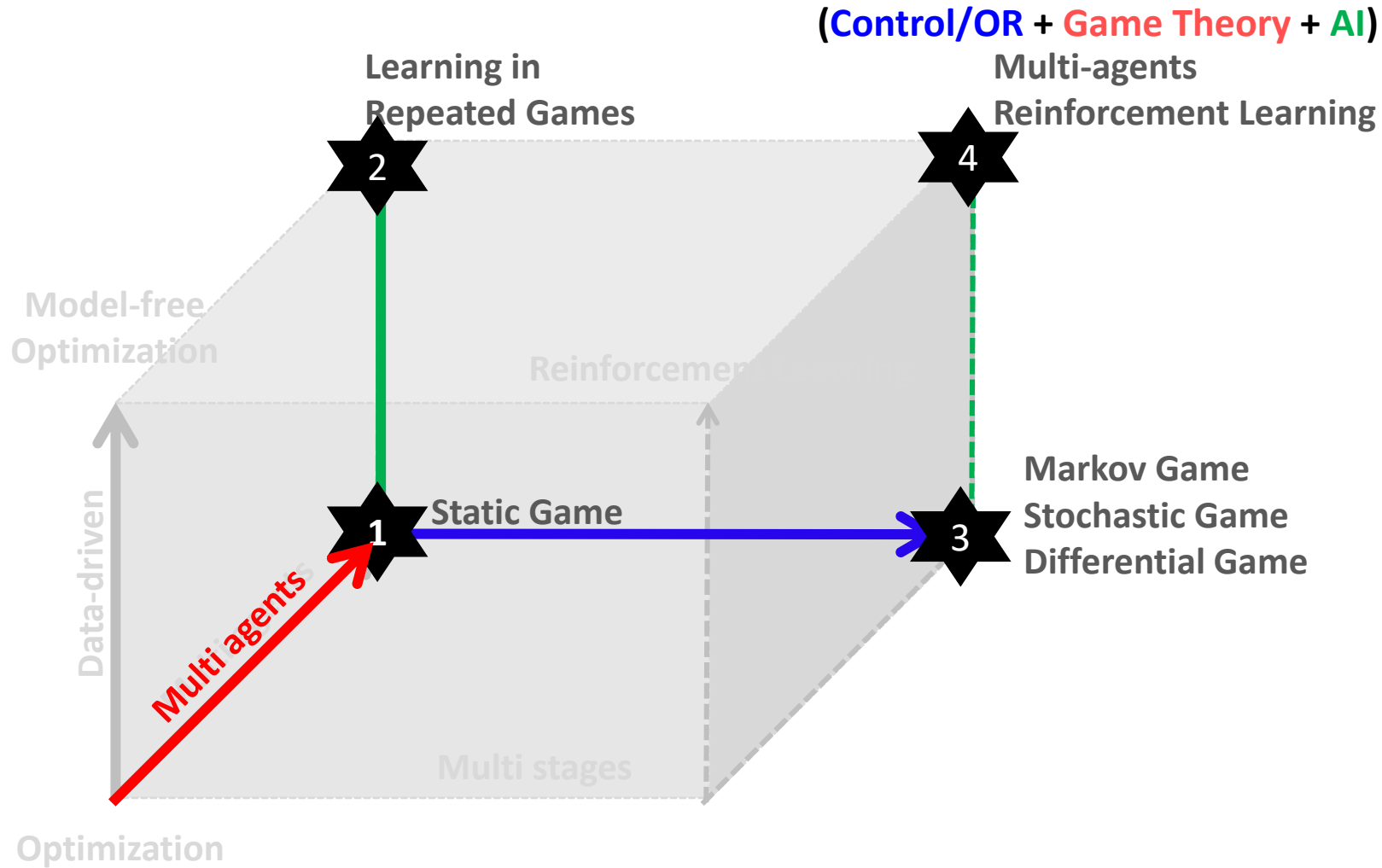
Extension to Data-Driven Decision Making



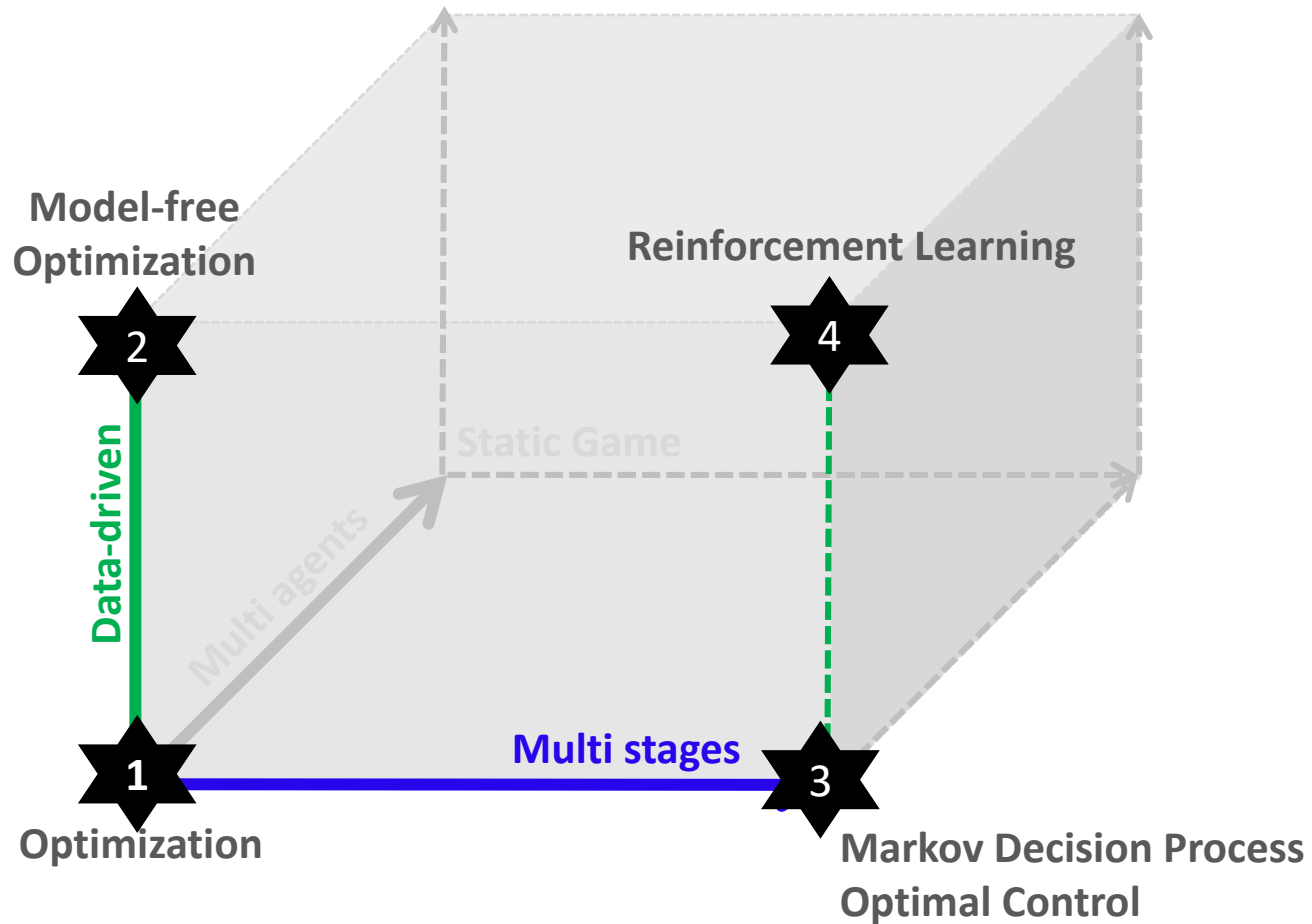
IE437 Data-Driven Decision Making and Control



IE579 Game Theory and Multi-Agent Reinforcement Learning



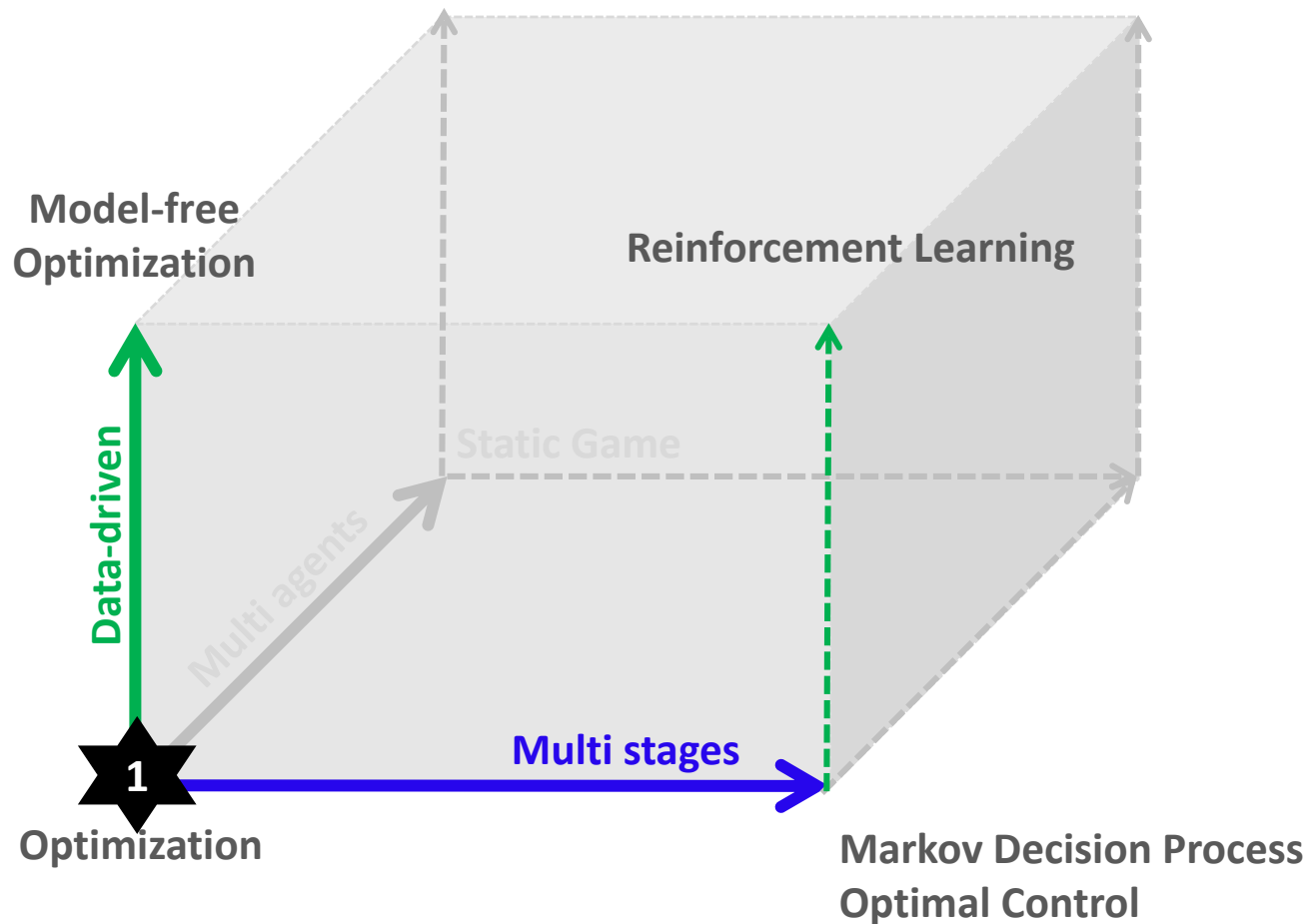
IE437 Data-Driven Decision Making and Control



Fundamentals on Optimization and Bayesian Statistics

“Decision makings under **uncertainties**”

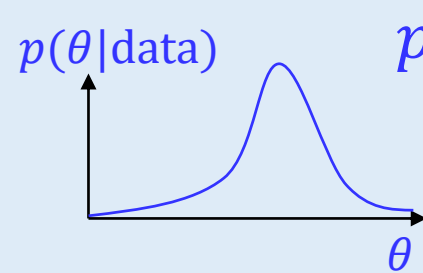
- Optimization Problem Modeling
- Fundamentals on Bayesian Statistics



Bayesian Statistics

The posterior :

The probability on the parameter θ given the evidence (data)



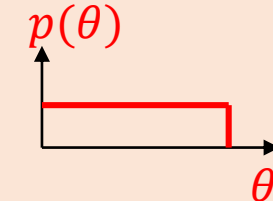
$$p(\theta|\text{data}) = \frac{p(\text{data}|\theta)p(\theta)}{p(\text{data})}$$

The evidence

The probability of getting this evidence given the parameter

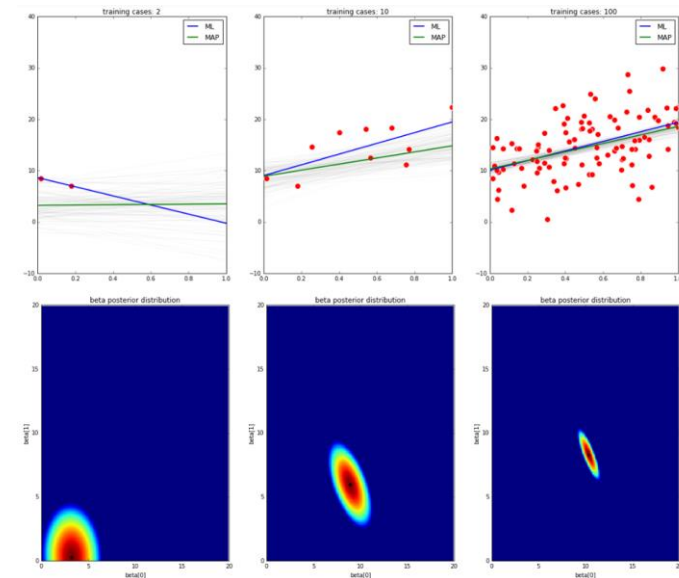
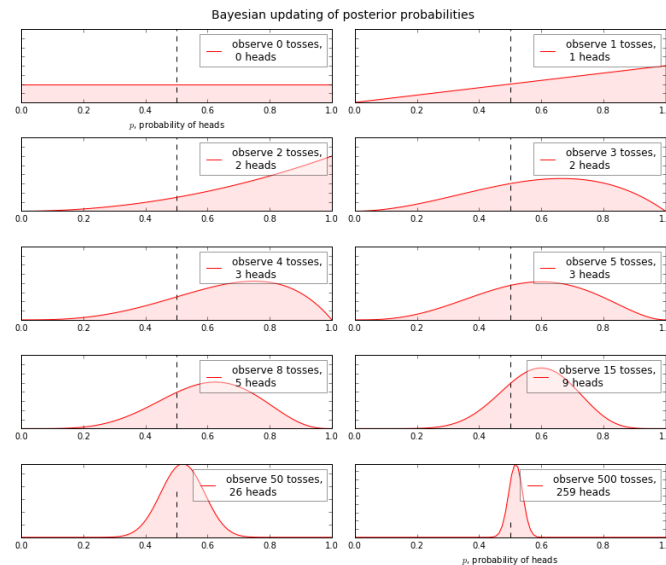
The Prior

The belief (uncertainty) about the parameter θ



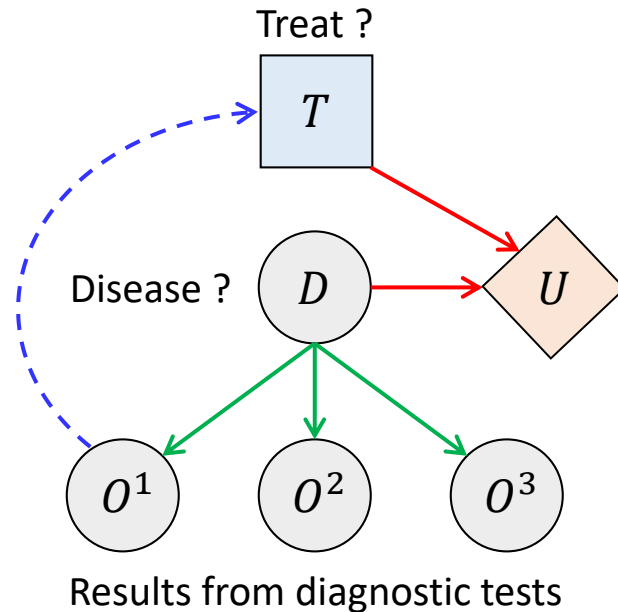
The Marginal probability

The probability of the evidence, Probability of data over all possibilities



Bayesian Statistics

Bayesian Network + Decision node + Utility node = Decision network (Influential Diagram)



T	D	$U(T, D)$
0	0	0
0	1	-10
1	0	-1
1	1	-1

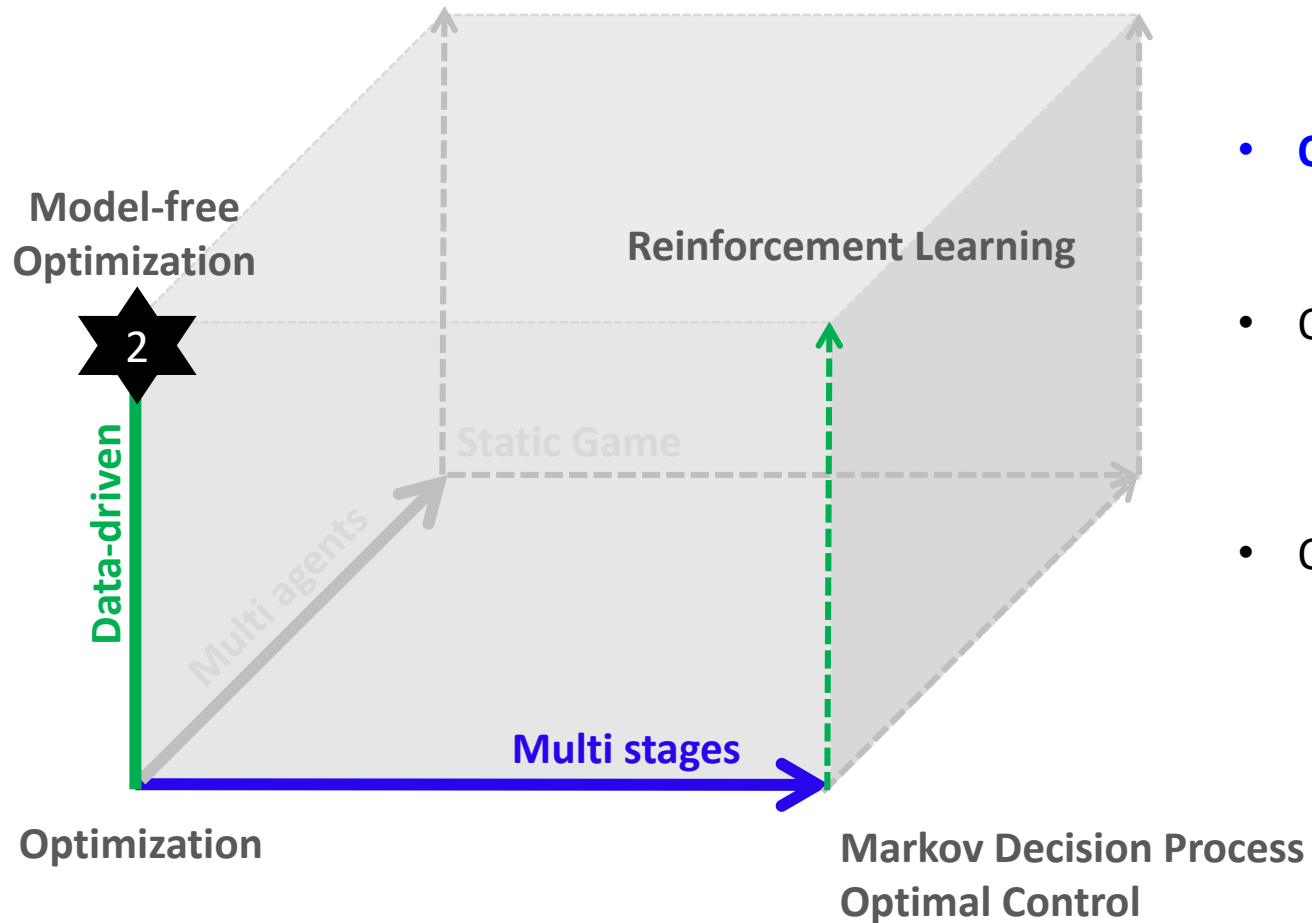
→ Conditional edge
→ Functional edge
-----> Information edge
(often omitted)

○ A chance node corresponds to a random variable

□ A decision node corresponds to each decision to be made

◇ A utility node corresponds to an additive utility component

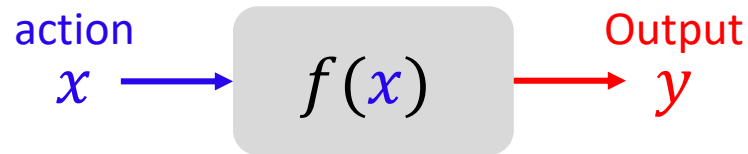
Data-Driven Static Decision Making



- **Online Optimization:**
 - **Bayesian Optimization**
- Offline Optimization
 - Surrogate Model Based
 - Generative Model Based
- Offline Meta Optimization + Online Adaptation

Bayesian Optimization

- **Bayesian Optimization (BO)** is a method to maximize (or minimize) a target value using measurement data from the **unknown target system**.
- BO is composed of three iterative steps:
 - (1) Learning : construct a probabilistic model function for a target system
 - (2) Optimization : select the next trial input to improve a target
 - (3) Observation : measure the output of a target system



- Policy π maps all the history to new action:

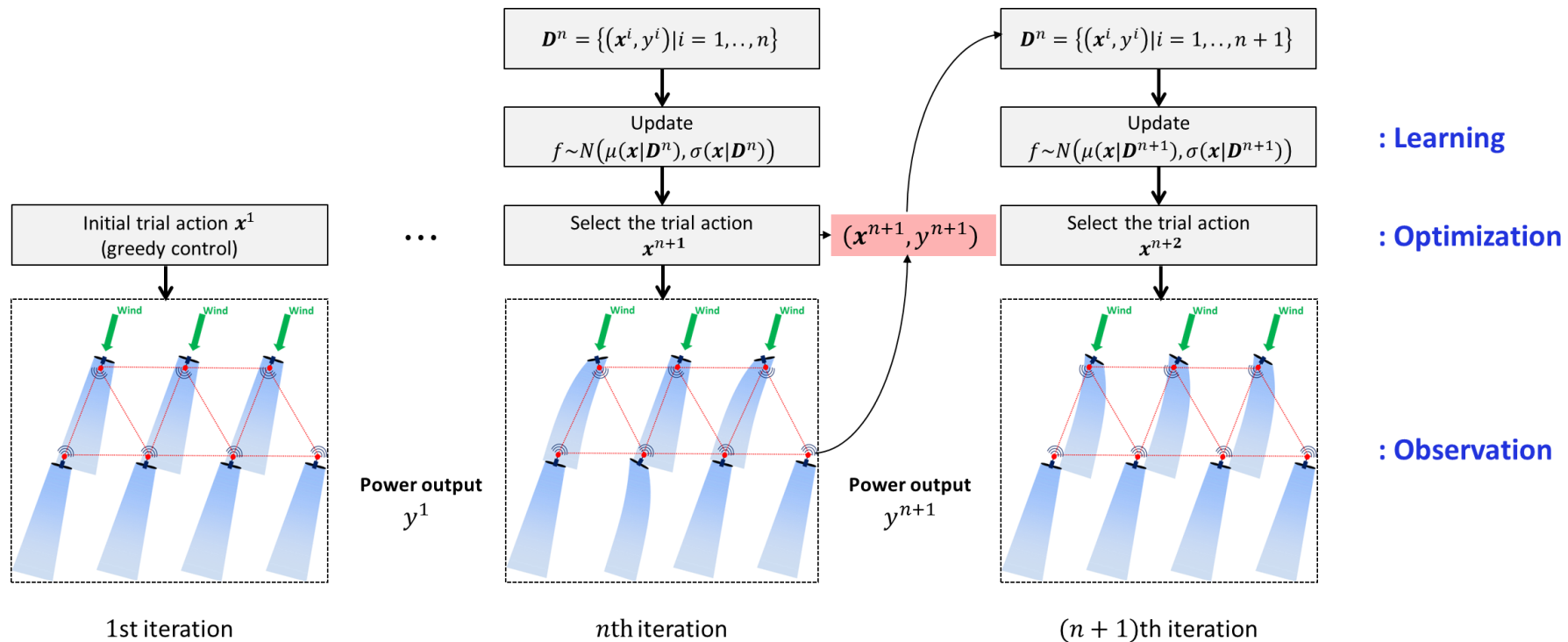
$$\pi: [(x^1, y^1), (x^2, y^2), \dots, (x^{n-1}, y^{n-1})] \rightarrow x^n$$

- Find the optimal policy π^* that maximizes $E[\sum_{t=1}^T y^t]$ or $E[y^T]$

$$x^* = \underset{x}{\operatorname{argmax}} f(x) \equiv E[y^T]$$

Bayesian Optimization

- **Bayesian Optimization (BO)** is a method to maximize (or minimize) a target value using measurement data from the **unknown target system**.



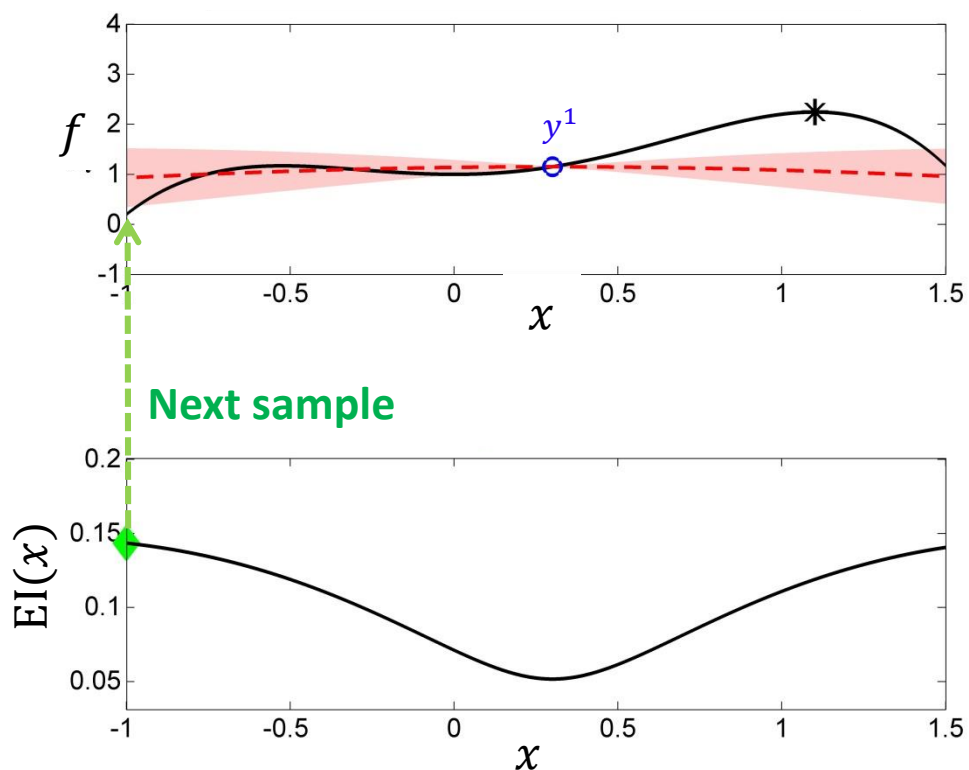
Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\underset{x}{\text{maximize}} \ f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon$$

$$\text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2)$$

$$y^1 \begin{bmatrix} 1.15 \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathbf{D}^{1:n}) = N(\mu(x|\mathbf{D}^{1:n}), \sigma^2(x|\mathbf{D}^{1:n}))$$

$$\mu(x|\mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x|\mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\mathbf{x}^2 = \arg \max_x \text{EI}(x) \triangleq \text{E}[\max\{0, f - f^{\max}\} | \mathbf{D}^{1:n}]$$

$$\mathbf{x}^2 = -1.00$$

Query the function value

$$y^2 = f(\mathbf{x}^2) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

$$y^2 = 0.18$$

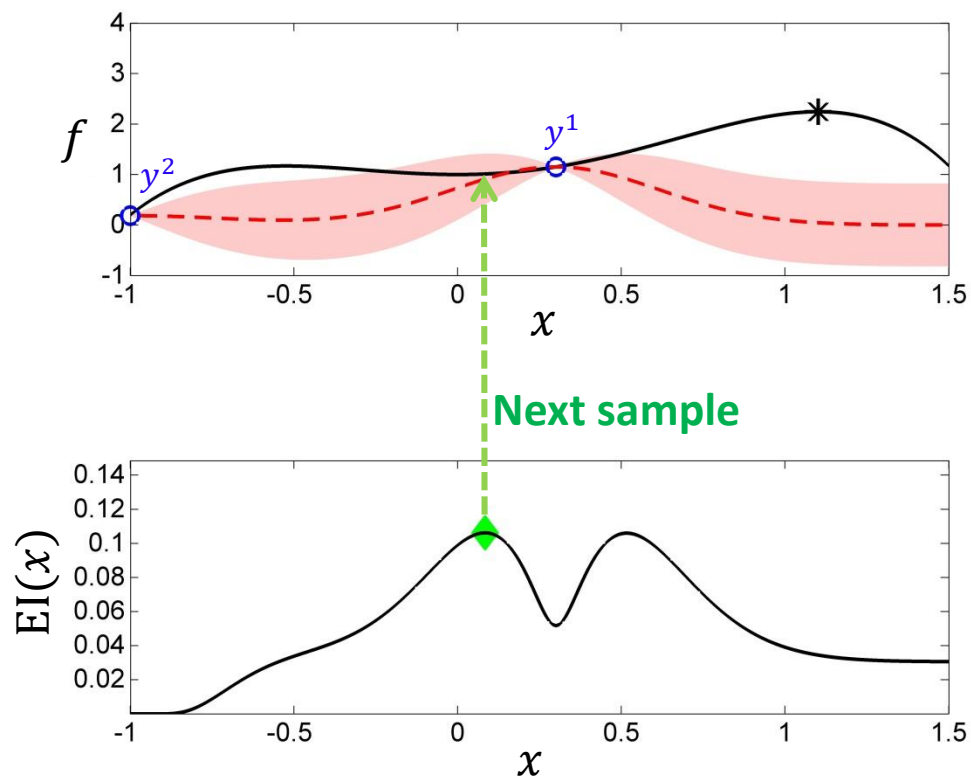
Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\underset{x}{\text{maximize}} \ f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon$$

$$\text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2)$$

$$\begin{matrix} y^1 \\ y^2 \\ f \end{matrix} \begin{bmatrix} 1.15 \\ 0.18 \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathbf{D}^{1:n}) = N(\mu(x|\mathbf{D}^{1:n}), \sigma^2(x|\mathbf{D}^{1:n}))$$

$$\mu(x|\mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x|\mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\mathbf{x}^3 = \arg \max_x \text{EI}(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathbf{D}^{1:n}]$$

$$\mathbf{x}^3 = 0.03$$

Query the function value

$$\mathbf{x}^3 = f(\mathbf{x}^3) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

$$\mathbf{x}^3 = 1.02$$

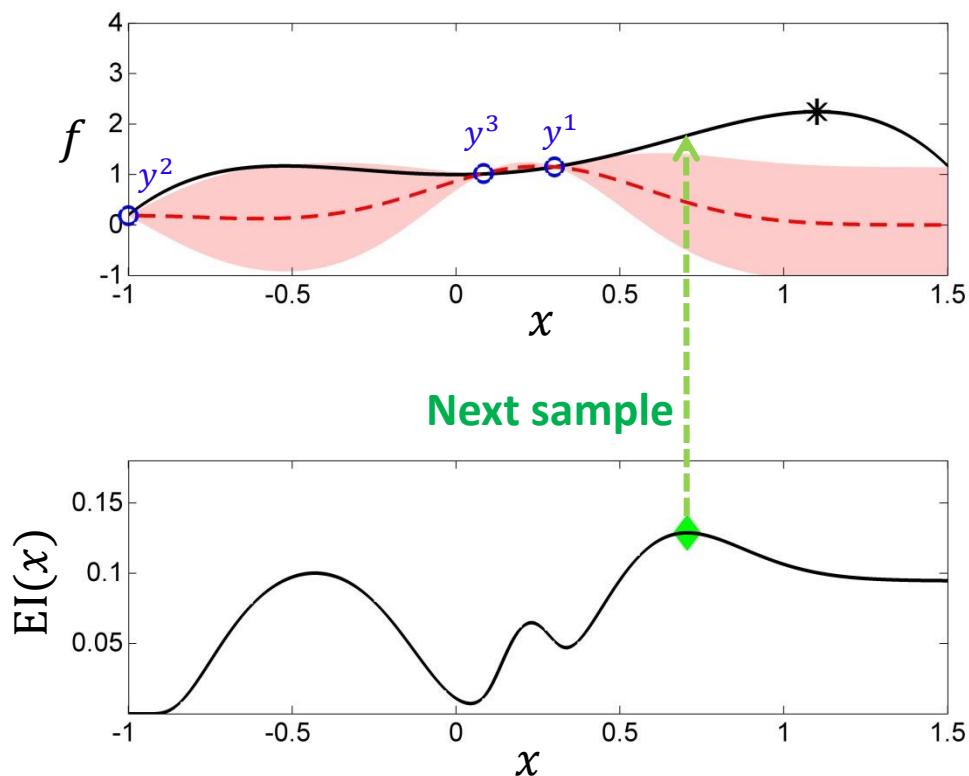
Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\underset{x}{\text{maximize}} \ f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon$$

$$\text{subject to} \ -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2)$$

$$\begin{matrix} y^1 \\ y^2 \\ y^3 \\ f \end{matrix} \begin{bmatrix} 1.15 \\ 0.18 \\ 1.02 \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f | \mathbf{D}^{1:n}) = N(\mu(x | \mathbf{D}^{1:n}), \sigma^2(x | \mathbf{D}^{1:n}))$$

$$\mu(x | \mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x | \mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\mathbf{x}^4 = \arg \max_x EI(x) \triangleq E[\max\{0, f - f^{max}\} | \mathbf{D}^{1:n}]$$

$$\mathbf{x}^4 = 0.70$$

Query the function value

$$y^4 = f(\mathbf{x}^4) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

$$y^4 = 1.76$$

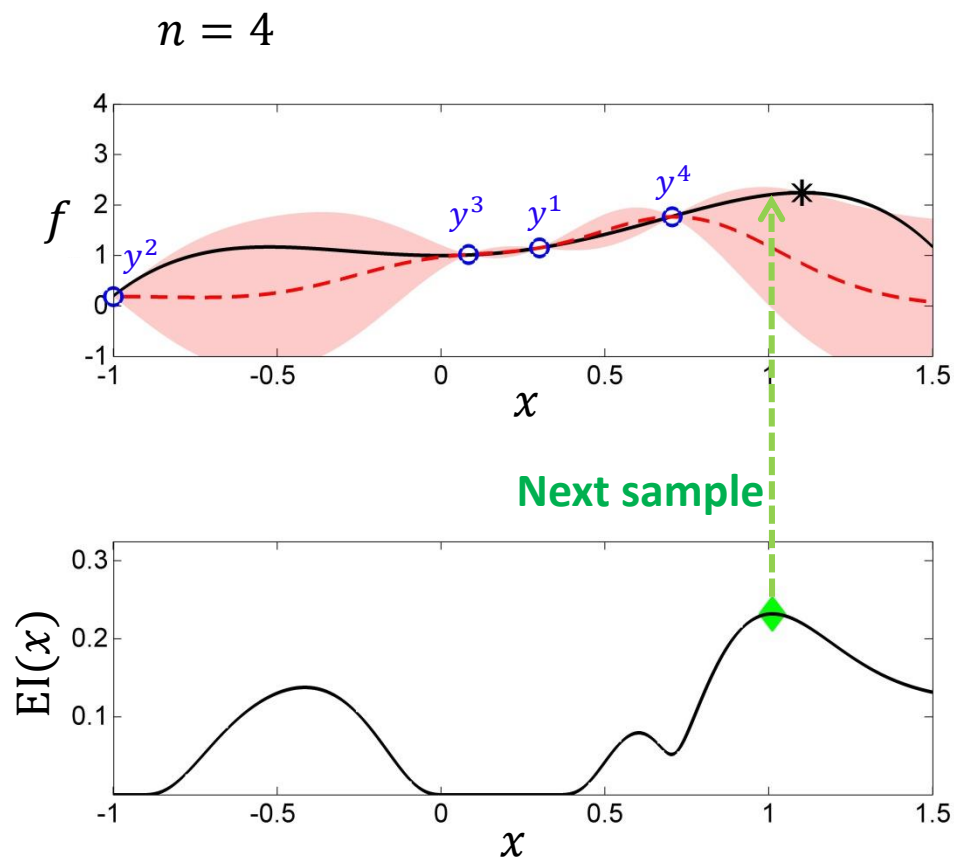
Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon$$

$$\text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2)$$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathbf{D}^{1:n}) = N(\mu(x|\mathbf{D}^{1:n}), \sigma^2(x|\mathbf{D}^{1:n}))$$

$$\mu(x|\mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x|\mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\mathbf{x}^5 = \arg \max_x EI(\mathbf{x}) \triangleq E[\max\{0, f - f^{max}\} | \mathbf{D}^{1:n}]$$

$$\mathbf{x}^5 = 1.01$$

Query the function value

$$y^5 = f(\mathbf{x}^5) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

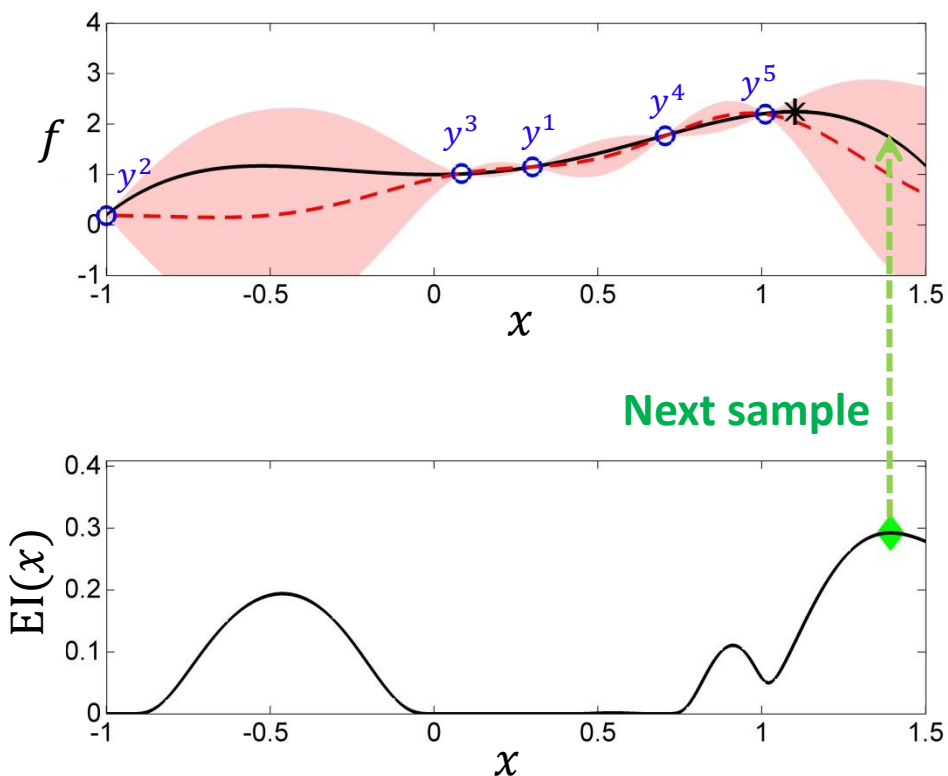
$$y^5 = 2.19$$

Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathbf{D}^{1:n}) = N(\mu(x|\mathbf{D}^{1:n}), \sigma^2(x|\mathbf{D}^{1:n}))$$

$$\mu(x|\mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x|\mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\begin{aligned} \mathbf{x}^6 &= \arg \max_x EI(x) \triangleq E[\max\{0, f - f^{max}\} | \mathbf{D}^{1:n}] \\ \mathbf{x}^6 &= 1.39 \end{aligned}$$

Query the function value

$$y^6 = f(\mathbf{x}^6) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

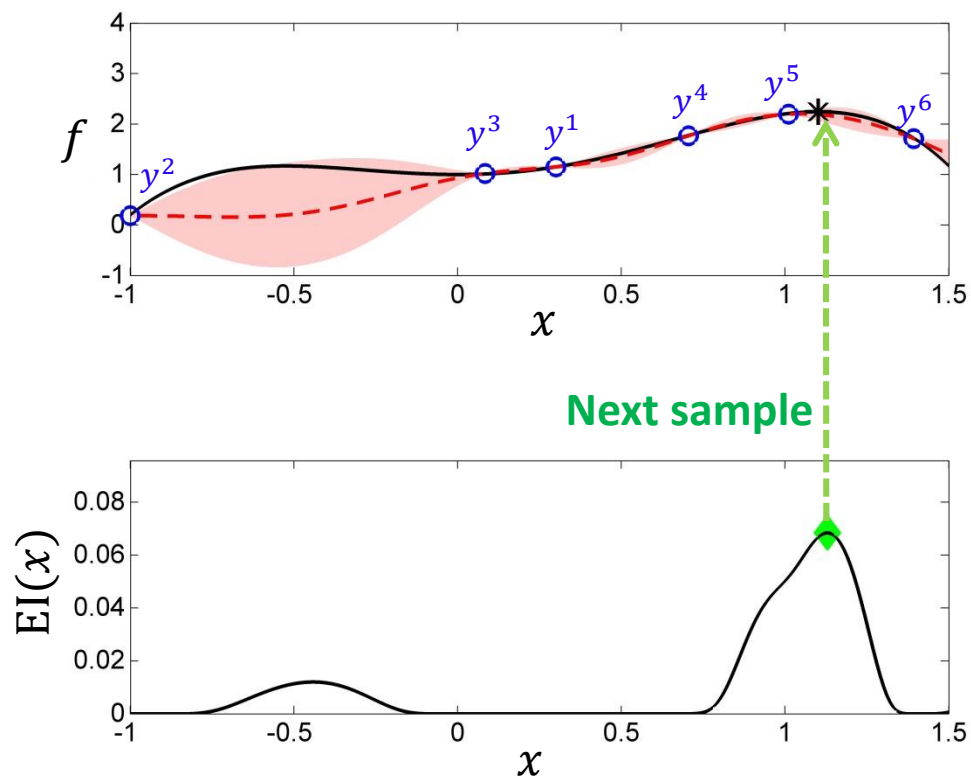
$$y^6 = 1.70$$

Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \\ y^6 \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathbf{D}^{1:n}) = N(\mu(x|\mathbf{D}^{1:n}), \sigma^2(x|\mathbf{D}^{1:n}))$$

$$\mu(x|\mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x|\mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\begin{aligned} \mathbf{x}^7 &= \arg \max_x EI(x) \triangleq E[\max\{0, f - f^{max}\} | \mathbf{D}^{1:n}] \\ \mathbf{x}^7 &= 1.13 \end{aligned}$$

Query the function value

$$y^7 = f(\mathbf{x}^7) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

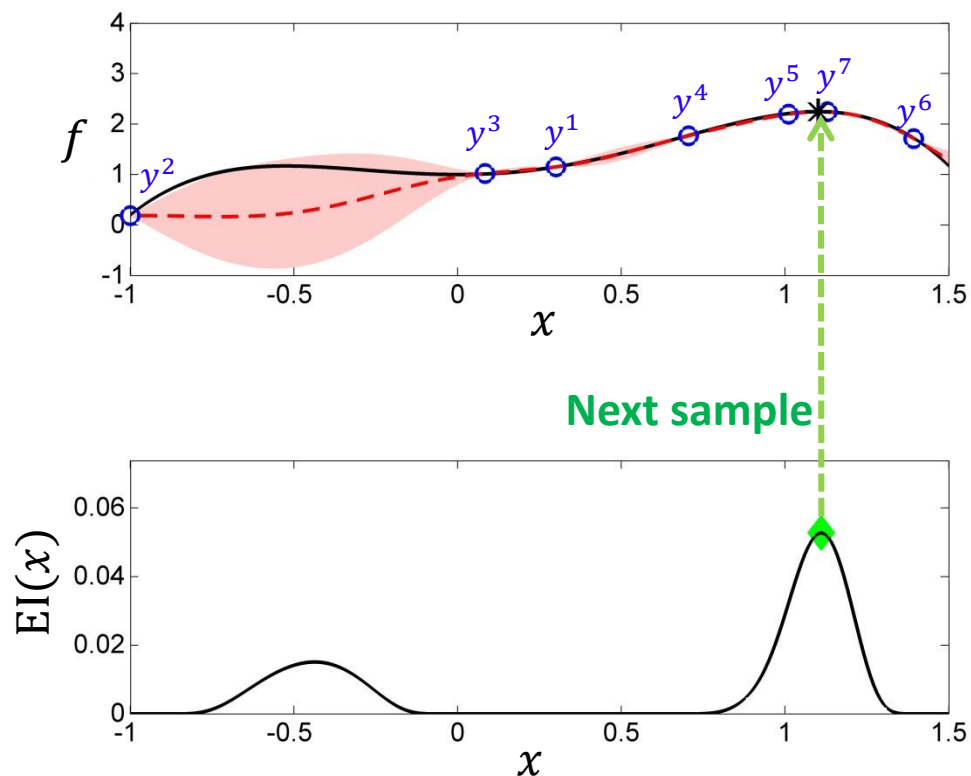
$$y^7 = 2.24$$

Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \\ y^6 \\ y^7 \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathbf{D}^{1:n}) = N(\mu(x|\mathbf{D}^{1:n}), \sigma^2(x|\mathbf{D}^{1:n}))$$

$$\mu(x|\mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x|\mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\mathbf{x}^8 = \arg \max_x EI(x) \triangleq E[\max\{0, f - f^{max}\} | \mathbf{D}^{1:n}]$$

$$\mathbf{x}^8 = 1.11$$

Query the function value

$$y^8 = f(\mathbf{x}^8) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

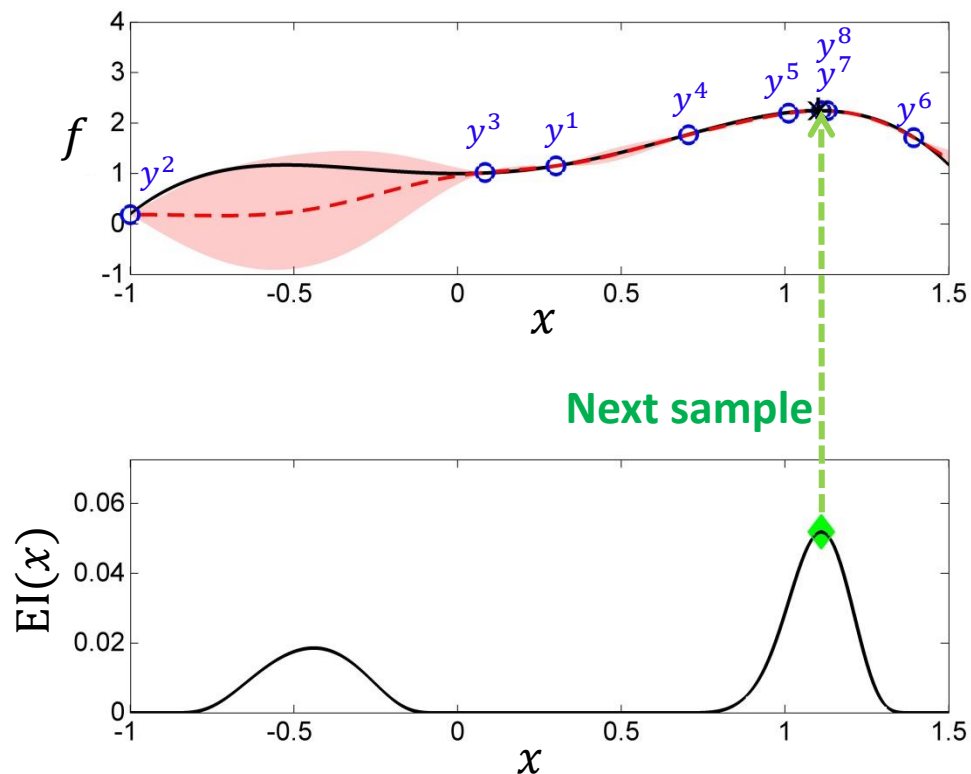
$$y^8 = 2.24$$

Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \\ y^6 \\ y^7 \\ y^8 \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathbf{D}^{1:n}) = N(\mu(x|\mathbf{D}^{1:n}), \sigma^2(x|\mathbf{D}^{1:n}))$$

$$\mu(x|\mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x|\mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\mathbf{x}^9 = \arg \max_x EI(x) \triangleq E[\max\{0, f - f^{max}\} | \mathbf{D}^{1:n}]$$

$$\mathbf{x}^9 = 1.11$$

Query the function value

$$y^9 = f(\mathbf{x}^9) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

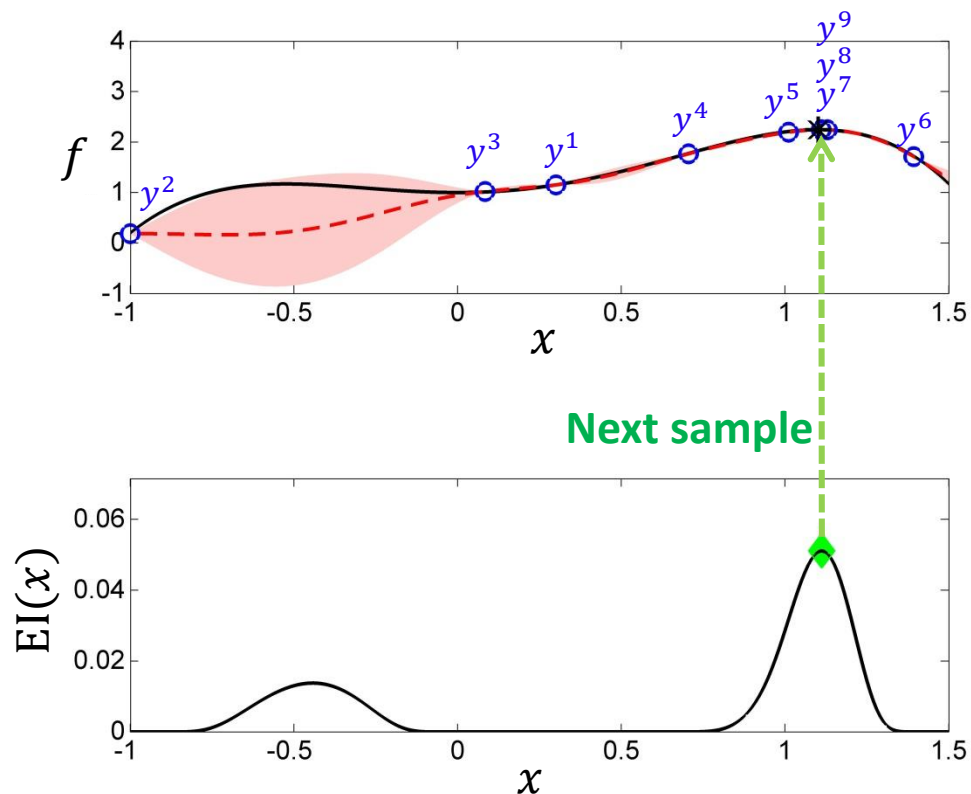
$$y^9 = 2.24$$

Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \\ y^6 \\ y^7 \\ y^8 \\ y^9 \\ f \end{bmatrix} \sim N \left(\begin{bmatrix} \mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathbf{D}^{1:n}) = N(\mu(x|\mathbf{D}^{1:n}), \sigma^2(x|\mathbf{D}^{1:n}))$$

$$\mu(x|\mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x|\mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\mathbf{x}^{10} = \arg \max_z EI(x) \triangleq E[\max\{0, f - f^{max}\} | \mathbf{D}^{1:n}]$$

$$\mathbf{x}^{10} = 1.11$$

Query the function value

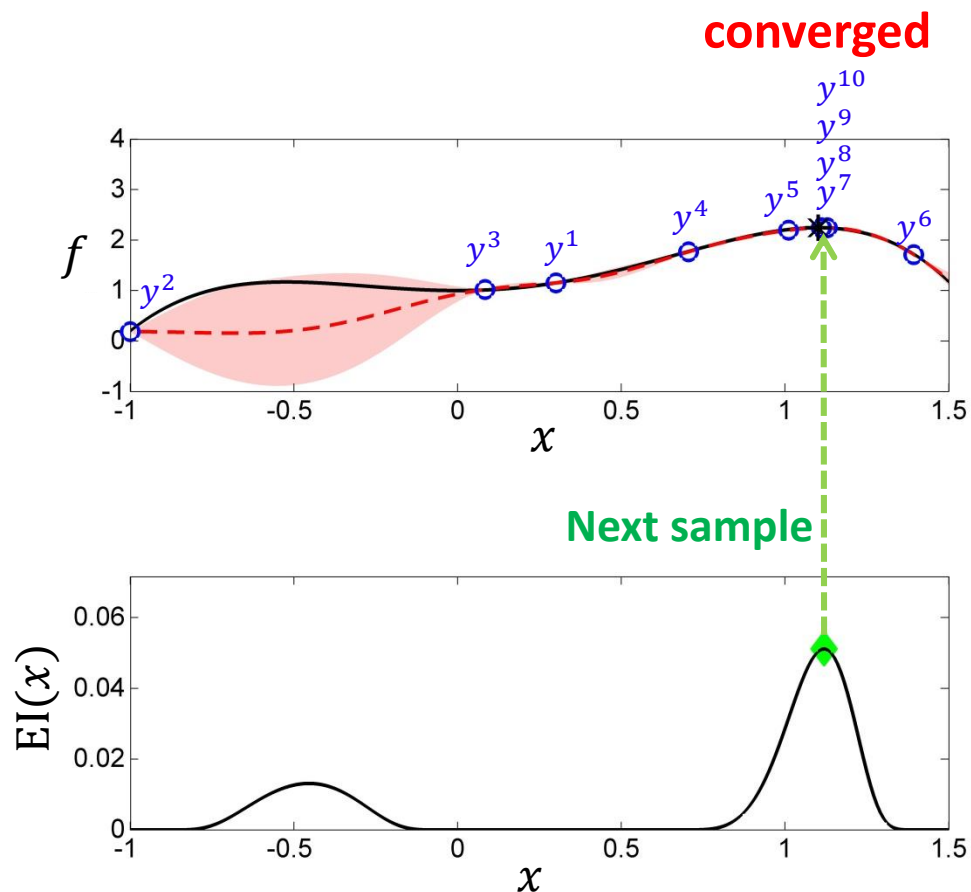
$$y^{10} = f(\mathbf{x}^{10}) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

$$y^{10} = 2.24$$

Bayesian Optimization with Expected Improvement Function Example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2) \end{aligned}$$



$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \\ y^6 \\ y^7 \\ y^8 \\ y^9 \\ y^{10} \\ f \end{bmatrix} \sim N \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{k} \end{bmatrix}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$

Construct probability distribution on the unknown function value

$$p(f | \mathbf{D}^{1:n}) = N(\mu(x | \mathbf{D}^{1:n}), \sigma^2(x | \mathbf{D}^{1:n}))$$

$$\mu(x | \mathbf{D}^{1:n}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}^{1:n}$$

$$\sigma^2(x | \mathbf{D}^{1:n}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$\mathbf{x}^{11} = \arg \max_x EI(x) \triangleq E[\max\{0, f - f^{max}\} | \mathbf{D}^{1:n}]$$

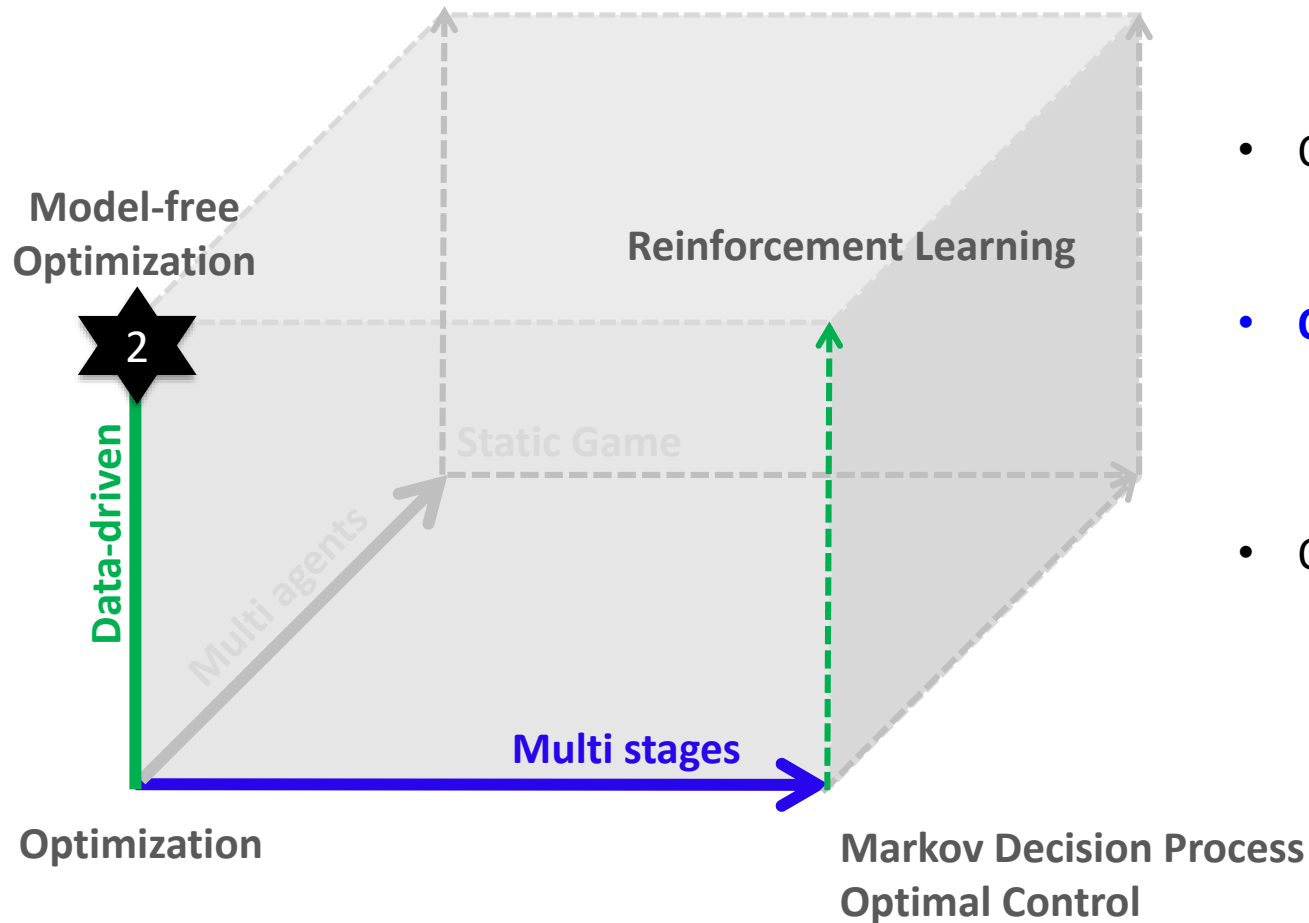
$$\mathbf{x}^{11} = 1.11$$

Query the function value

$$y^{11} = f(\mathbf{x}^{11}) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

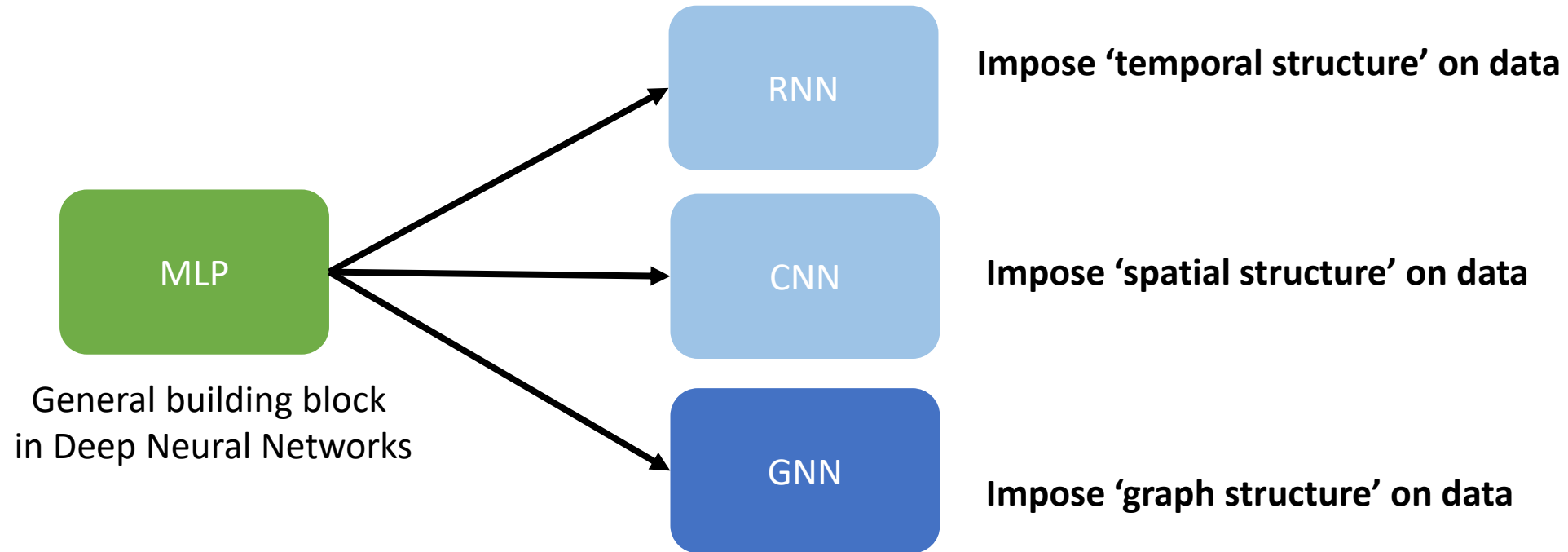
$$y^{11} = 2.24$$

Data-Driven Static Decision Making



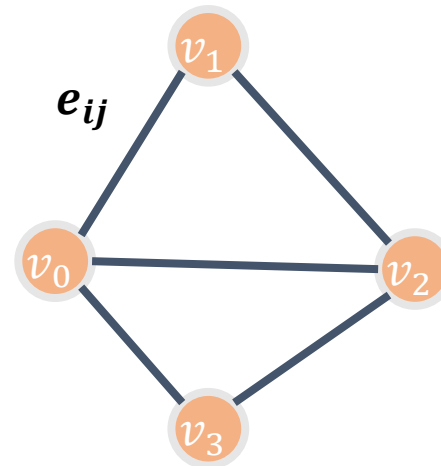
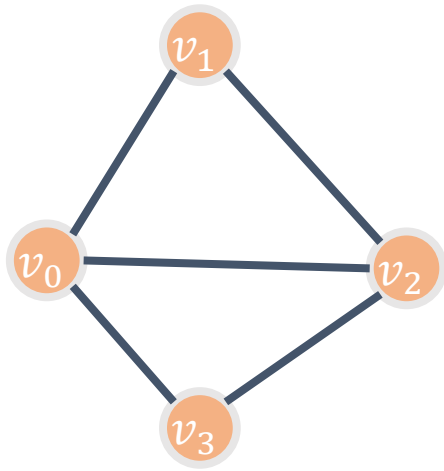
- Online Optimization:
 - Bayesian Optimization
- **Offline Optimization**
 - **Surrogate Model Based**
 - Generative Model Based
- Offline Meta Optimization + Online Adaptation

Graph Neural networks (GNN) – A general way for imposing inductive bias



- GNN learns **pairwise interactions among nodes** as we did for CNN and RNN.
- The learned interactions (usually represented as outputs of NN) can be used to perform inference tasks on differently composed graphs. e.g.) More/less nodes or edges than training cases.

Add edge feature on graph



Graph $\mathcal{G} = \langle \mathcal{V}, A \rangle$

$\mathcal{V}$: Set of node features

A : Adjacency matrix

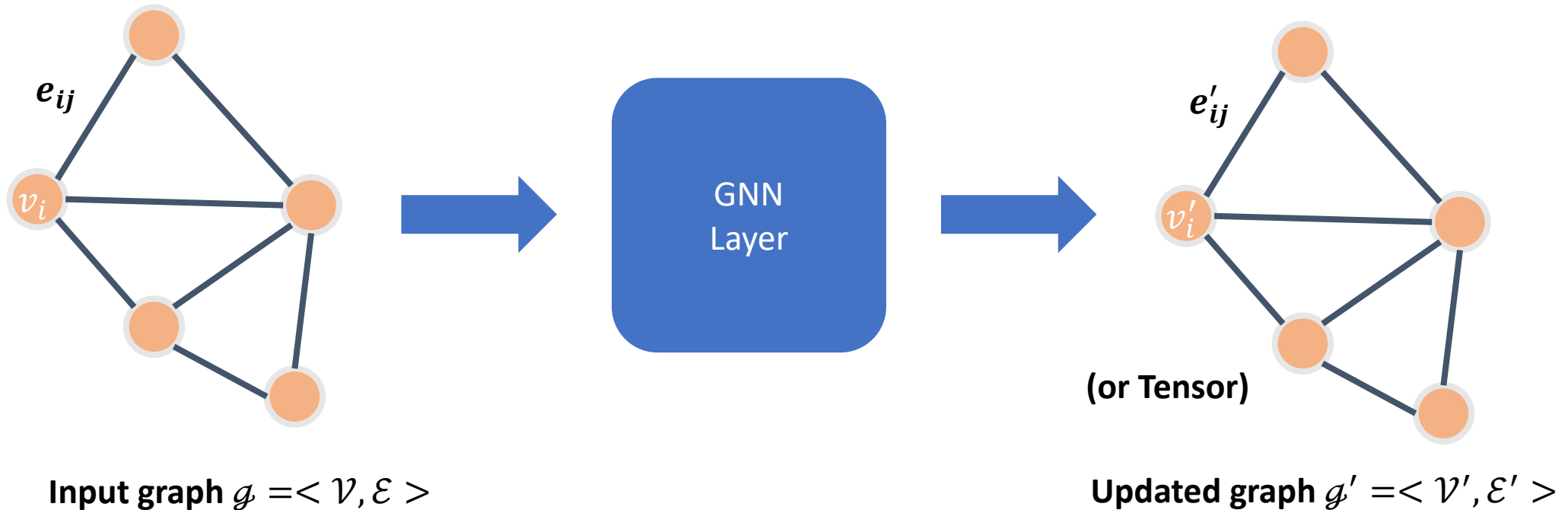
Graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$

$\mathcal{V}$: Set of node features

$\mathcal{E}$: Set of Edge features

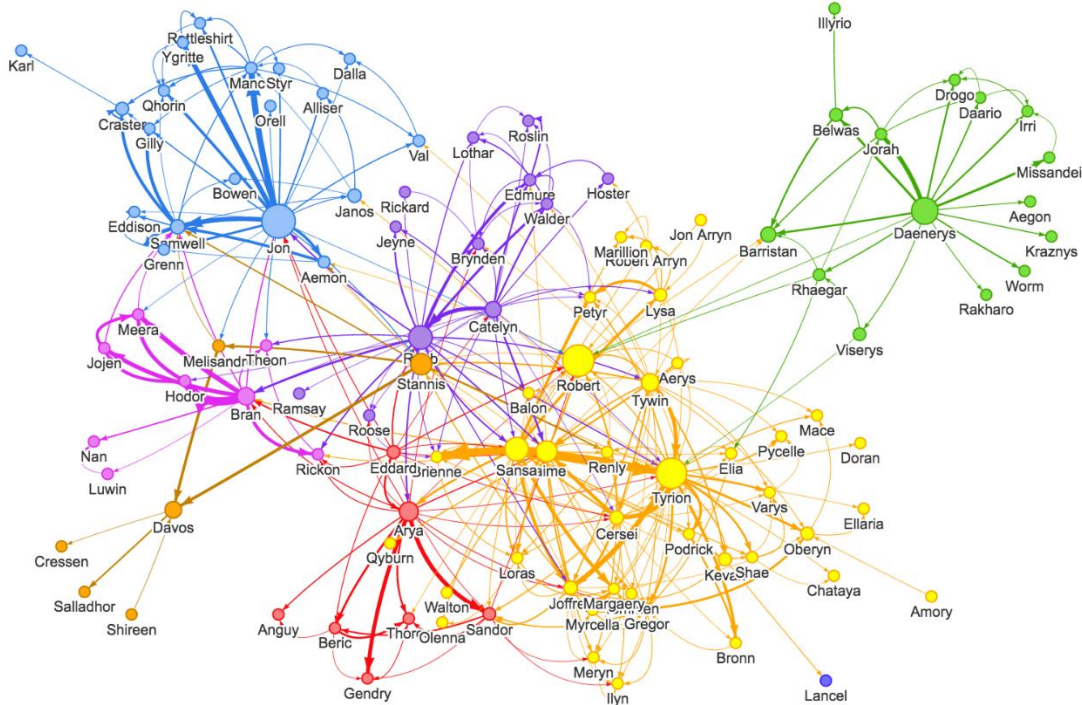
Graph Neural Networks (GNN)

Graph Neural Network (GNN) is a neural network that takes **graph(s)** as inputs

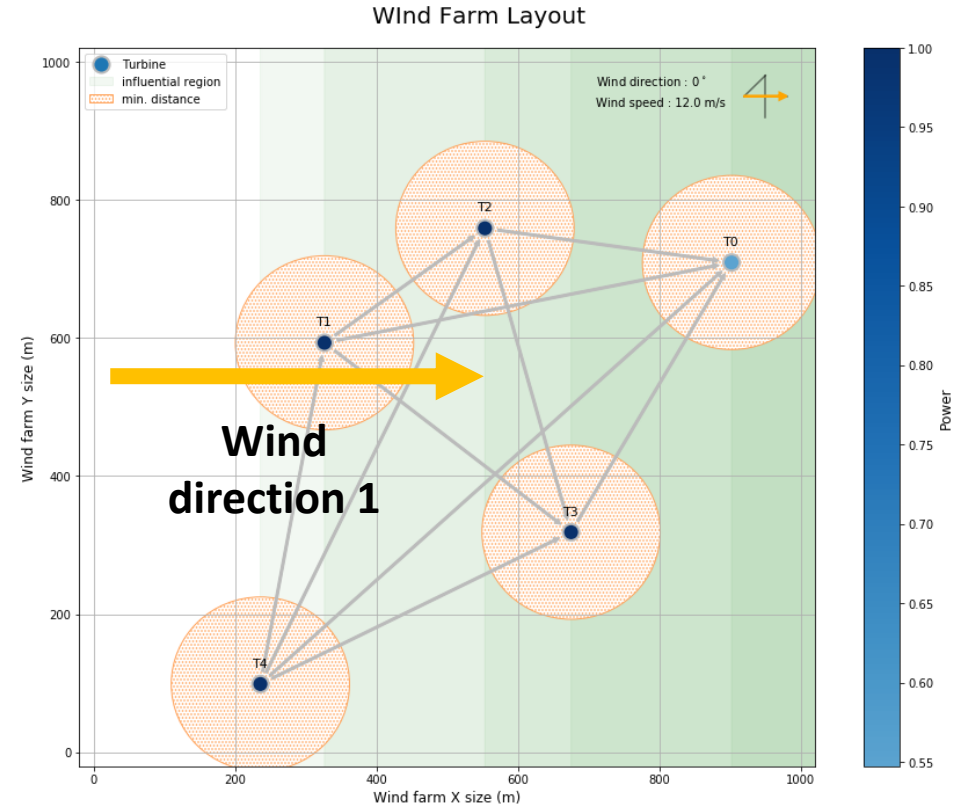


Almost every GNN defines $\begin{cases} \text{(1) update rule for a node/edge} \\ \text{(2) apply the rule on entire (partial) nodes/edges} \end{cases}$ to get updated features

Introductory example



Node clustering on social influence graph

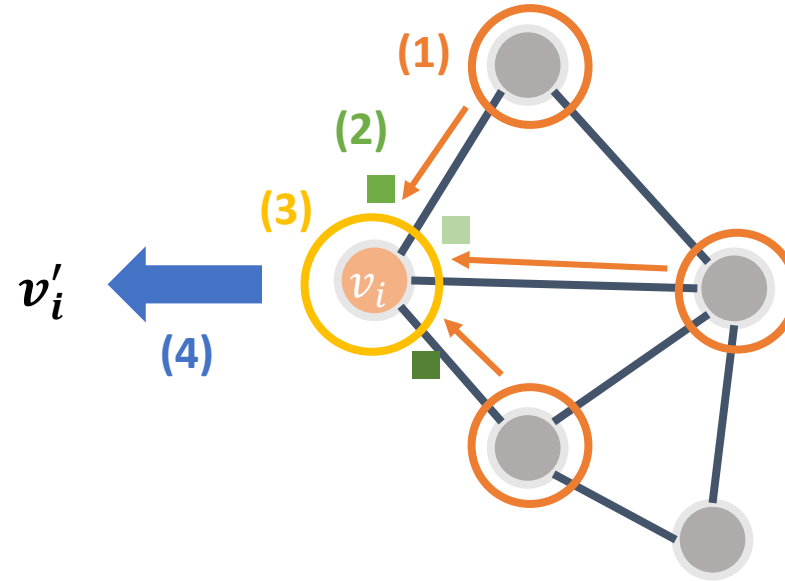


Wind turbine power estimation

Typical Node update procedure in GNNs

— Often referred as “message passing”

- (1) Generate message
- (2) Weighting messages
- (3) Aggregate message
- (4) Update node feature



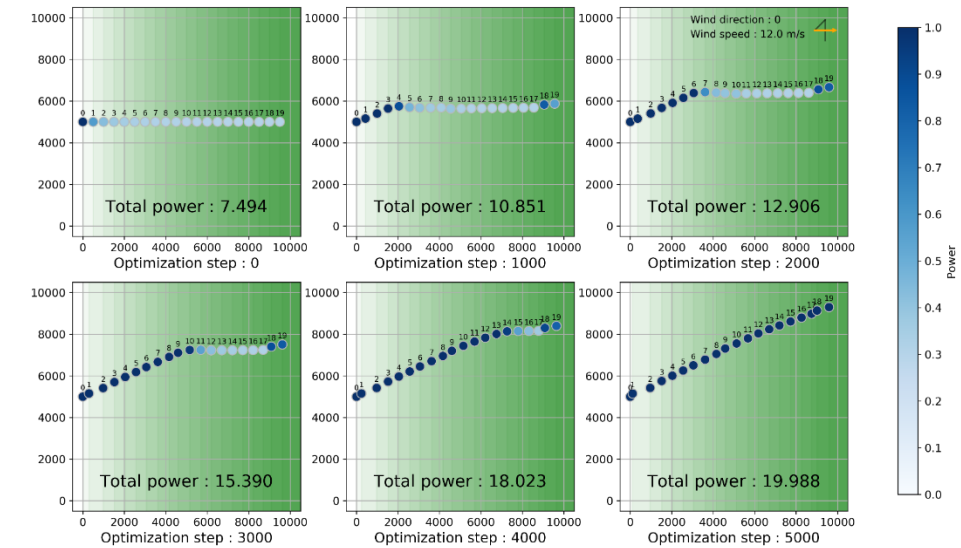
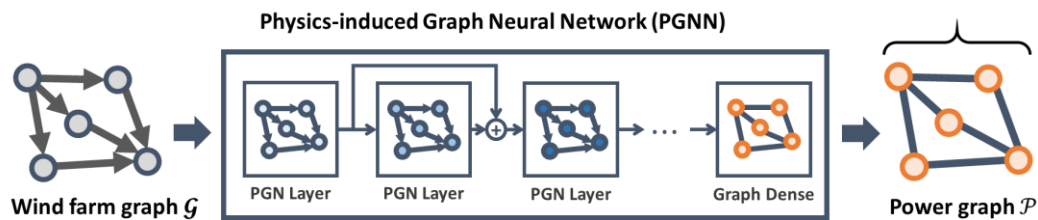
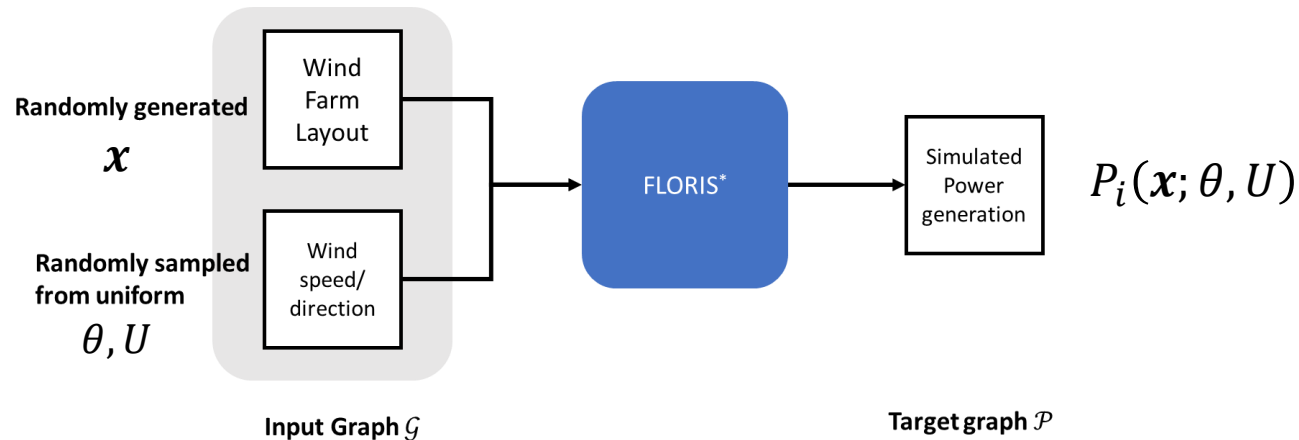
$$v'_i = f_{\theta} \left(v_i, \text{aggregate} \left(w_{j \rightarrow i}(v_i, v_j) * g_{\theta}(v_i, v_j) \right)_{j \in \mathcal{N}_i} \right)$$

Neighborhood set

(Set of directly connected nodes)

- v_i : Node i feature
- f_{θ}, g_{θ} : Differentiable function with parameter θ
- $w_{j \rightarrow i}$: Weight coefficient from node j to node i
- $\text{aggregate}(\cdot)$: Aggregator functions which satisfies ‘permutation invariants’. e.g.) Mean, Sum, Max ...

Application Example 1: Data-Driven Wind Farm Layout Optimization

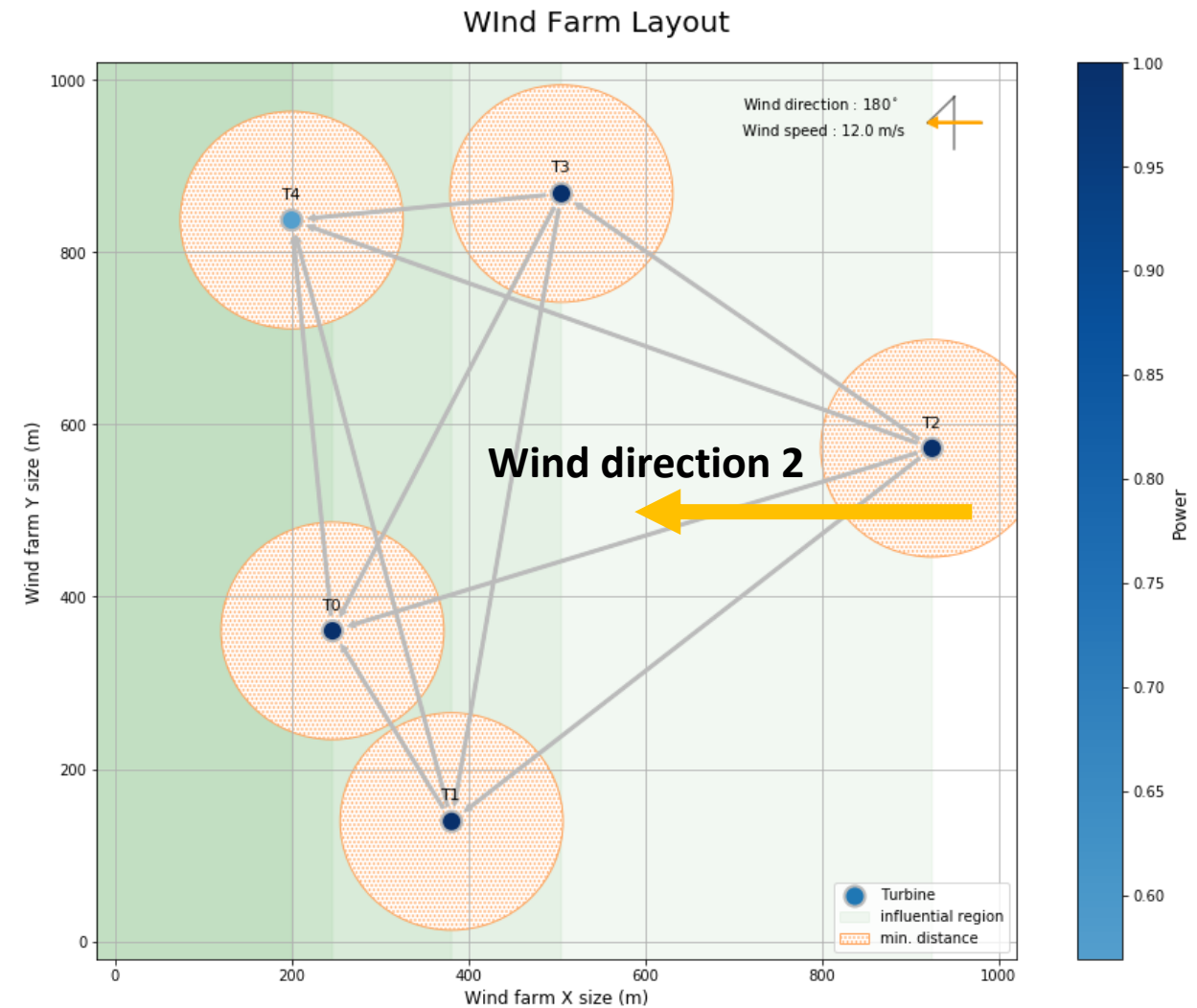
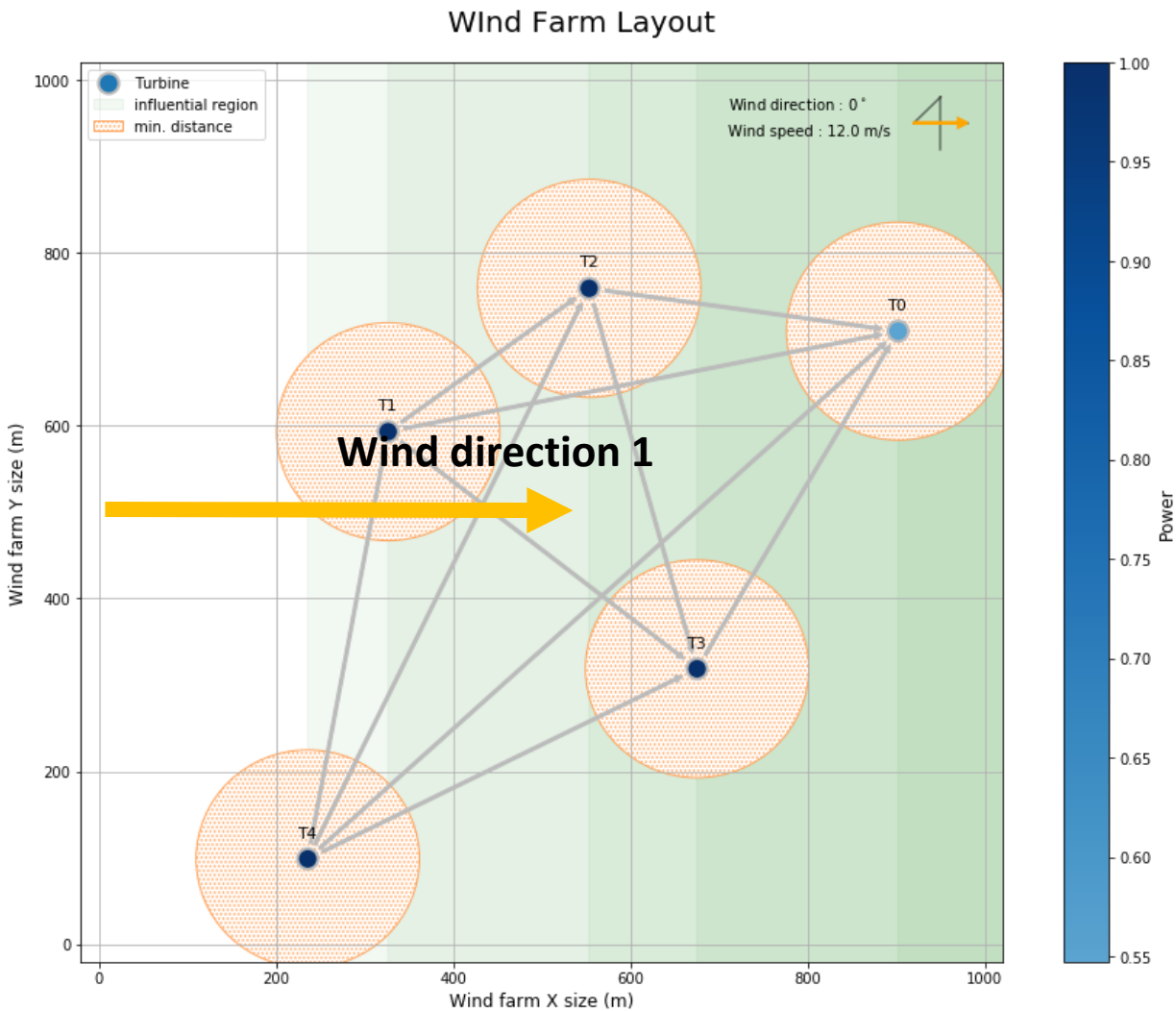


$$\underset{\mathbf{x}}{\text{maximize}} \quad E_{\theta, U} \left[\sum_{i=1}^N P_i(\mathbf{x}; \theta, U) \right]$$

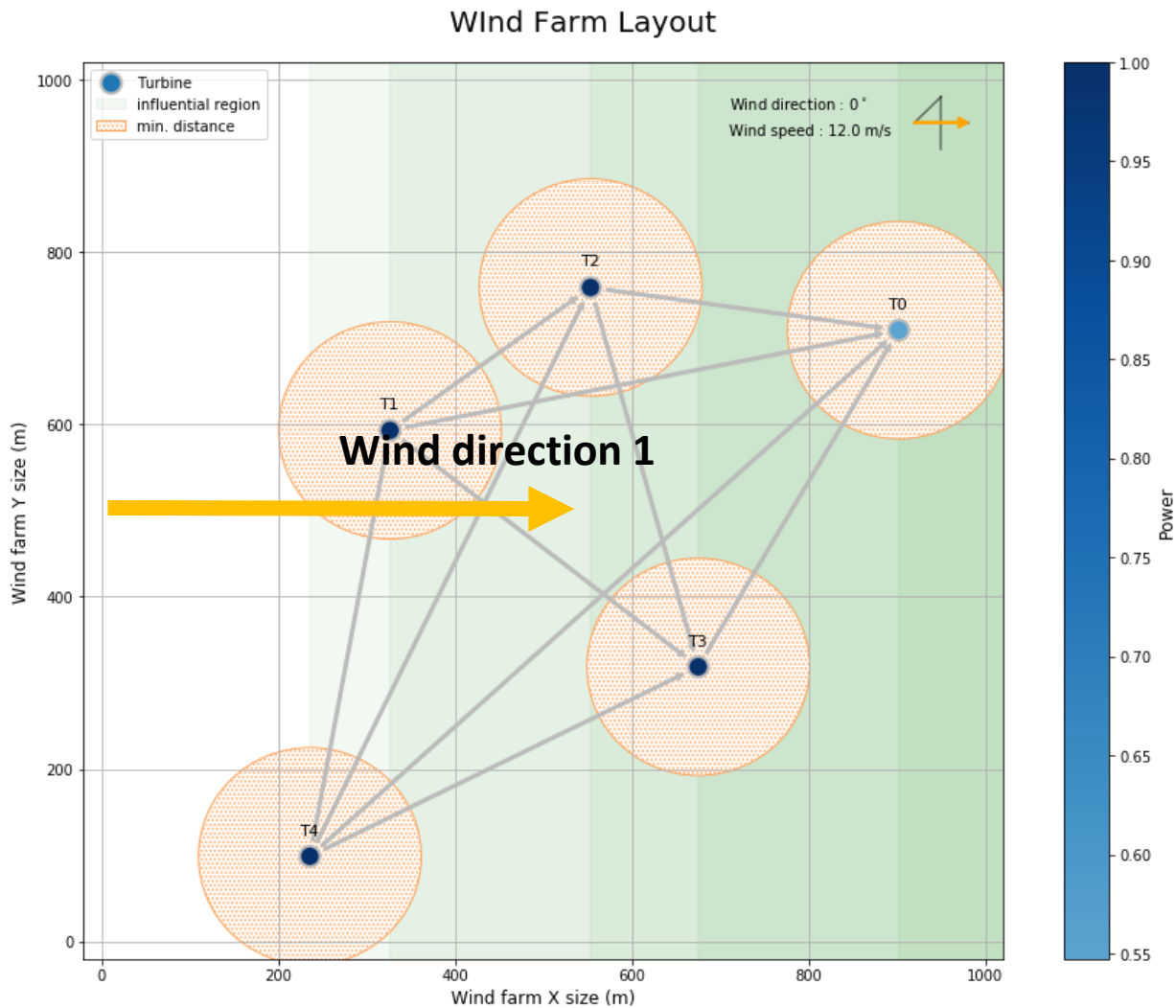
Energy Production Estimation GNN Model

Layout Design Optimization using
The Prediction Model

Wind farm power estimation task



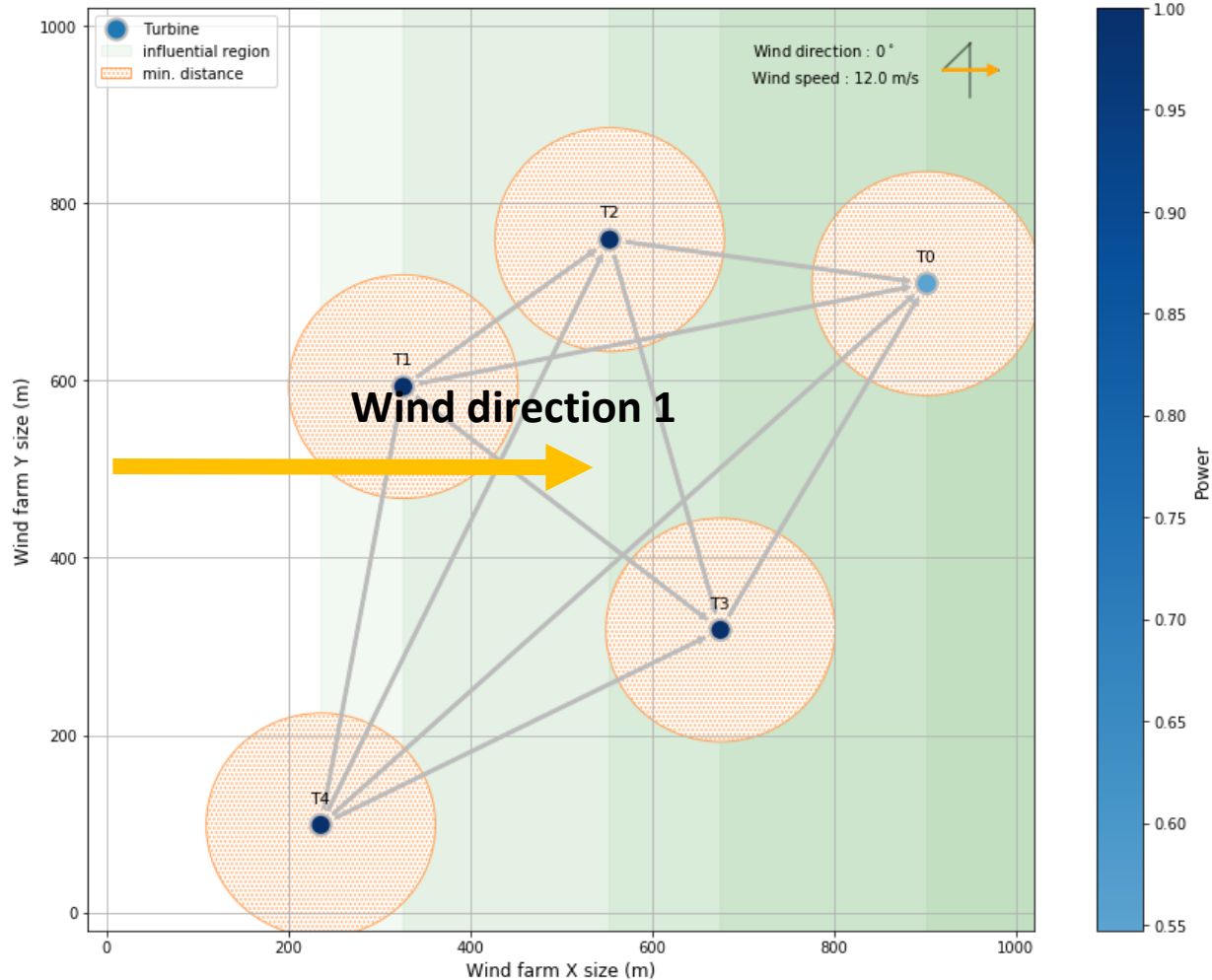
Wind farm power estimation task



- Farm-level power estimation
Wind-farm power = ??
- Turbine-level power estimation
Wind turbine powers = ??

Wind Farm and Its Graph Representation

Wind Farm Layout

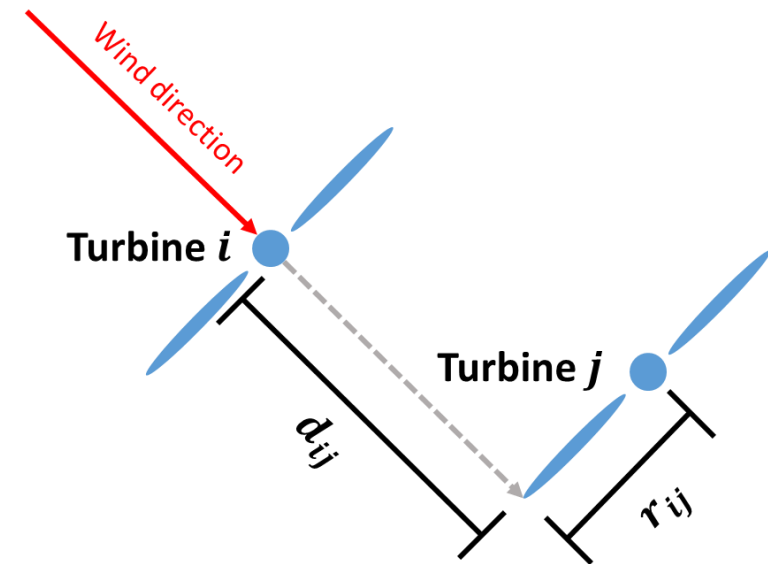


$$\mathcal{G} = (\mathcal{N}, \mathcal{E}, \mathcal{g})$$

Node features $\mathcal{N} = \{\text{free flow wind speed}\}_{\forall i \in \text{turbine index}}$

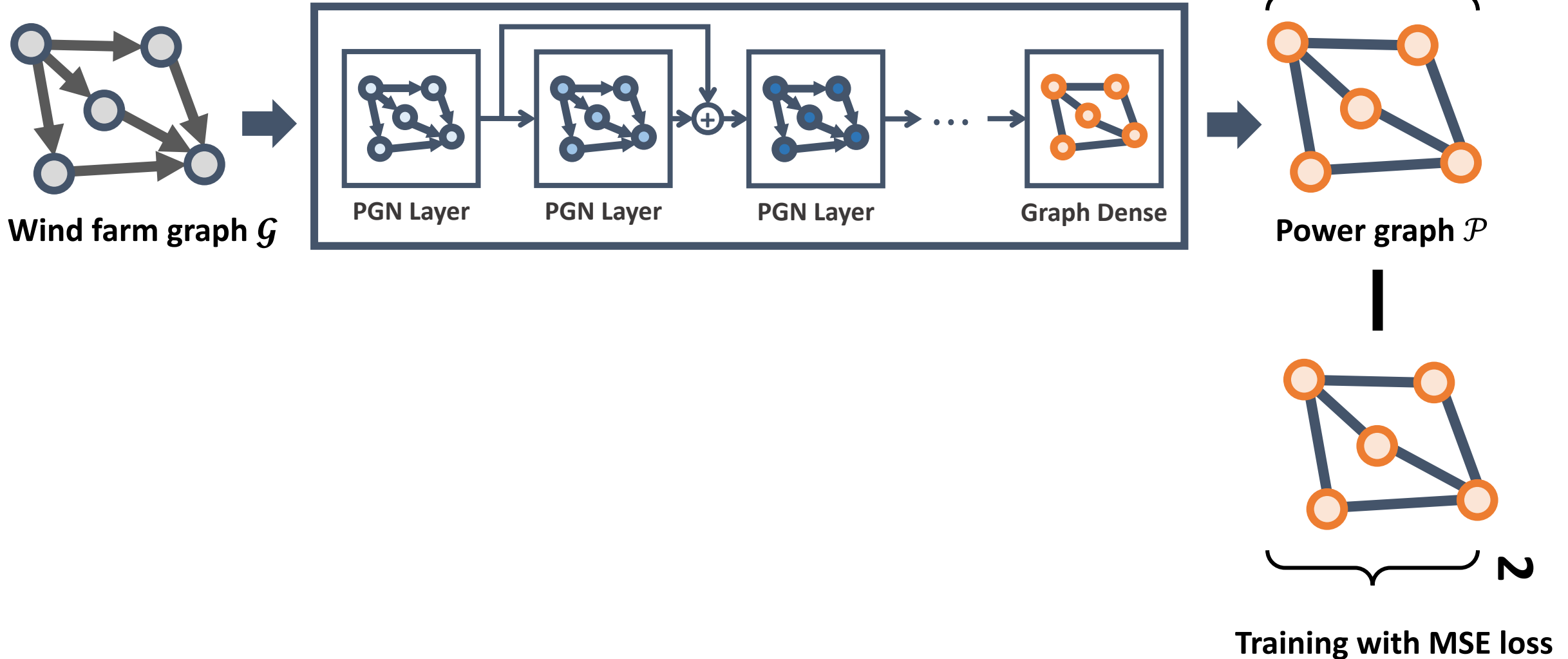
Edge features $\mathcal{E} = \left\{ \begin{pmatrix} \text{the down-stream wake distance } d, \\ \text{the radial-wake distance } r \end{pmatrix} \right\}_{\forall (i,j)^*}$

Global features $\mathcal{g} = \{\text{free flow wind speed}\}_{i,j \text{ are turbine index}}$
 $* \forall (i,j) \in \text{interaction turbines}$



Model Overview

Physics-induced Graph Neural Network (PGNN)



Physics-induced Graph Network (PGN) block



PGN block maintains 4 differentiable, also trainable, modules:

- Edge updater $f_e(\cdot)$
 - Node updater $f_n(\cdot)$
 - Global updater $f_g(\cdot)$
 - Physics-induced weight function $f_w(\cdot)$
- } MLPs

GN block also maintains 3 aggregating modules:

- Edge aggregator for node update $\rho^{e \rightarrow v}(\cdot)$
- Edge aggregator for global feature update $\rho^{e \rightarrow u}(\cdot)$
- Node aggregator for global feature update $\rho^{v \rightarrow u}(\cdot)$

- Edge update

$$e'_{ij} = f_w(e_{ij}) \times f_e(e_{ij}, v_i, v_j, u)$$

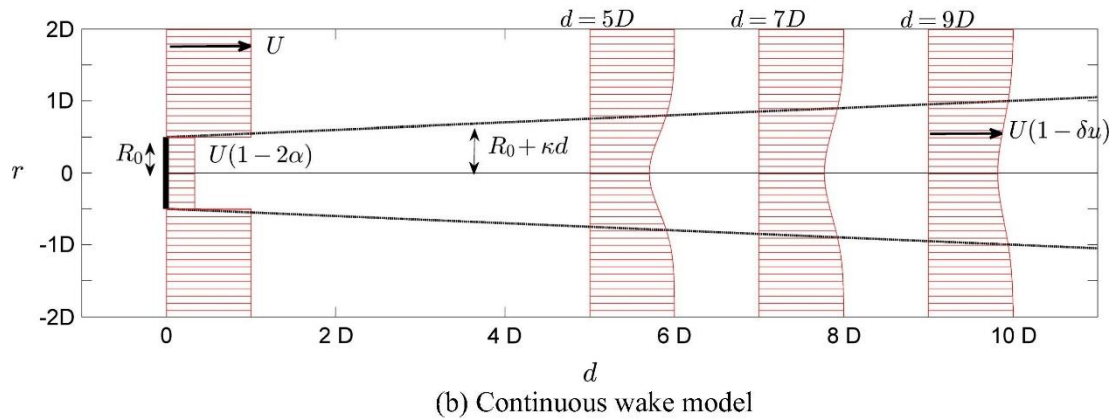
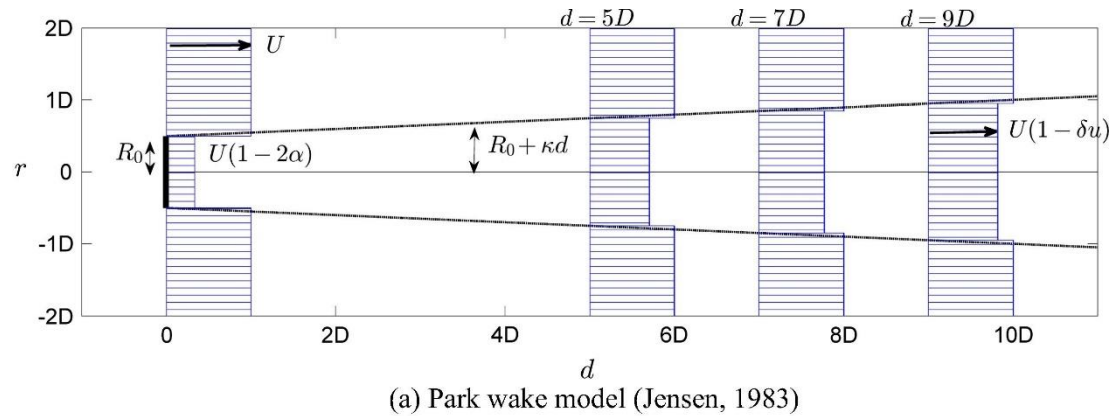
- Node update

$$v'_i = f_n\left(v_i, u, \rho^{e \rightarrow v}\left(\{e'_{ij}\}_{j \in \mathcal{N}_i}\right)\right)$$

- Global feature update

$$u' = f_g(u, \rho^{e \rightarrow u}(\mathcal{E}'), \rho^{v \rightarrow u}(\mathcal{V}'))$$

Details on physics-induced weight function



JK park, and K.H. law suggest the continuous deficit factor $\delta u(d, r, \alpha)$ as

$$\delta u(d, r) = 2\alpha \left(\frac{R_0}{R_0 + \kappa d} \right)^2 \exp \left(- \left(\frac{r}{R_0 + \kappa d} \right)^2 \right)$$

R_0 : Rotor diameter

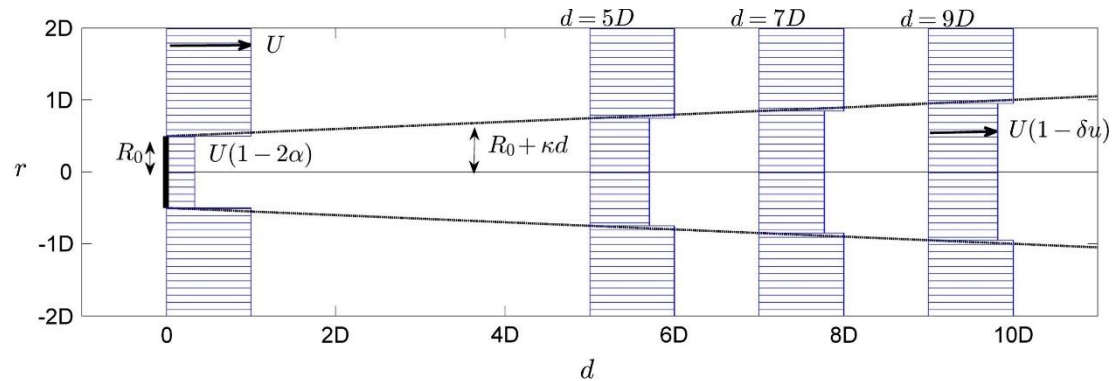
d : Down-stream wake distance

r : Radial wake - distance

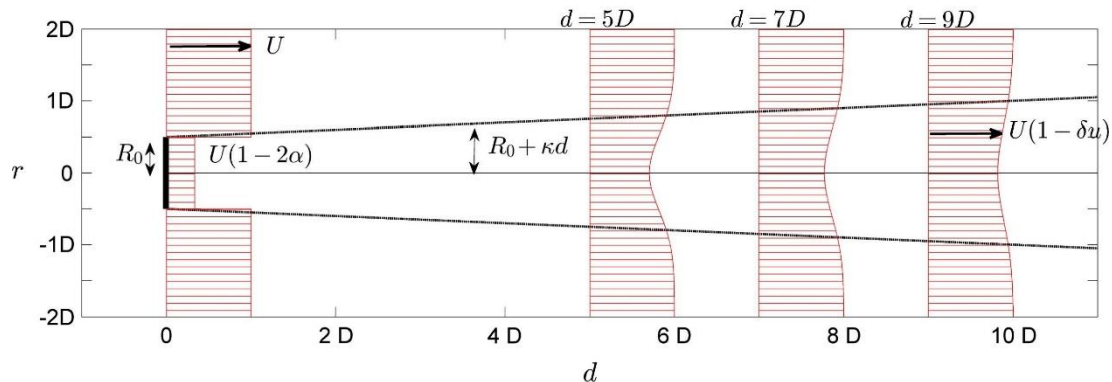
α, κ : Tunable parameters

Figure source <Cooperative wind turbine control for maximizing wind farm power using sequential convex programming by Jinkyoo Park, Kincho H.Law >

Details on physics-induced weight function



(a) Park wake model (Jensen, 1983)



(b) Continuous wake model

$$\delta u(d, r) = 2\alpha \left(\frac{R_0}{R_0 + \kappa d} \right)^2 \exp \left(- \left(\frac{r}{R_0 + \kappa d} \right)^2 \right)$$

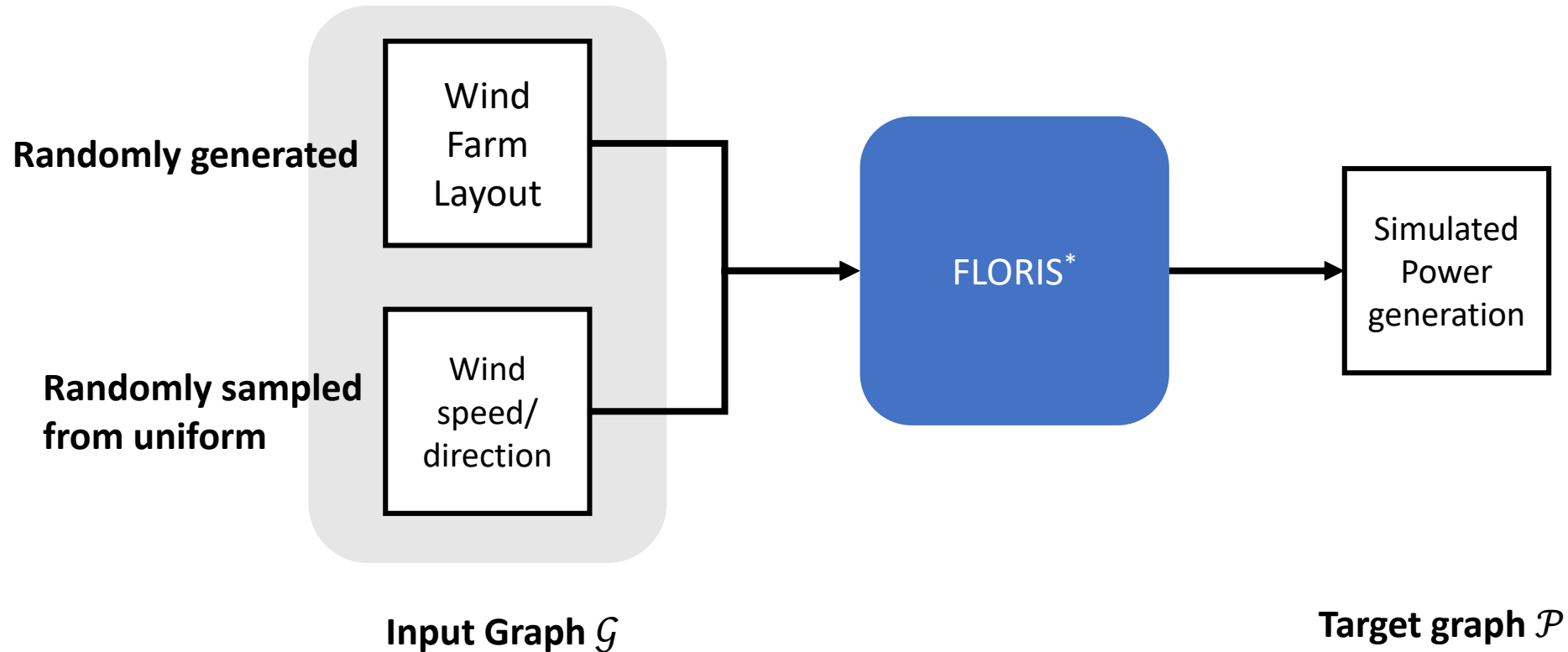
$\delta u(d, r)$ indicates
 ‘How much the down stream turbine is affected
 Due to the upstream turbines’
 → Weighting Factor w !

However, they tuned the parameters α, κ to the observed data

Let neural network learn α, κ, R_0 !

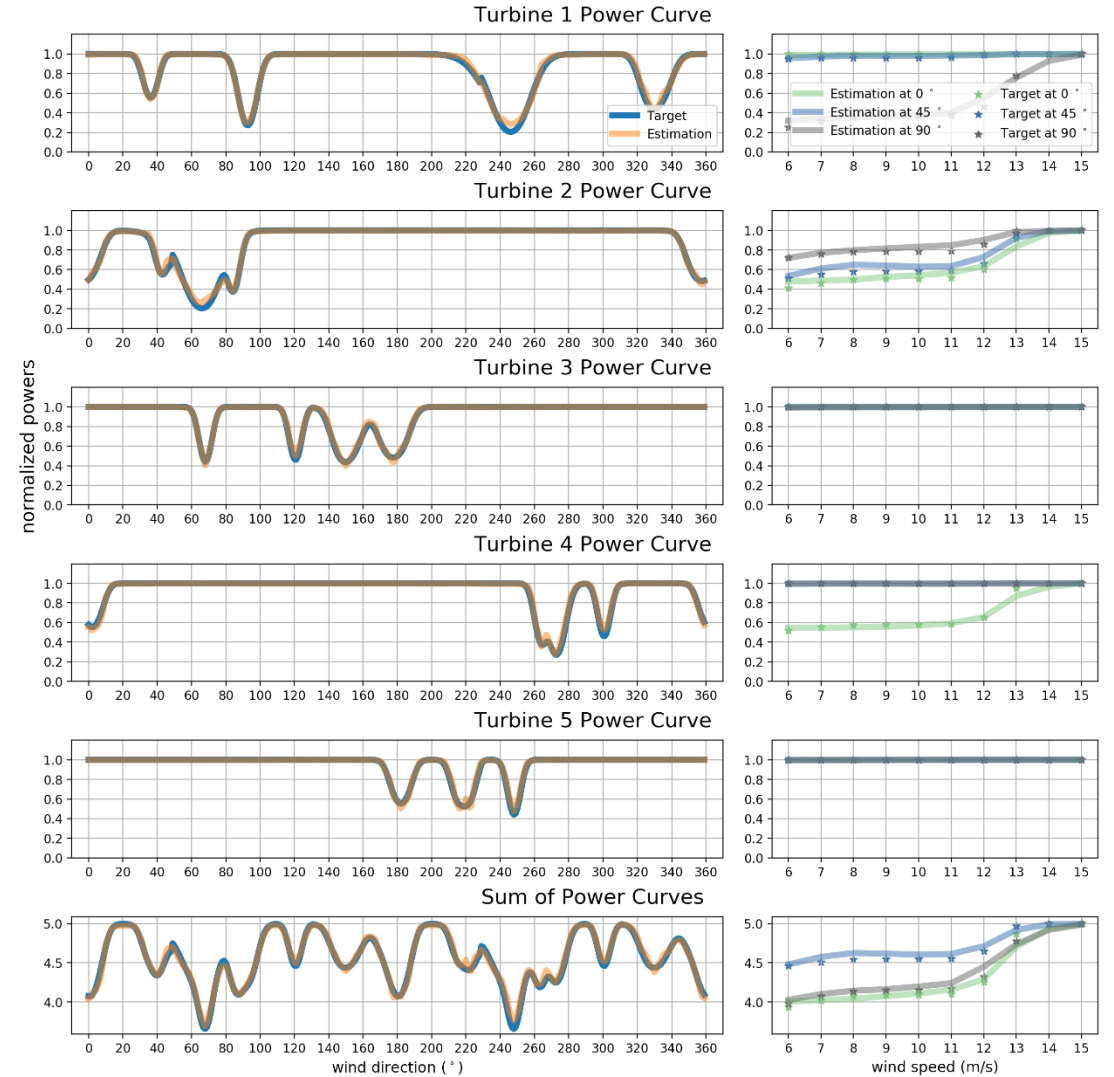
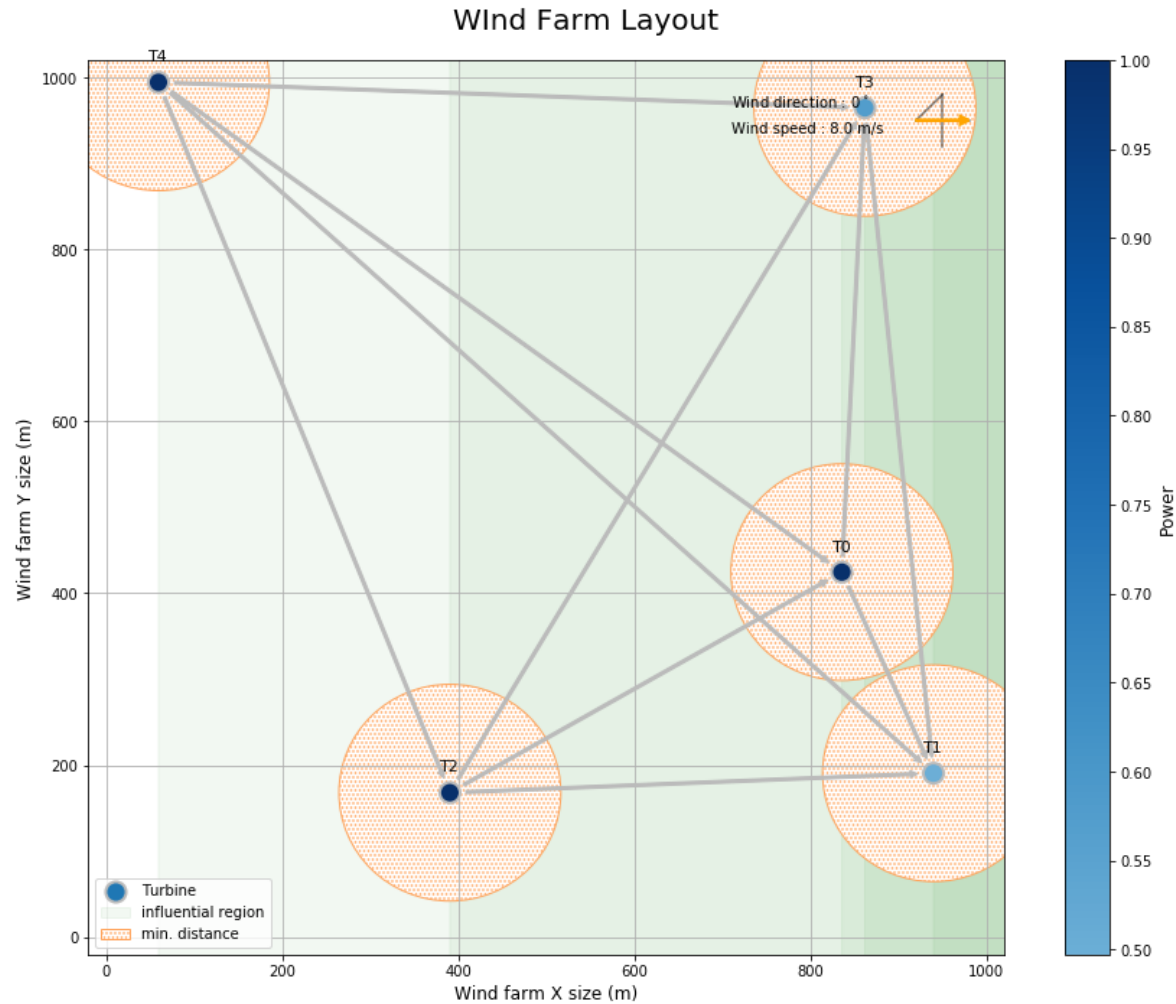
Figure source <Cooperative wind turbine control for maximizing wind farm power using sequential convex programming by Jinkyoo Park, Kincho H. Law >

Training data preparation



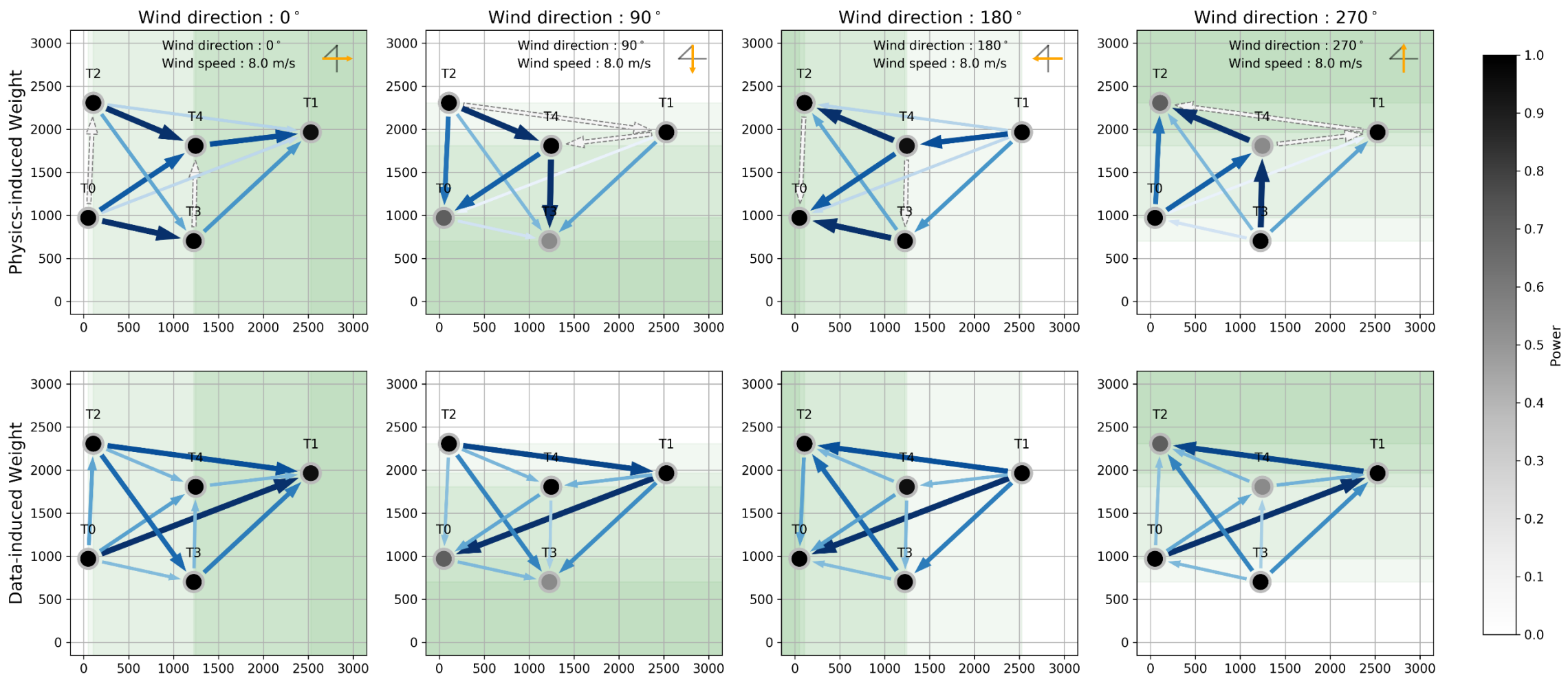
- We used 900 independent training pairs. $D = \{\mathcal{G}_i \mathcal{P}_i\}_{i=1, \dots, 900}$
- #. Turbines were set to be [5,10,15,20].
- We normalize power outcomes [0,1] by dividing the maximum power that turbine would generate when wind speed is given.
 - Power can be interpreted as “Power generation efficiency”

Prediction result on fixed layout

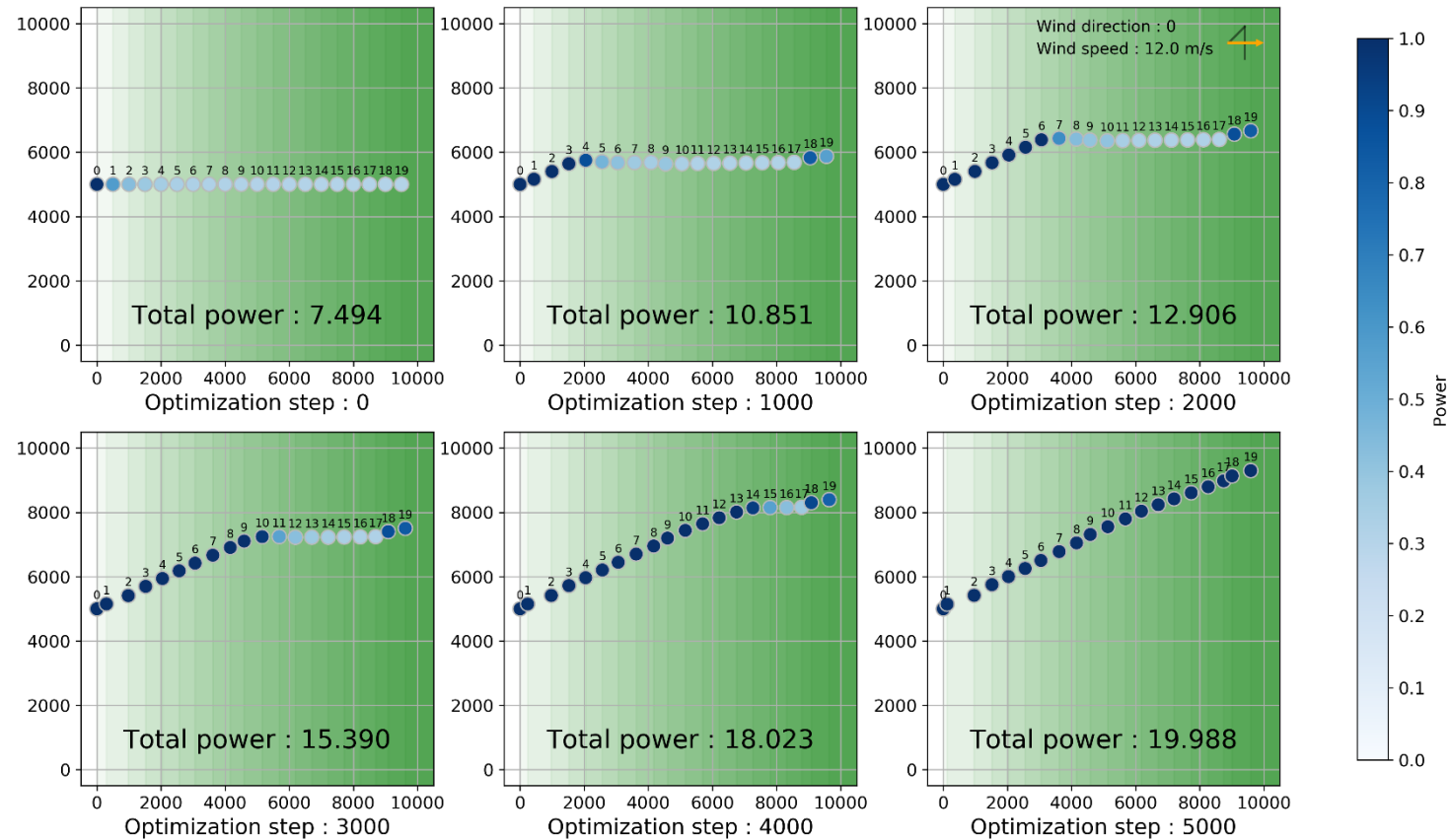


1.5% MAPE error

Visualizing learned weight

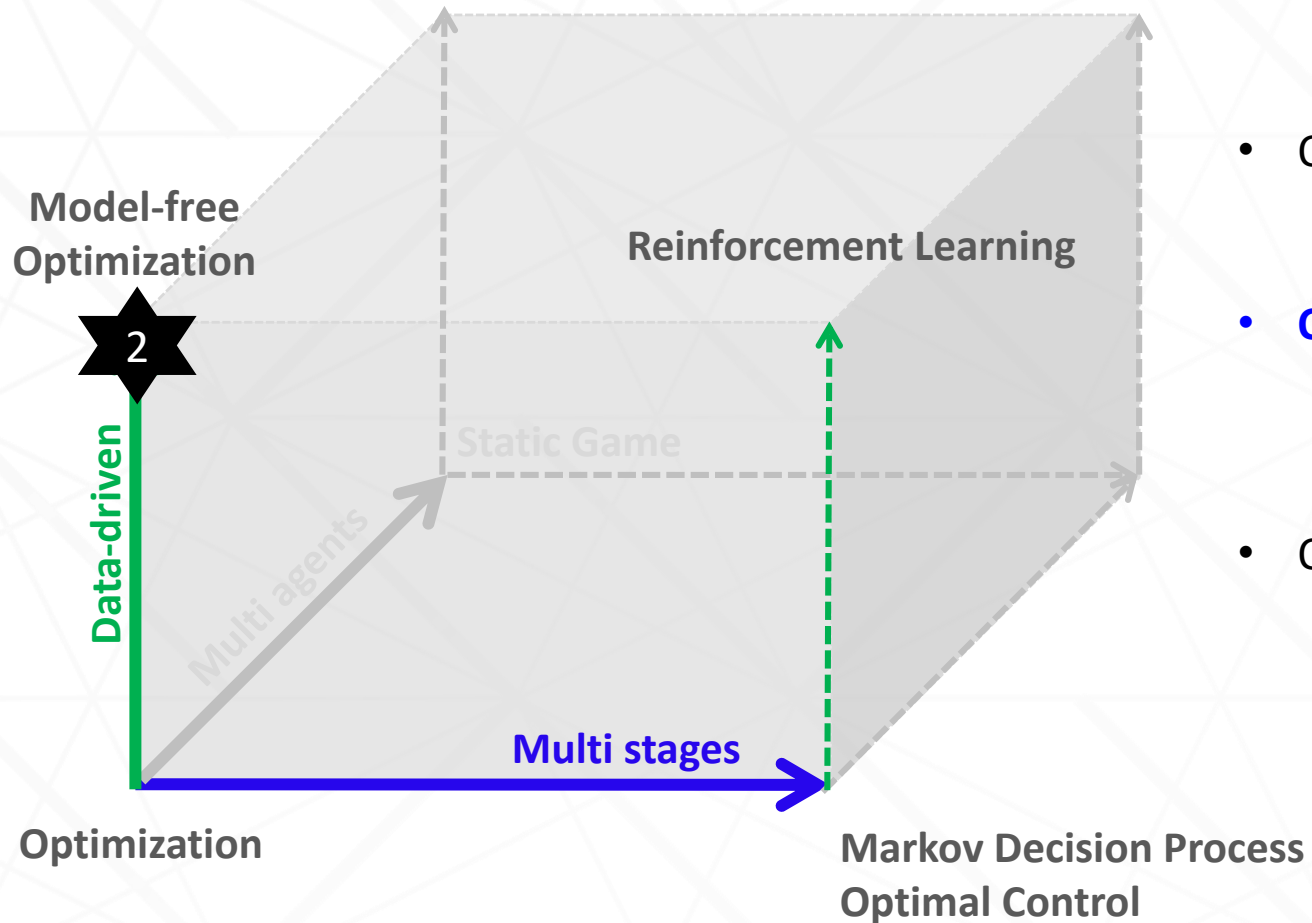


Application to layout optimization



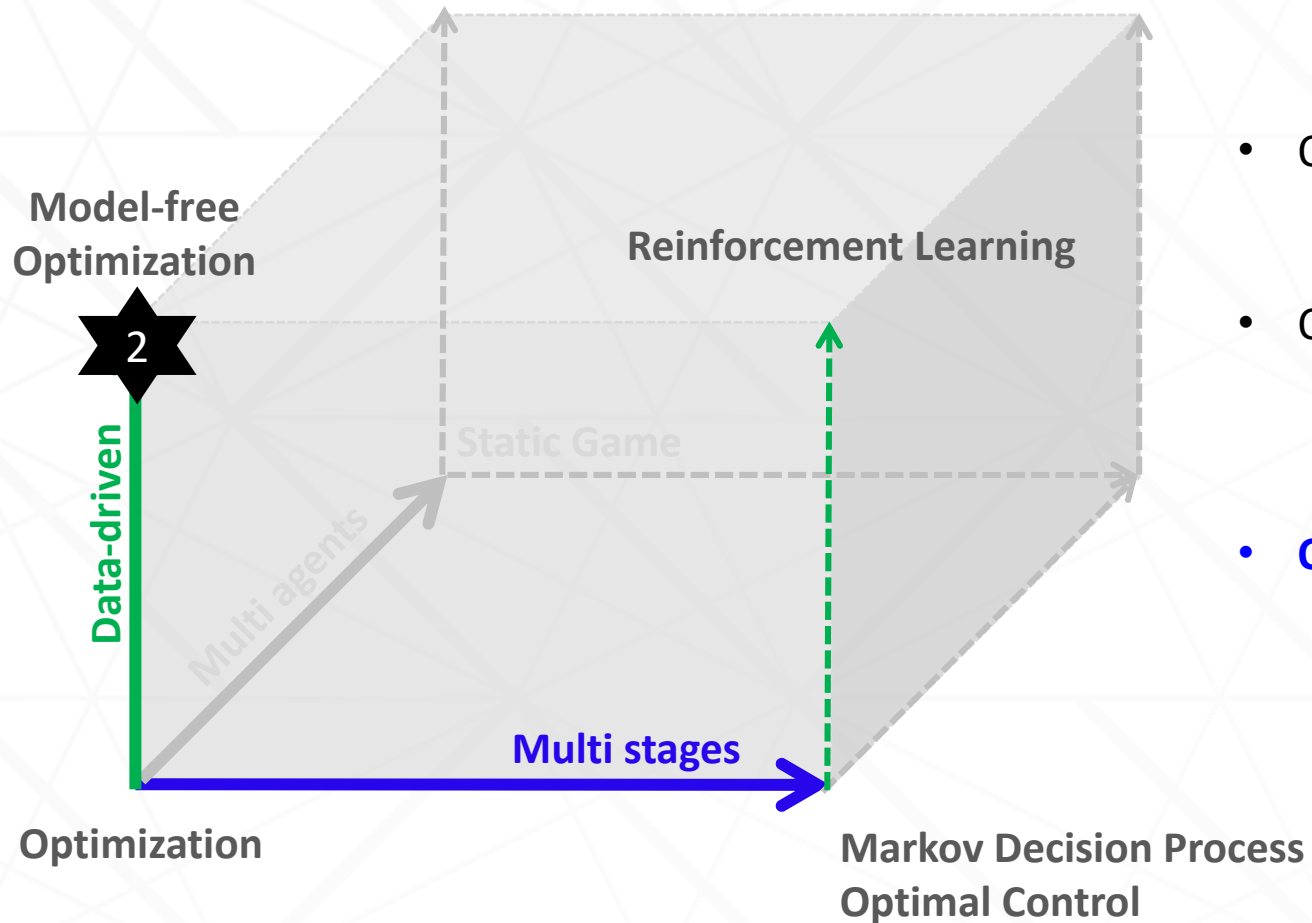
We employed PGNN to optimize wind farm layout. Due to the fact that PGNN is differentiable, we can optimize wind farm layout with gradient methods.

Data-Driven Static Decision Making



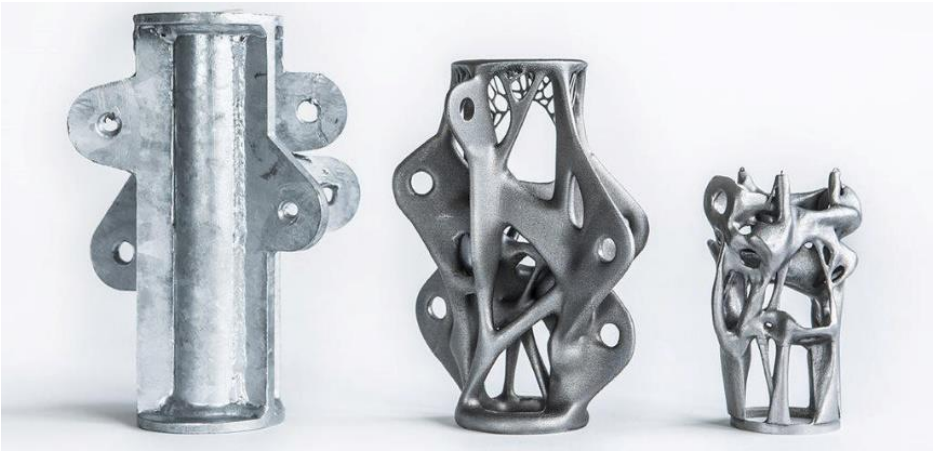
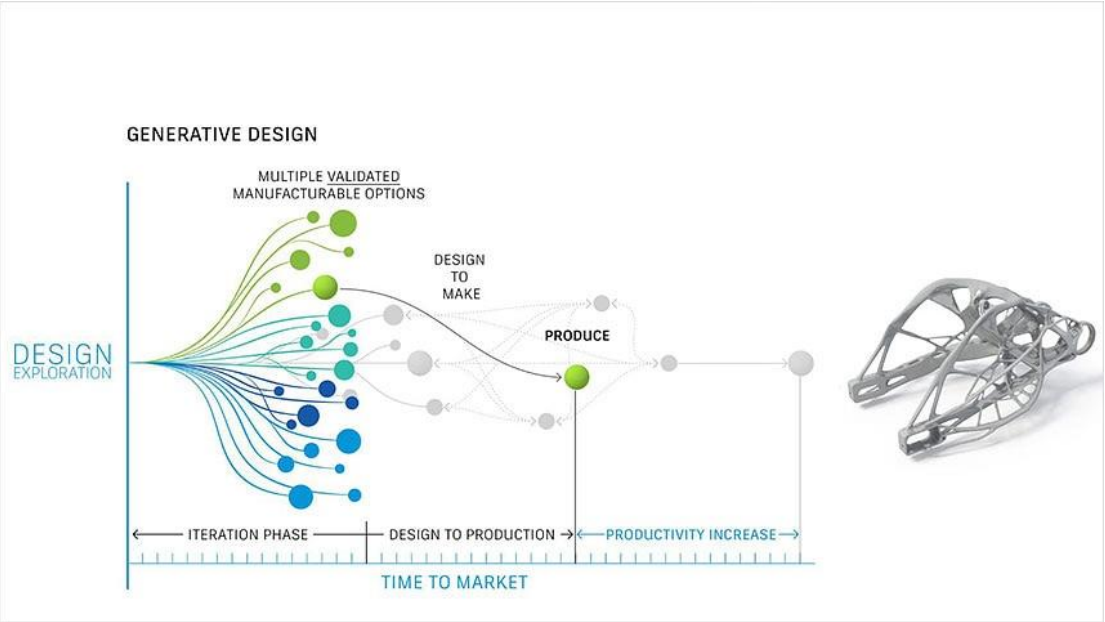
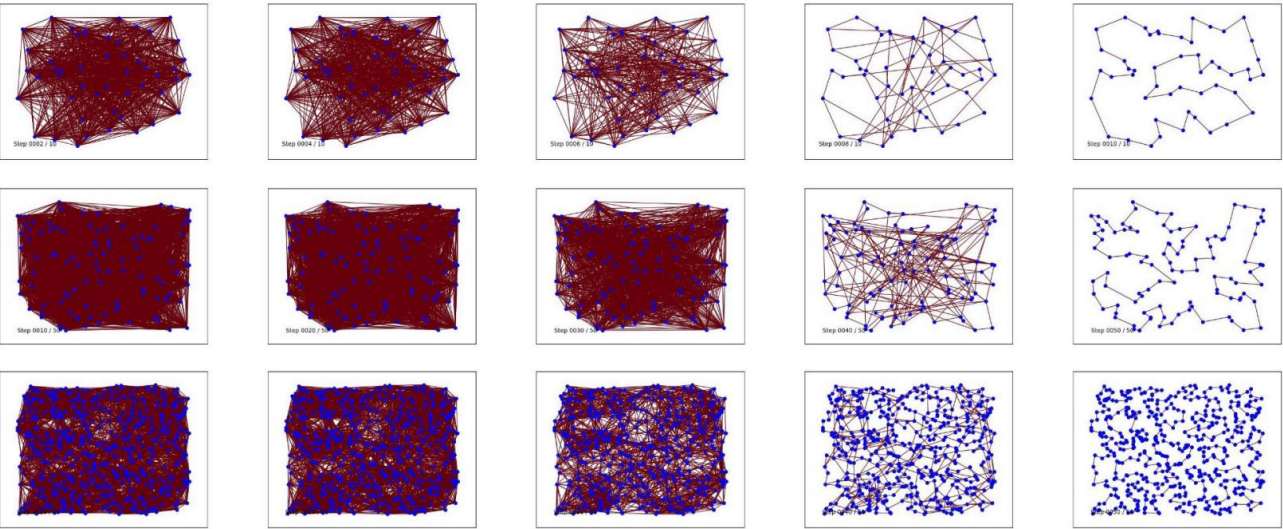
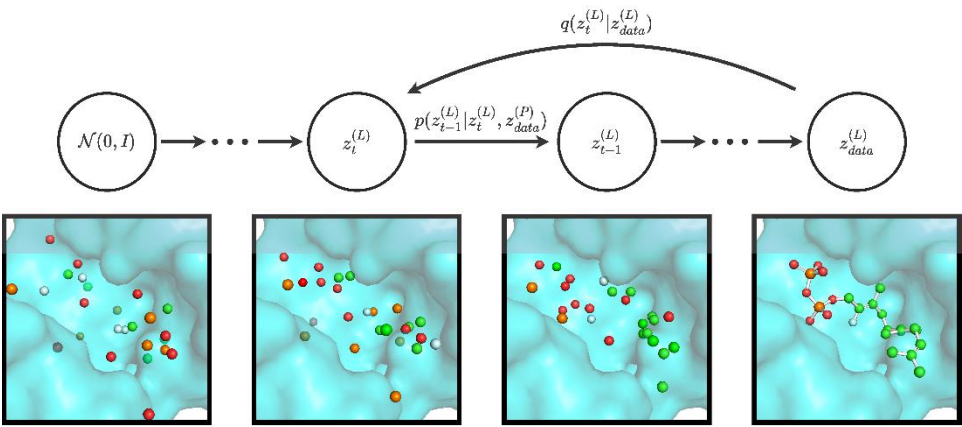
- Online Optimization:
 - Bayesian Optimization
- **Offline Optimization**
 - Surrogate Model Based
 - **Generative Model Based**
- Offline Meta Optimization + Online Adaptation

Data-Driven Static Decision Making

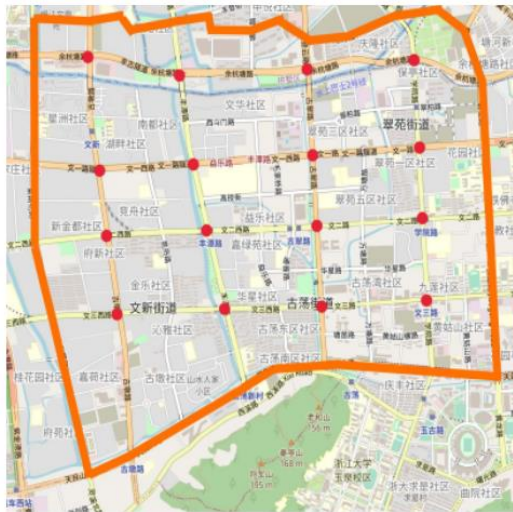


- Online Optimization:
 - Bayesian Optimization
- Offline Optimization
 - Surrogate Model Based
 - Generative Model Based
- **Offline Meta Optimization + Online Adaptation**

Design Optimization using Generative Models



Application Example 2: Offline Meta Data-Driven Traffic Signal Split Optimization



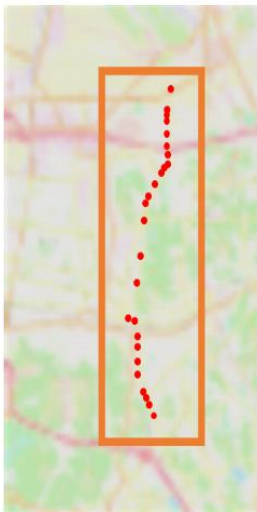
(a) Hangzhou, China



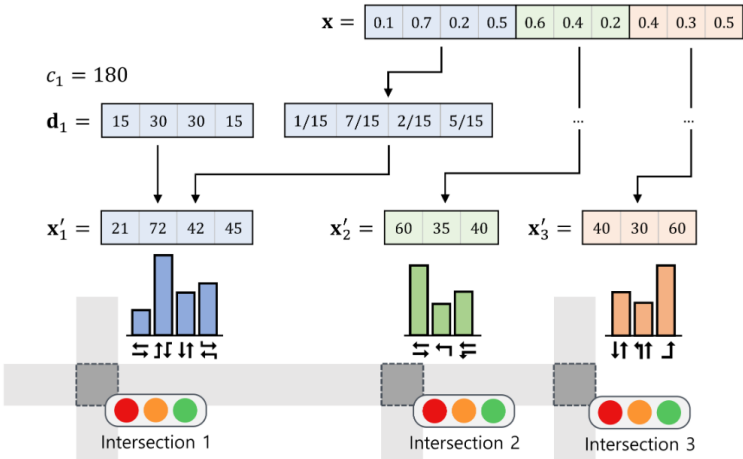
(b) Manhattan, USA



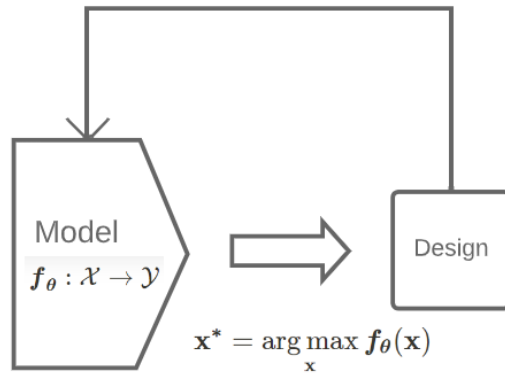
(c) District A



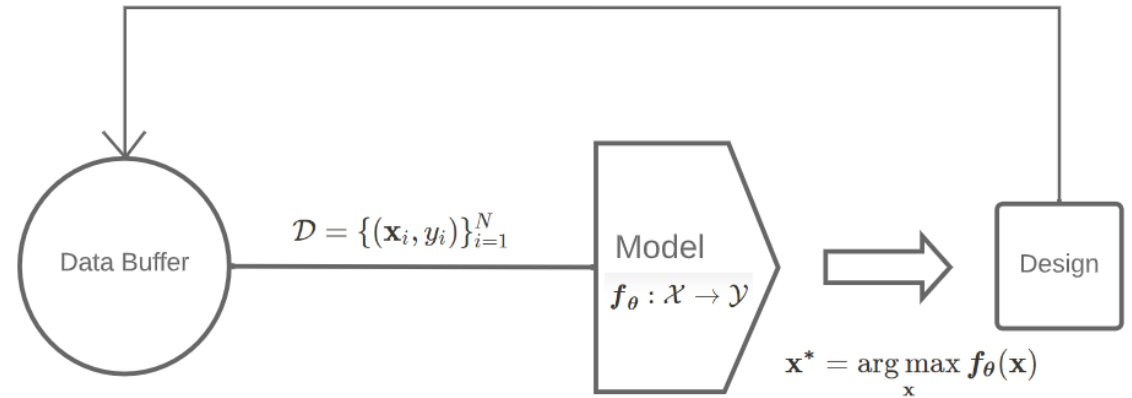
(d) District B



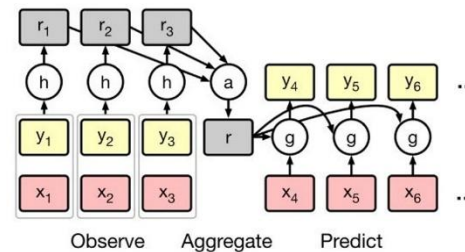
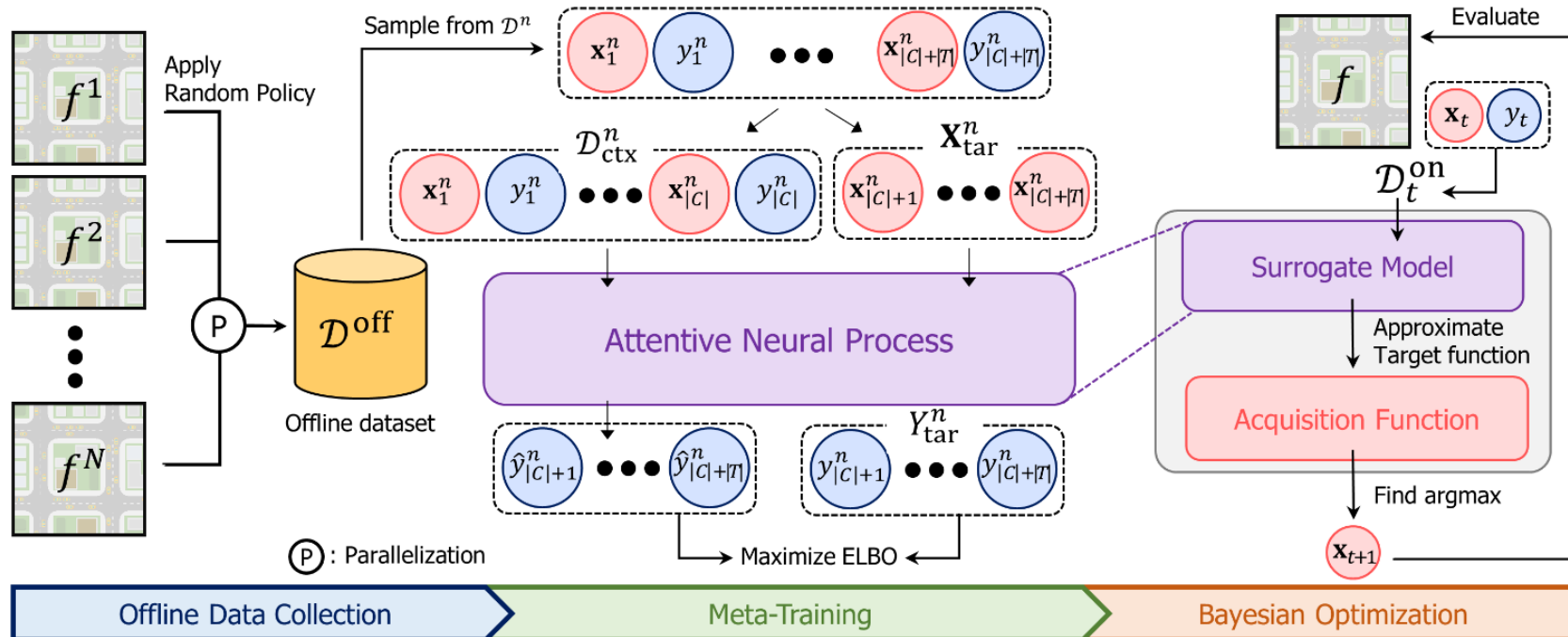
Bayesian Optimization vs. Meta Bayesian Optimization



Online Adaptation

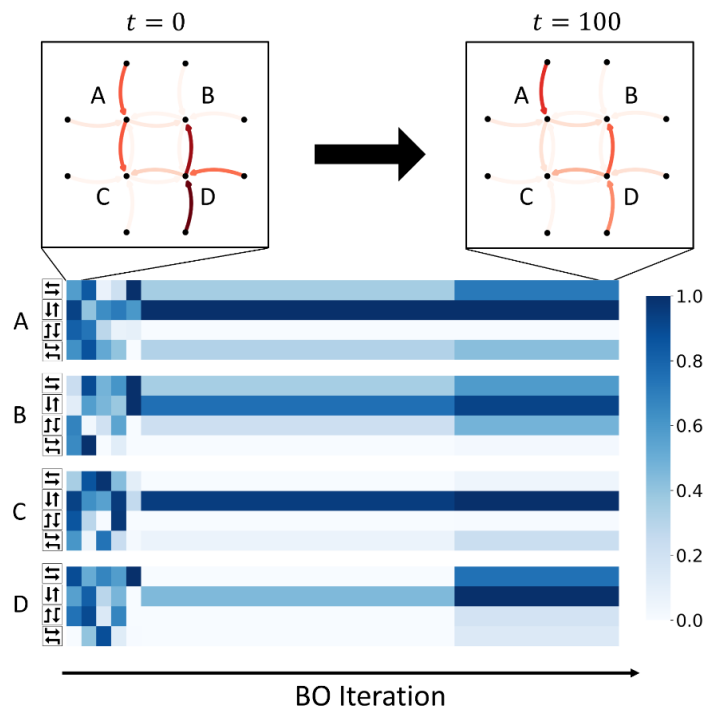


Offline + Online Adaptation

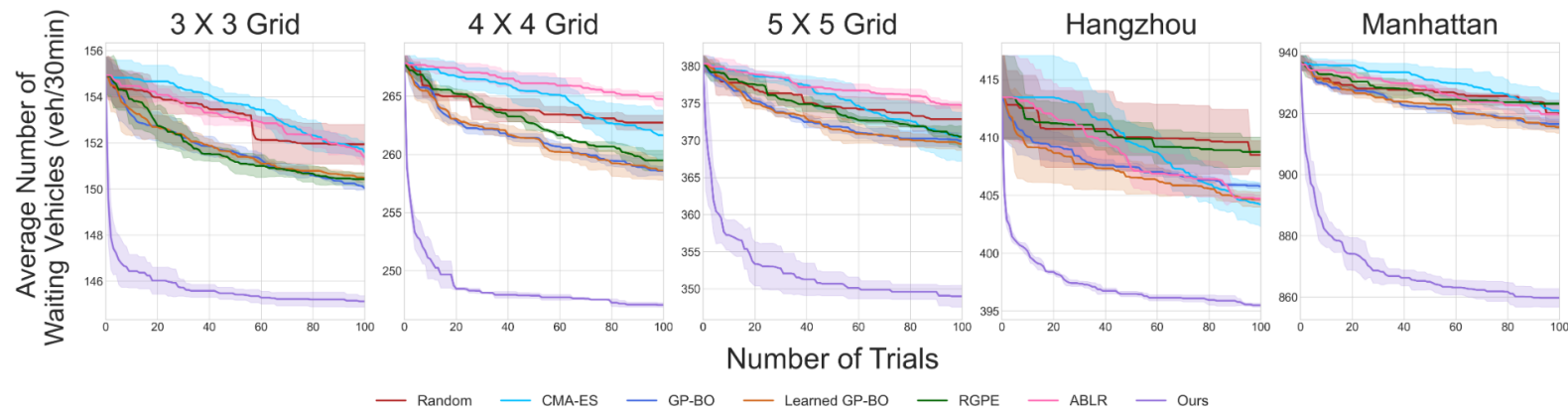


Conditional Neural Process, Marta Garnelo et al.

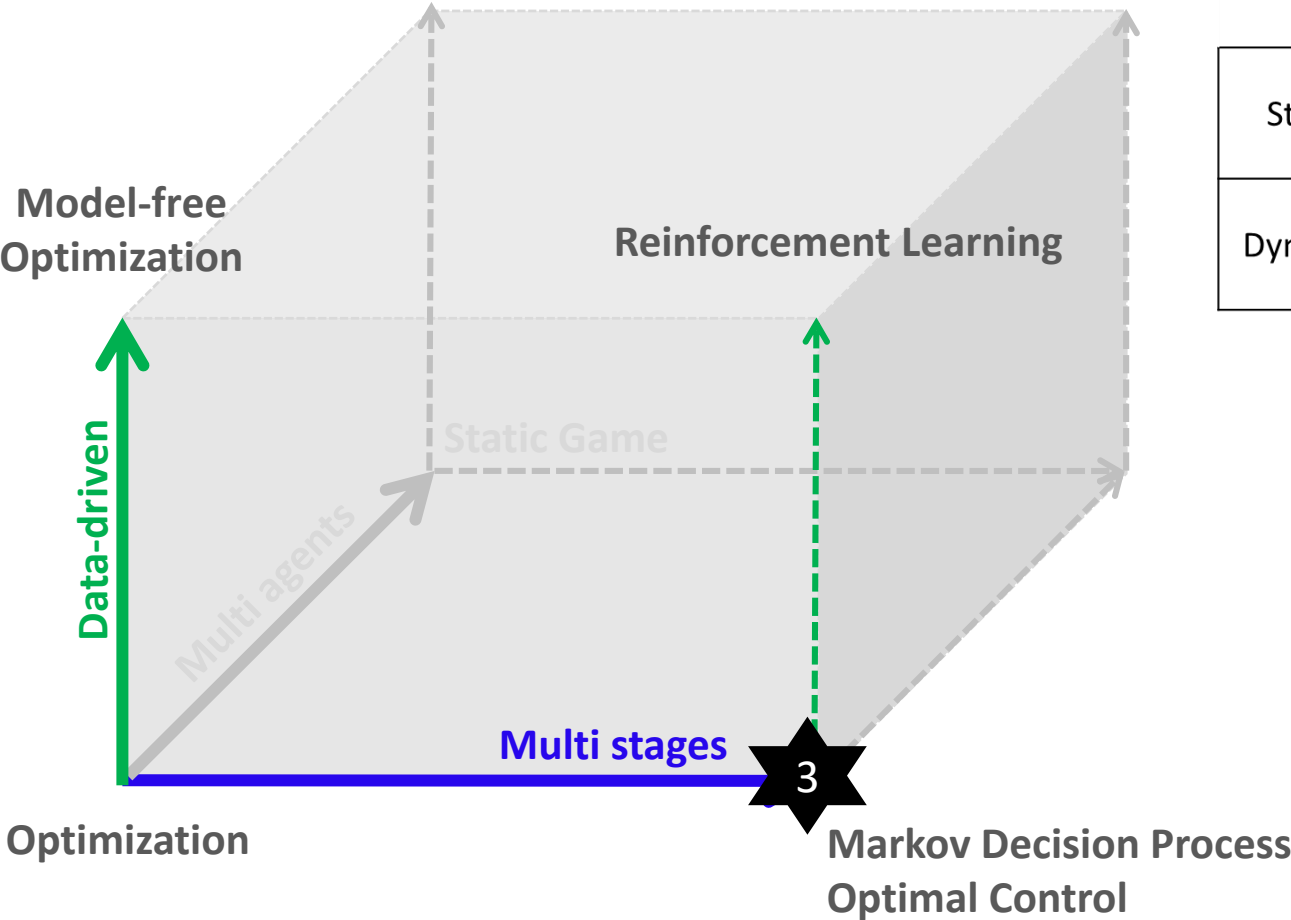
Problem Formulation for Traffic Light Control



Method	Average Number of Waiting Vehicles ($\downarrow$)				
	Synthetic Dataset			Public Real-world Dataset	
	3 × 3 Grid	4 × 4 Grid	5 × 5 Grid	Hangzhou	Manhattan
Random	151.94 ± 0.86	262.72 ± 0.64	372.85 ± 1.88	408.49 ± 1.24	920.07 ± 4.24
CMA-ES	151.58 ± 0.39	261.63 ± 2.15	370.47 ± 3.29	404.18 ± 1.89	920.79 ± 5.76
GP-BO	150.02 ± 0.07	258.56 ± 0.41	369.93 ± 0.92	405.75 ± 0.22	916.57 ± 1.62
Learned GP-BO	150.49 ± 0.21	258.62 ± 0.82	369.52 ± 0.84	404.64 ± 0.65	915.48 ± 1.92
RGPE	150.43 ± 0.28	259.48 ± 0.88	370.49 ± 1.46	408.76 ± 1.27	923.19 ± 0.45
ABLR	151.35 ± 0.31	264.72 ± 0.60	374.72 ± 0.39	404.71 ± 0.30	919.57 ± 3.50
Ours	145.13 ± 0.24	247.06 ± 0.19	348.99 ± 1.46	395.51 ± 0.17	859.74 ± 3.02



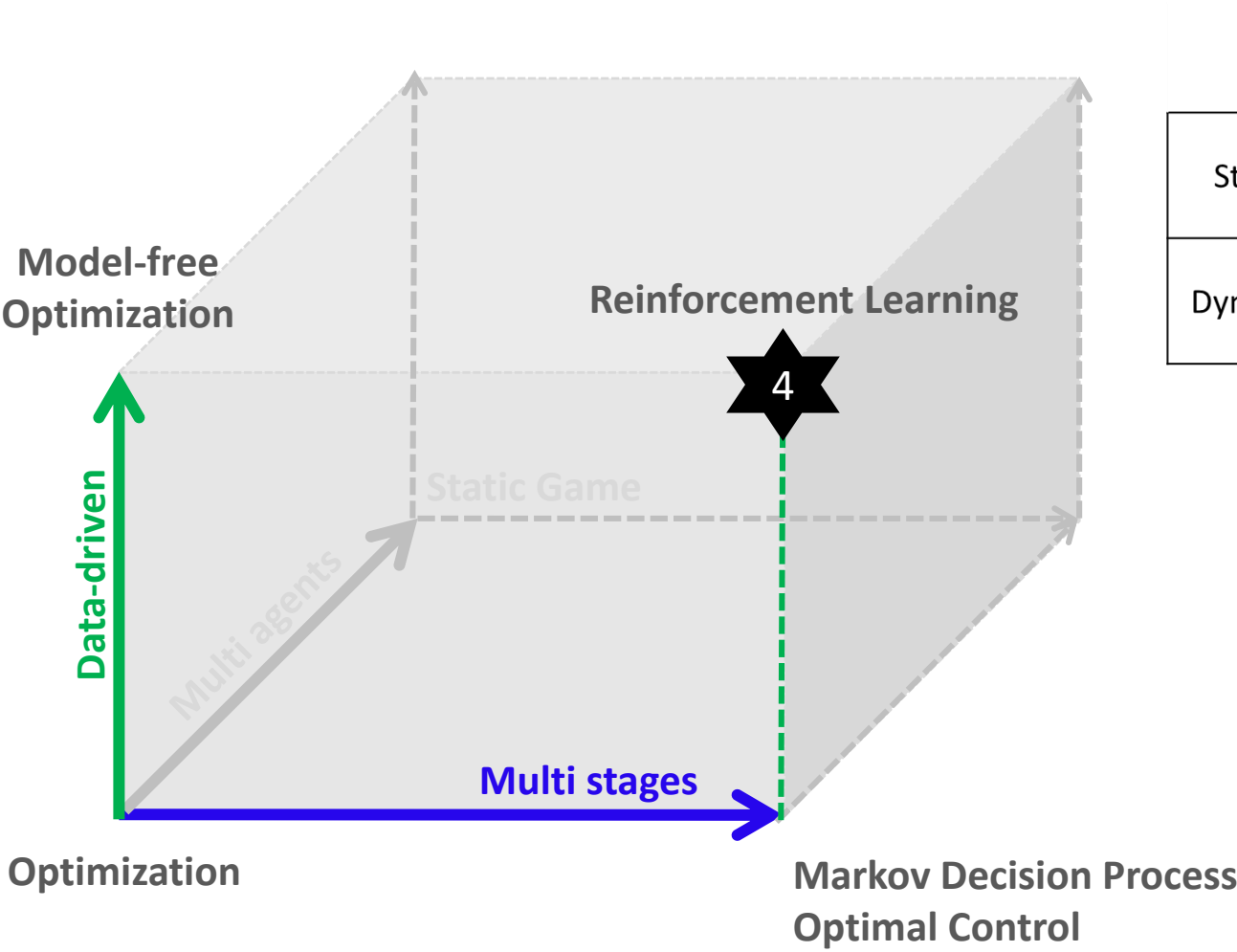
Fundamentals on Dynamic Decision Making (Optimal Control)



	Single Agent	Multi Agent
Static	Static optimization	Static Game
Dynamic	Dynamic Optimization	Dynamic Game

Action space			
Time space	Model based		
		Finite	Infinite
	Discrete	Discrete time MDP $P(s_{t+1} s_t, a_t)$	Discrete-time dynamic system $x_{t+1} = f(x_t, u_t)$
	Continuous	Continuous time MDP $P(s_{t+h} s_t, a_t)$	Continuous-time dynamic system $\dot{x}_t = f(x_t, u_t)$

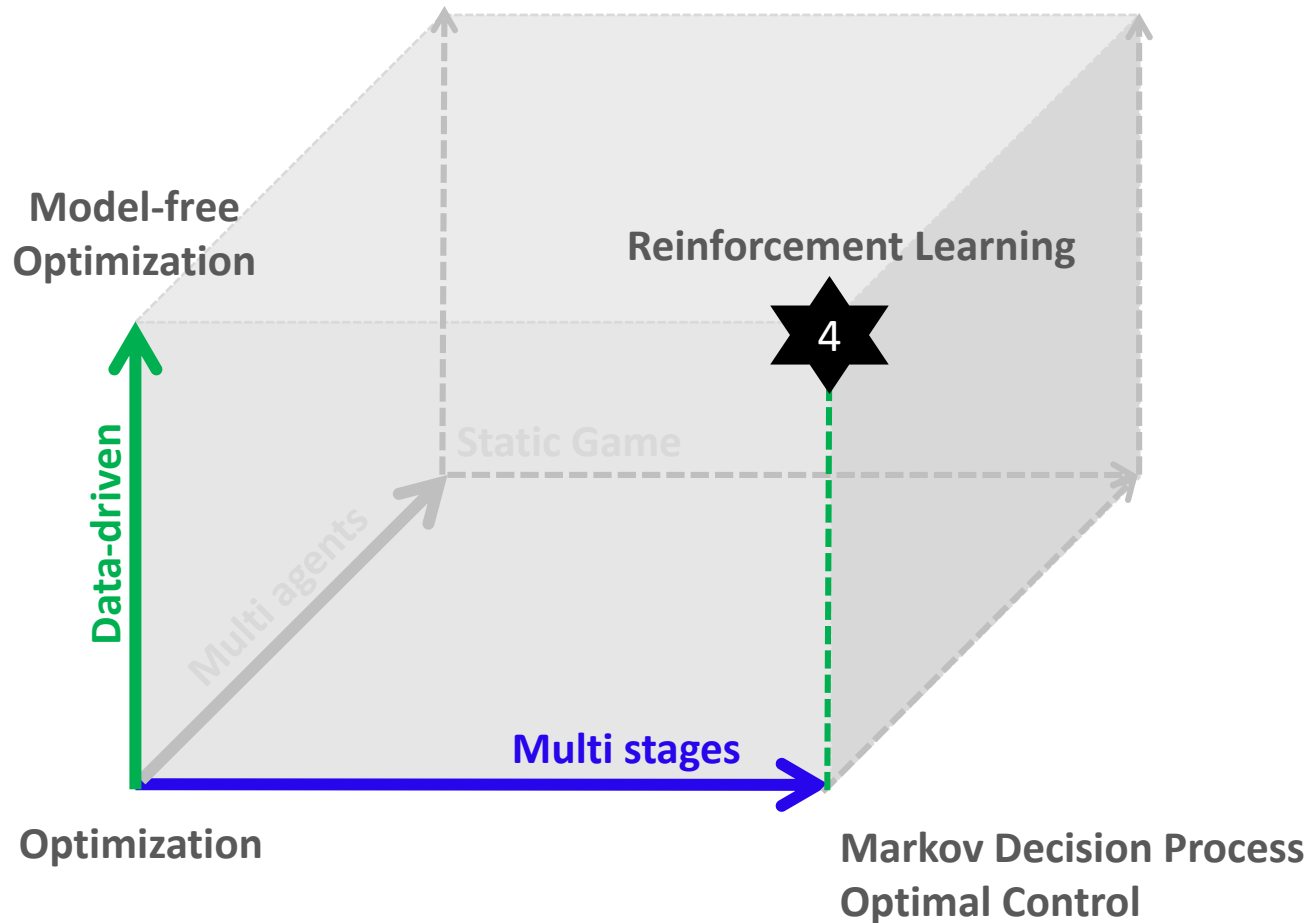
Data-Driven Dynamic Decision Making



	Single Agent	Multi Agent
Static	Static optimization	Static Game
Dynamic	Dynamic Optimization	Dynamic Game

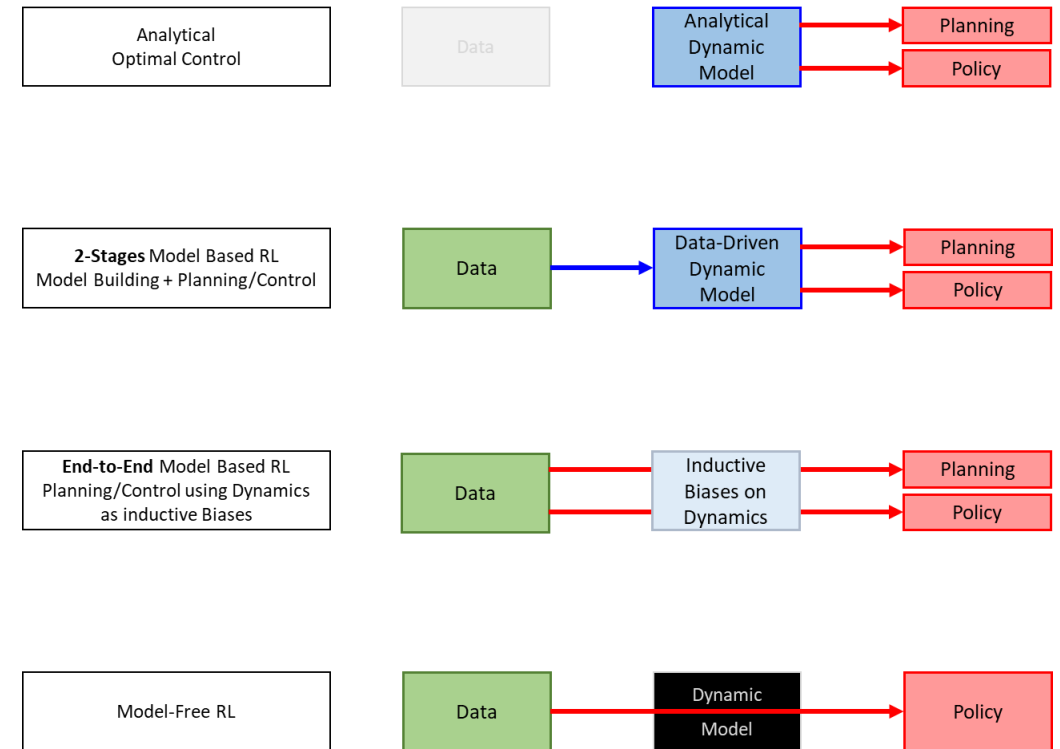
		Action space	
		Finite	Infinite
Time space	Model free	Value-based Reinforcement Learning	Policy-based Reinforcement Learning
	Discrete		
	Continuous		

Data-Driven Dynamic Decision Making



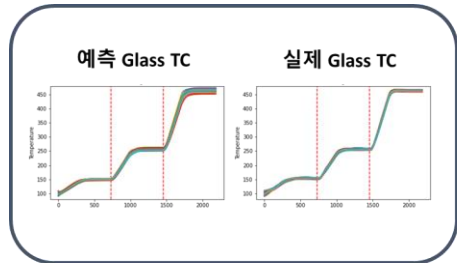
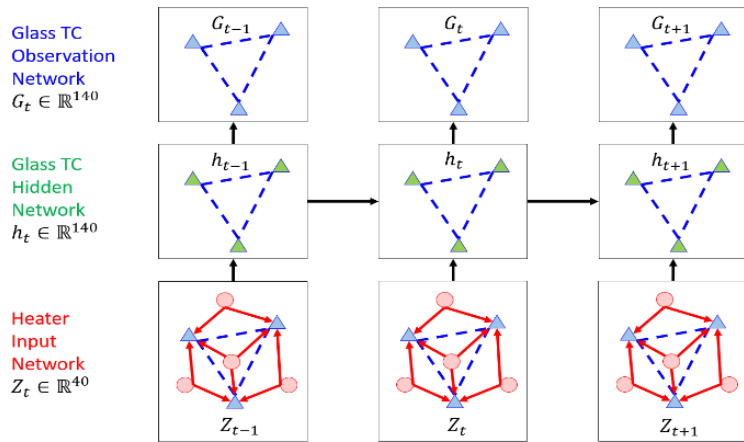
Extension Toward Practical RL

1. **Model Based RL**
2. Meta RL
3. Offline RL

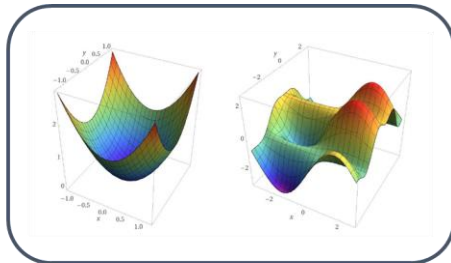


Example Application: Model Based RL for Semiconductor Furnace Heat Control

Input Convex Graph Recurrent Neural Network

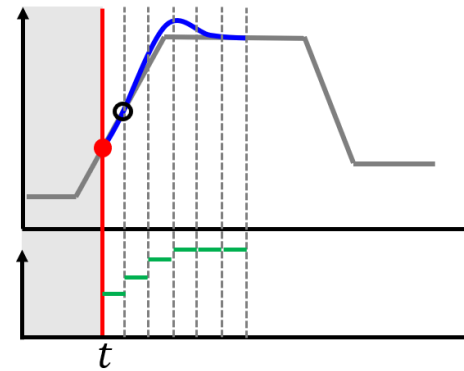


Model expressivity



Optimization solvability

Model Predictive Control

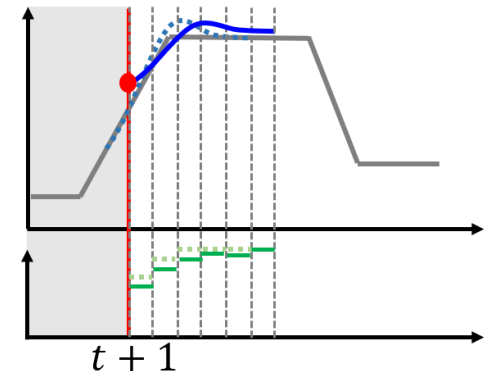


$$\min_{Z_t^{1:40}} \frac{1}{W} \sum_{\tau=t}^W \sum_{i=1}^{140} (\hat{G}_{\tau+1}^i - R_{\tau+1}^i)^2$$

s. t. $\hat{G}_{\tau+1}^{1:140} = f(G_{\tau}^{1:140}, Z_{\tau}^{1:40})$

Execute the first action:

Observe
 $G_{t+1}^{1:140}$



$$\min_{Z_{t+1:t+W}^{1:40}} \frac{1}{W+1} \sum_{\tau=t+1}^{W+1} \sum_{i=1}^{140} (\hat{G}_{\tau+1}^i - R_{\tau+1}^i)^2$$

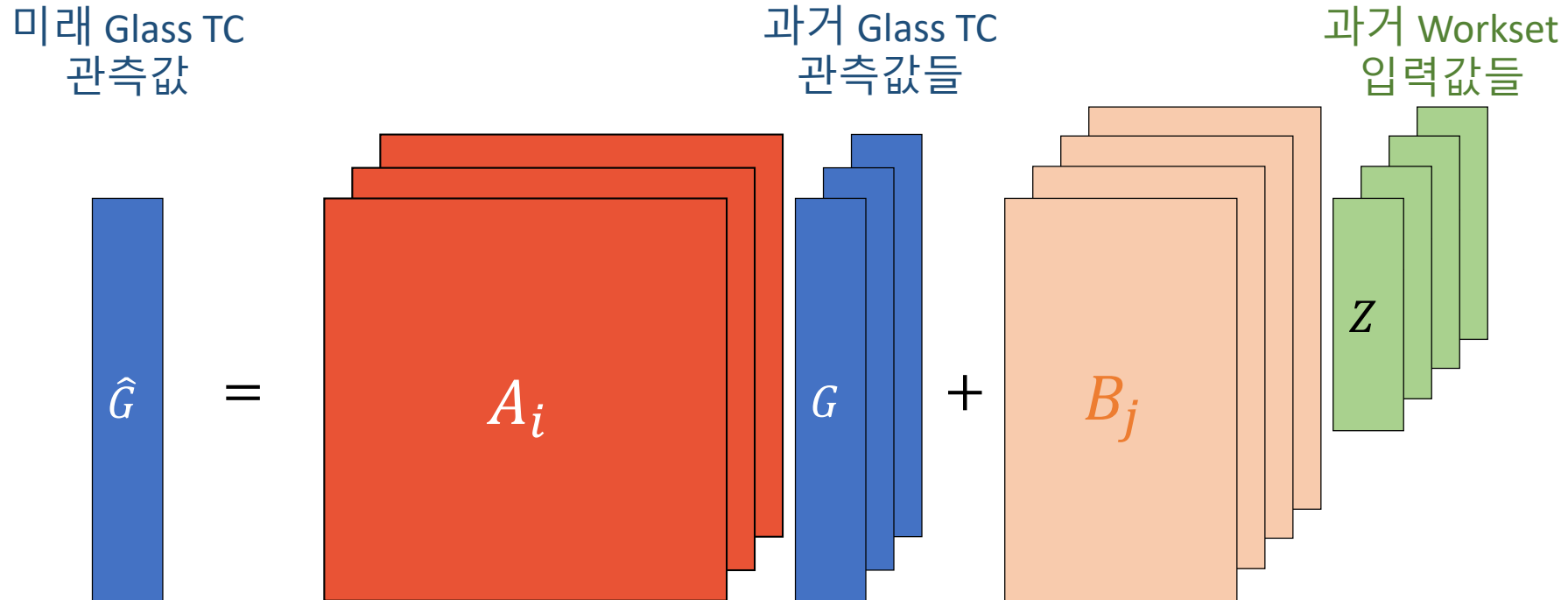
s. t. $\hat{G}_{\tau+1}^{1:140} = f(G_{\tau}^{1:140}, Z_{\tau}^{1:40})$

...

GRNN based Heat Transfer Model

Furnace Temperature Control
using the GRNN based Heat Transfer Model

Dynamic Model Design: Multi-step Linear Model



$$\hat{G}_{t+1}^{1:140} = \sum_{i=0}^{I-1} A_i G_{t-i}^{1:140} + \sum_{j=0}^{J-1} B_j Z_{t-j}^{1:40}$$

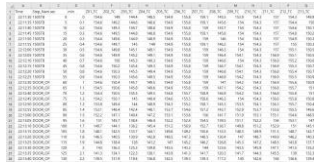
$A_i \in \mathbb{R}^{140 \times 140}$

$B_j \in \mathbb{R}^{140 \times 40}$

- 현재의 Glass TC와 Work set 뿐만 아니라 과거 값도 사용 → Furnace의 관성 모델링 가능
- Model expressivity와 Optimization solvability 를 적절히 밸런싱.

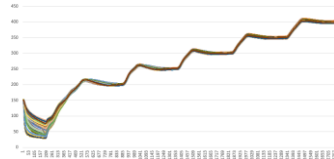
Dynamic Model Fitting

학습 데이터



기존 운영 Log

+



추가 실험 Log

다양한 학습 기법 사용

☒ **Multi-step prediction error**

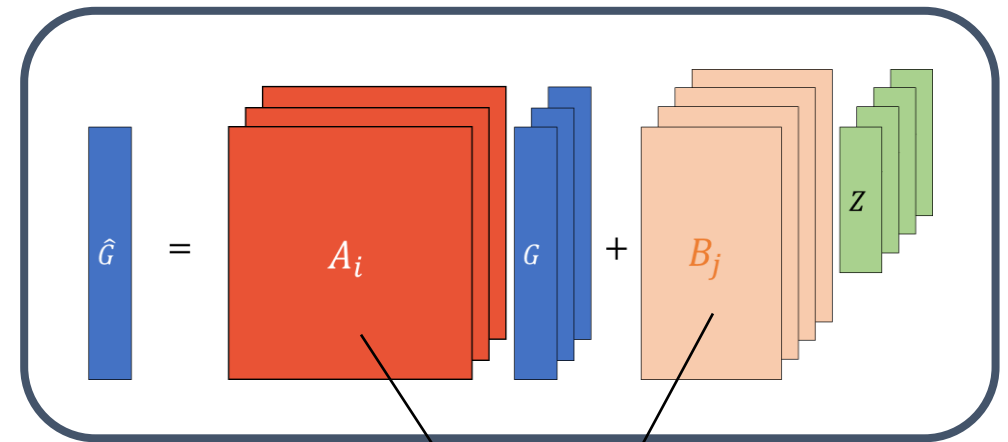
☐ Data preprocessing

☐ Parameter initialization w/ prior knowledge

☐ Hyperparameter tuning



예측 모델



학습을 해야하는 파라미터들

Evaluating the Predictive Performance of the Trained Model

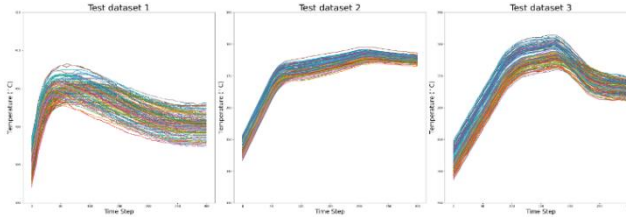
Hyperparameter 에 따른 예측 정확도 변화



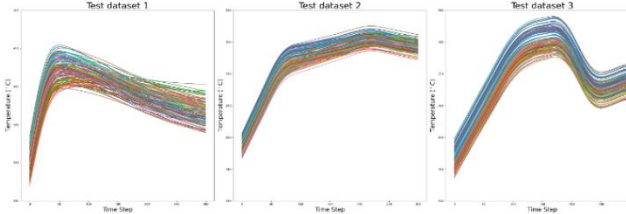
- 최적의 Hyper-parameter 탐색
- 100 step 예측 오차: **평균 2.03°C**

Overshoot 구간의 예측 정확도

실제 Glass TC



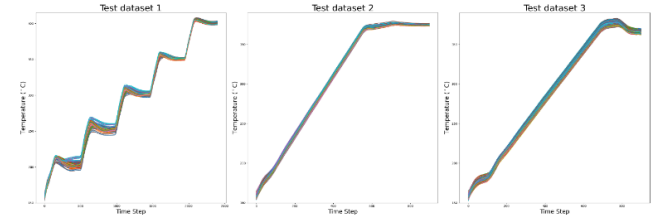
예측 Glass TC



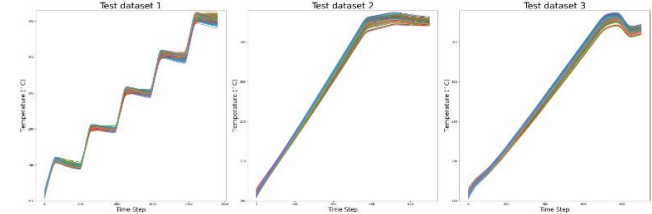
- Overshoot 구간의 300 step 예측
- **Overshoot 예측 가능**

장기 예측 안정성

실제 Glass TC

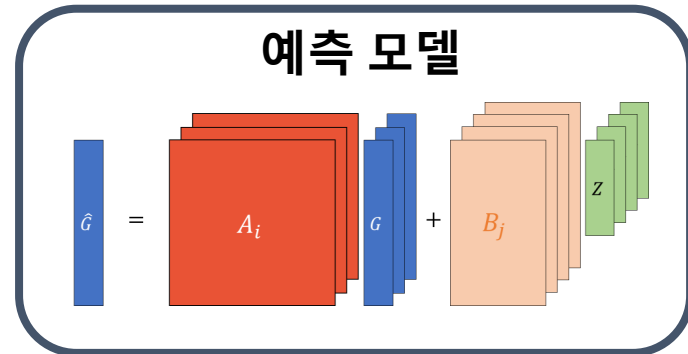


예측 Glass TC



- 900 step 예측 후 실제 값과 비교
- 장시간 예측 시 **예측 안정성 확인**

Optimizing Control Variables using Model Predictive Control



최적화 문제 Set-up

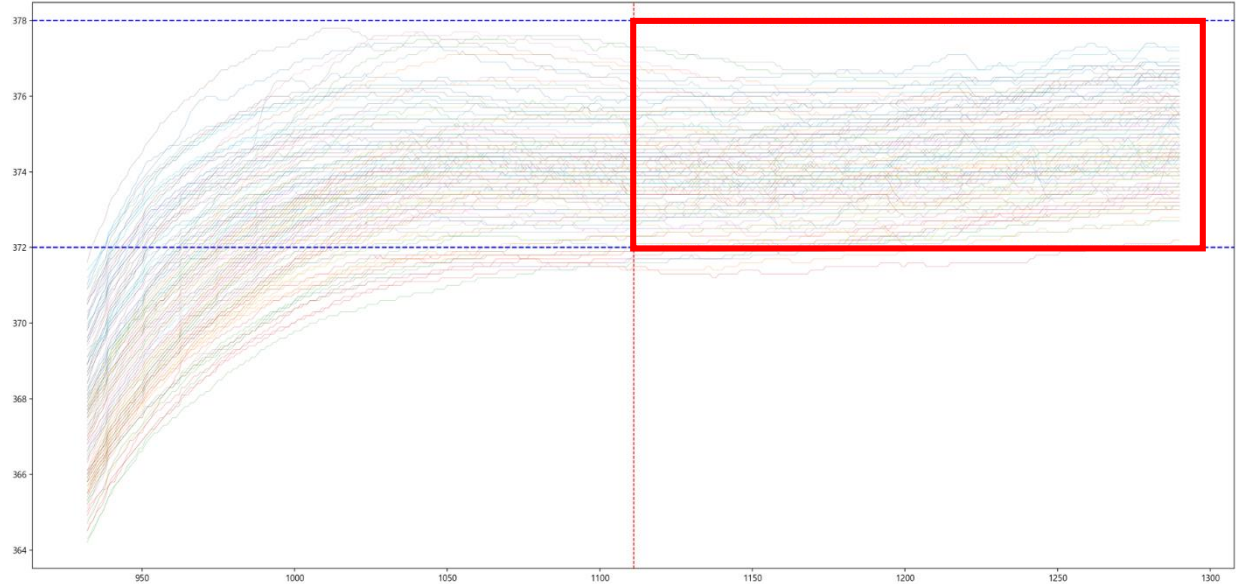
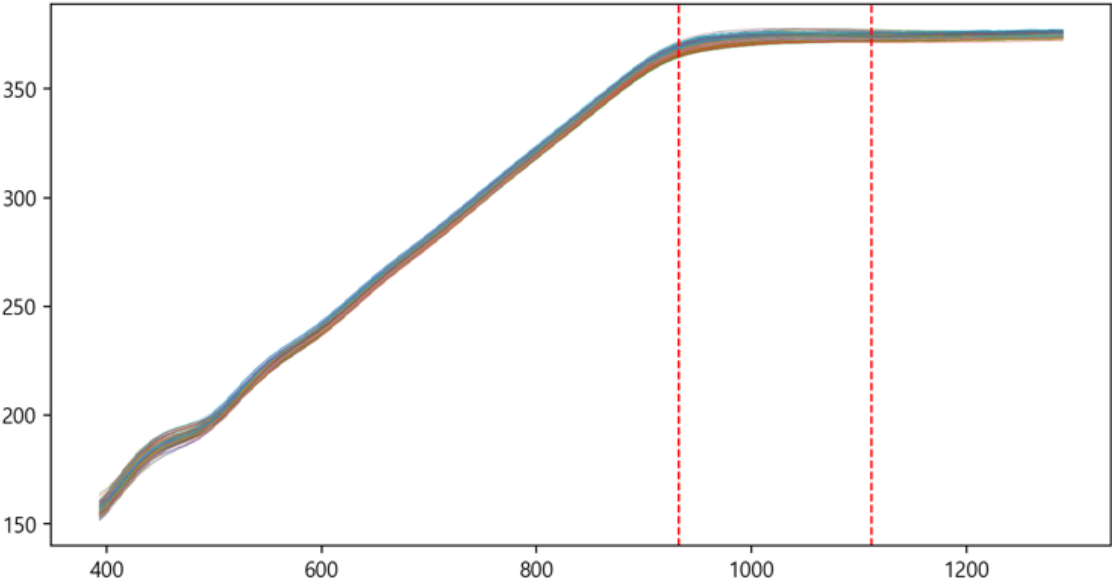
$$\min_{\substack{Z_{t:t+H-1}^{1:40}}} \frac{1}{H} \sum_{\tau=t}^{t+H-1} \sum_{i=1}^{140} (\hat{G}_{\tau+1}^i - R_{\tau+1}^i)^2$$
$$\text{s.t. } \hat{G}_{\tau+1}^{1:140} = f(G_{\tau}^{1:140}, Z_{\tau}^{1:40})$$

최적 Work set
 $\{Z_{\tau}^*\}_{\tau=t}^{t+H-1}$

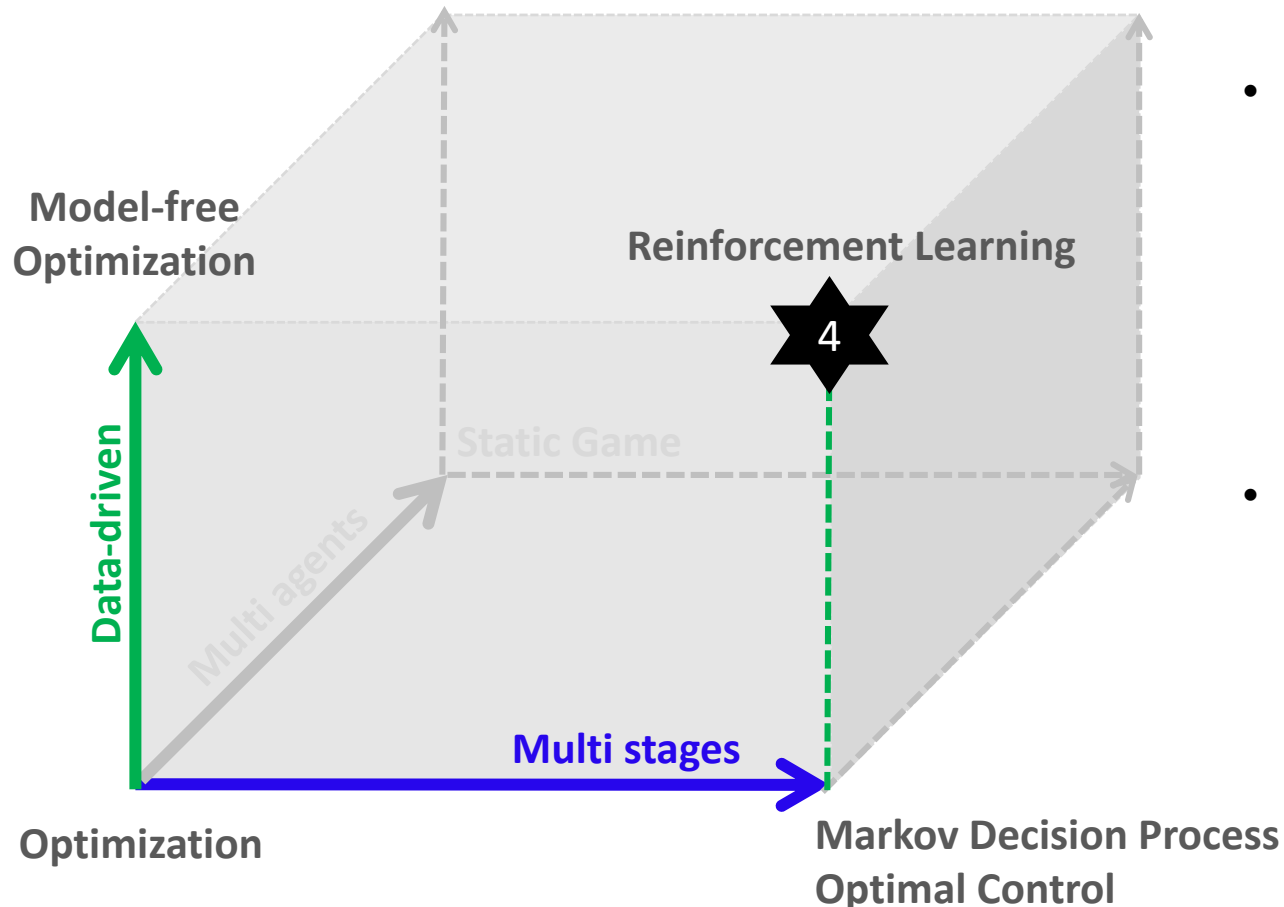
MPC Solver 개발/최적화/연동

- GPU acceleration
- Regularization
- Initial solution search
- Active Workset bound
- Hyperparameter tuning
- Time synchronization

Results



Data-Driven Dynamic Decision Making



Extension Toward Practical RL

1. Model Based RL

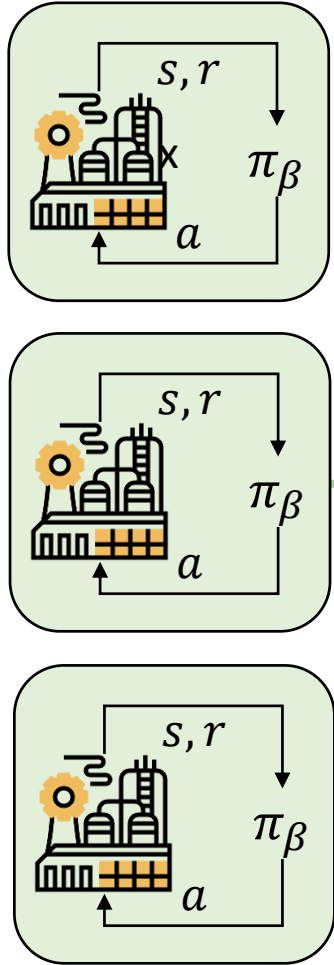
2. Meta RL

3. Offline RL

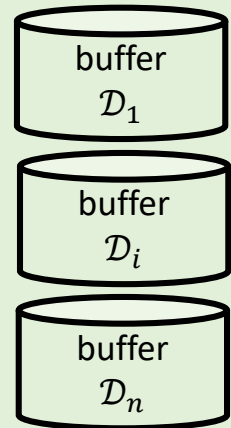
- Why Meta RL?
 - In industrial applications, the dynamics and objectives often change frequently
 - a single policy should be able to solve various tasks with different dynamics or objectives while characterizing a target task and properly adapting to it.
- Why Offline RL?
 - Industrial systems typically do not have simulators, and it can be prohibitive to learn a policy by directly interacting with the real system.
 - Deriving a policy using previously collected data (offline data) is preferable.

Practical Data-Driven Dynamic Decision Making

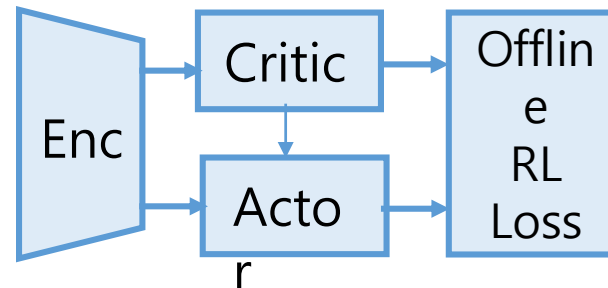
Training Tasks



Offline Data

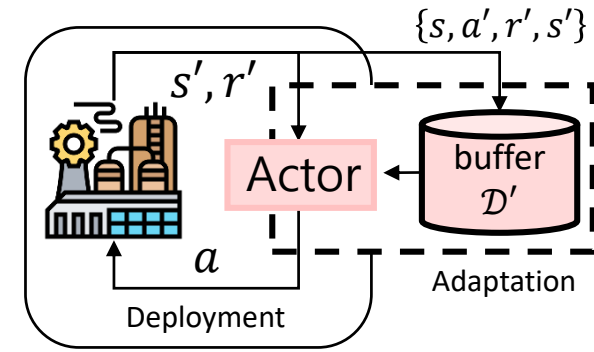


- Offline RL
- Meta RL
- Multi-agent RL



**General Controller based on
Fundamental Decision Making Modules
(Diffusion model, Transformer, etc.)**

Target Task



Collect Operation Data under Various Tasks

Offline Meta Distributed Policy Learning

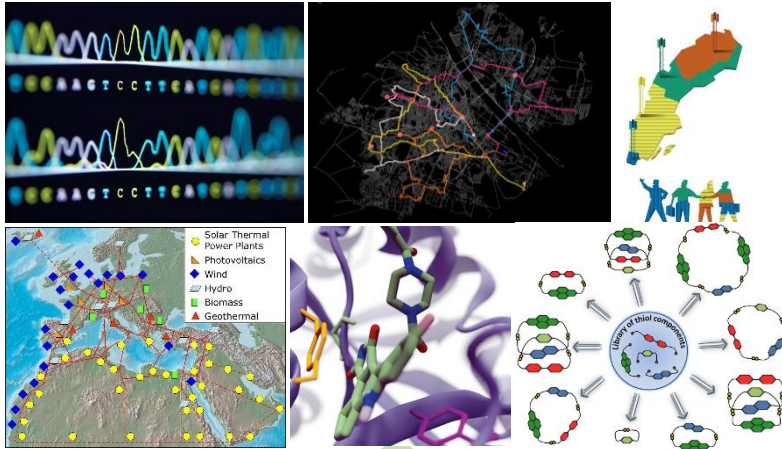
Online Adaptation



Neural Combinatorial Optimization

Motivation for NCO Research

Real-world task



Formal task (model)

$$\begin{aligned} \min & \sum_{i=0}^n \sum_{j \neq i, j=0}^n c_{ij} x_{ij} \\ 0 \leq x_{ij} & \leq 1 & i, j = 0, \dots, n \\ x_{ij} & \text{ integer} & i, j = 0, \dots, n \\ \sum_{i=0, i \neq j}^n x_{ij} & = 1 & j = 0, \dots, n \\ \sum_{j=0, j \neq i}^n x_{ij} & = 1 & i = 1, \dots, n \\ u_i - u_j + n x_{ij} & \leq n - 1 & 1 \leq i \neq j \leq n. \end{aligned}$$

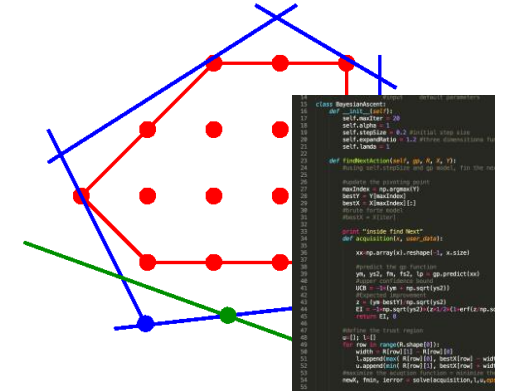
Modeling



Solving



Algorithm (program)

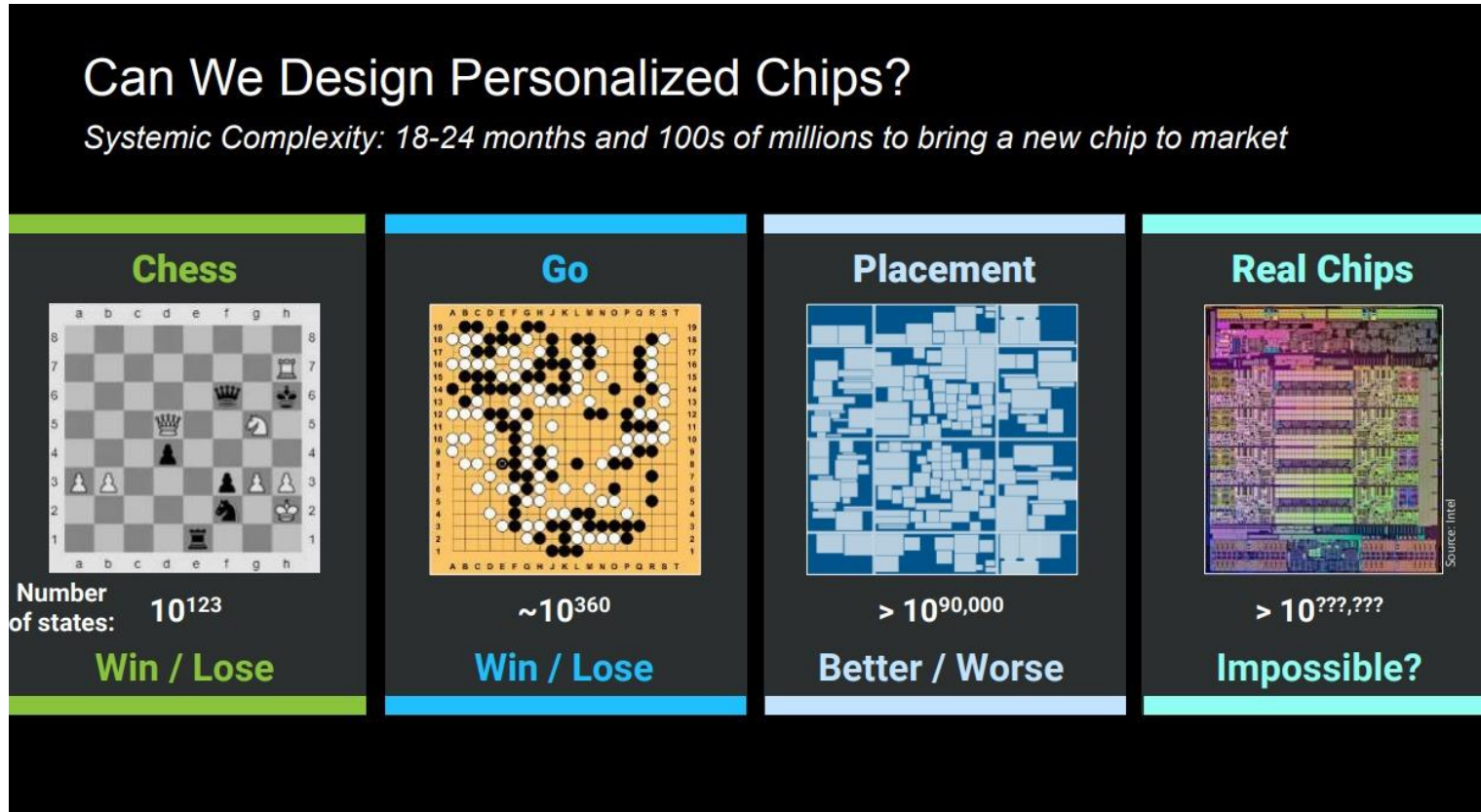


Solution

Is this solution is good for the target system?

“While maintaining the core structural characteristics and complexity of the real problem, the algorithm is developed in the mathematical domain by abstractly expressing the specific content of the problem”

Motivation for NCO Research

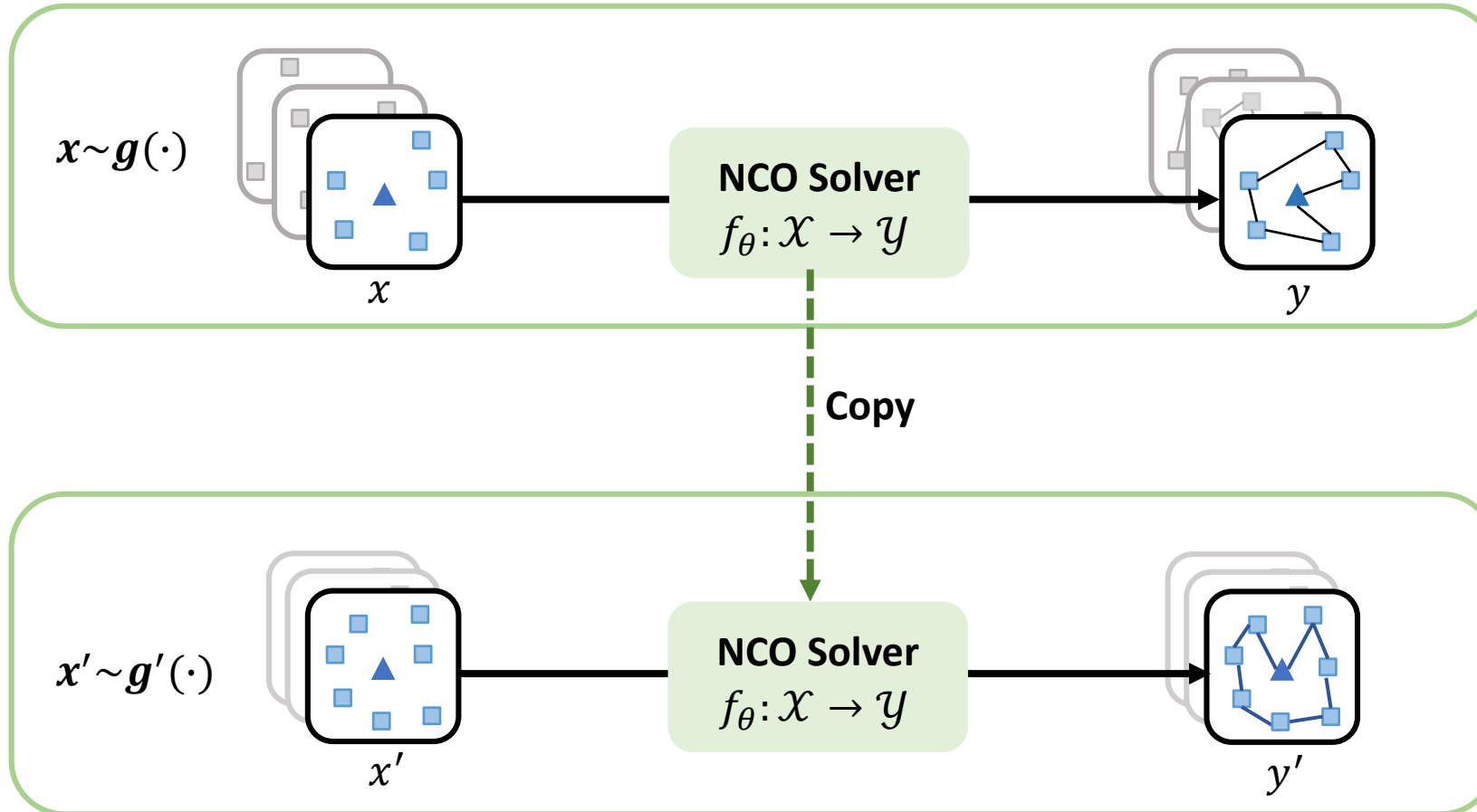


https://www.deccanherald.com/sites/dh/files/article_images/hc33%202.jpg

- Combinatorial optimizations are typically NP hard problems
 - Typical AI algorithms cannot effectively search over the combinatorial solution space
 - An effective (1) representation learning, (2) architecture design, (3) learning method are required.

NCO Problem Definition

- The objective of NCO is to learn a “Solver” to generate a good solution to a given target CO problem



NCO solver Design:

- Representation
 - Graph
 - Coordinate
- Decision making structure
 - Heuristics Based
 - Math Programming based
- Learning Methods
 - Imitation Learning
 - Reinforcement Learning

Performance Metrics:

- Generalization
- Optimality
- Speed

- Learning a NCO solver is solving **multi-task learning**:
→ **Meta-Learning** techniques is inherently need to be considered

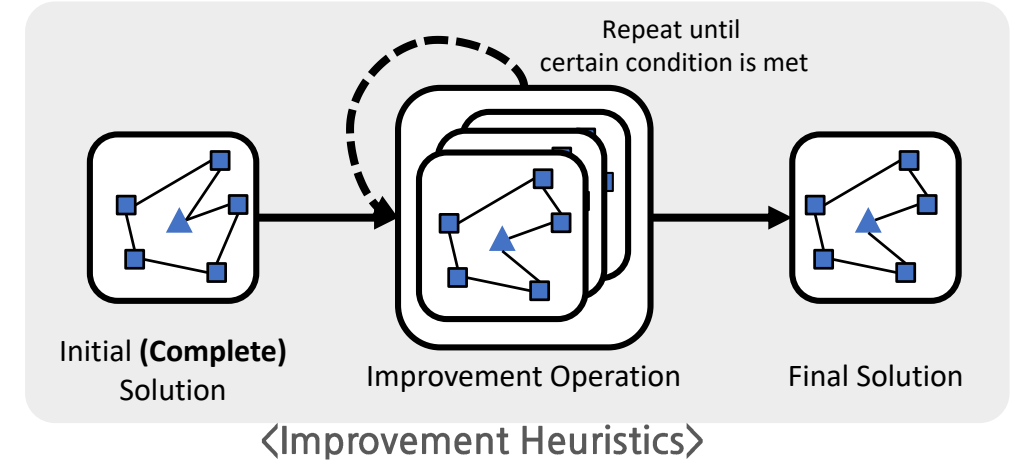
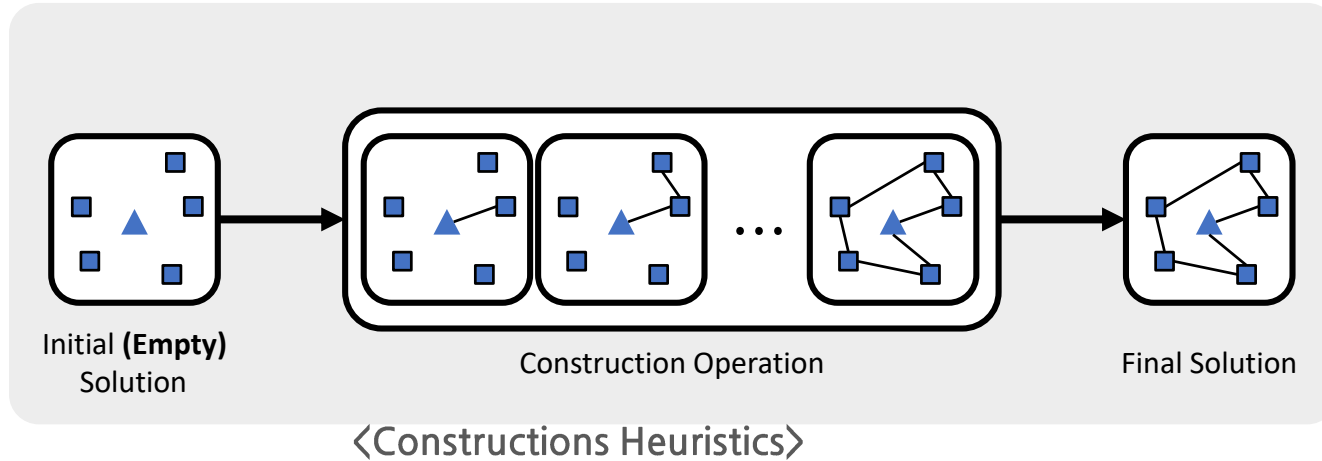
Classification for NCO Research

NCO algorithms can be categorized depending on:

- Problem Formulation:
 - Heuristic Based Methods
 - ✓ Constructive heuristic (Auto Regressive)
 - ✓ Improvement heuristics
 - Mathematical Programming Based Methods
 - ✓ Learning to Configure
 - ✓ ML-Augmented
- Learning Method
 - Imitation Learning
 - Reinforcement Learning

Classification for NCO Research

Heuristics Based Methods: Constructive heuristics vs. Improvement Heuristics (vs. Hybrid heuristics)



<Constructions Heuristics>

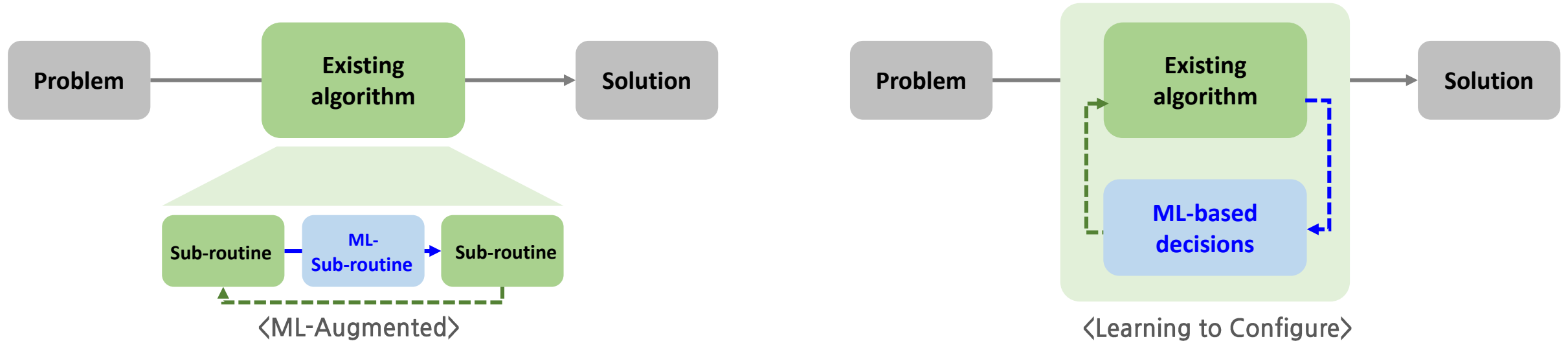
- Greedy Algorithm, Insertion Algorithm, Minimum Spanning Tree Algorithm
- It has the fastest execution speed, but the performance is poor. Mainly used as an internal algorithm for more complex algorithms
- When combined with reinforcement learning methods, it can improve both execution speed and performance.

<Improvement Heuristics>

- **Neighborhood Search Algorithms:** k-opt exchange, Lin-Kernighan-Helsgaun, Cross-exchange, Simulated Annealing
- **Advanced Meta-Heuristics Algorithms:** Adaptive Large Neighborhood Search (ALNS), Hybrid Genetic Algorithm
- New algorithms can be developed by combining multiple algorithms depending on the problem type

Classification for NCO Research

Mathematical Programming Based Methods: ML-Augmented vs. Learning to Configure



Based on Branch-and-Bound, Column Generation, Cutting Plane, Branch-Pricing-Cut

<ML-Augmented>

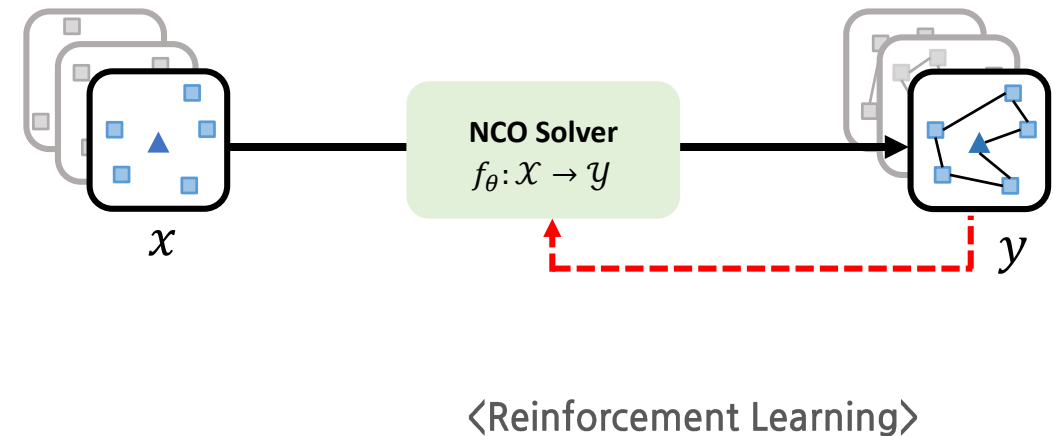
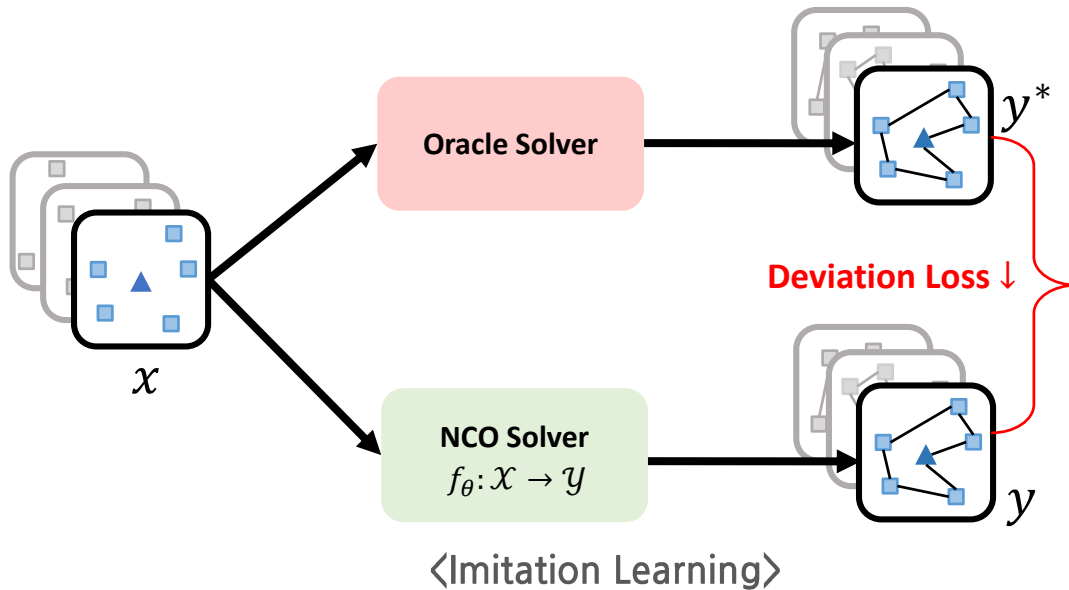
- ML-based sub-routine replace sub-routine with high computational complexity in existing algorithms Theoretical analysis is easy, but it requires expert knowledge on existing algorithms, difficult to use commercial solvers, etc.

<Learning to Configure>

- ML component is designed to determine parameters of the existing algorithm for improving performance.
- Because the ML component operates outside of the underlying algorithms, it is easy to use a commercial solver, etc., and it is possible to combine various algorithms.

Classification for NCO Research

Learning Methods: Reinforcement Learning vs. Imitation Learning



<Imitation Learning: optimization for minimizing deviation>

- Pros: easy to learn, fast to train
- Cons: requires the expert solutions, tends to over-fit

<Reinforcement Learning: optimization for maximizing performance>

- Pros: labels (optimal solutions) is not required, inherent exploration $\rightarrow$ better generalization
- Cons: poor sample efficiency,



Q & A